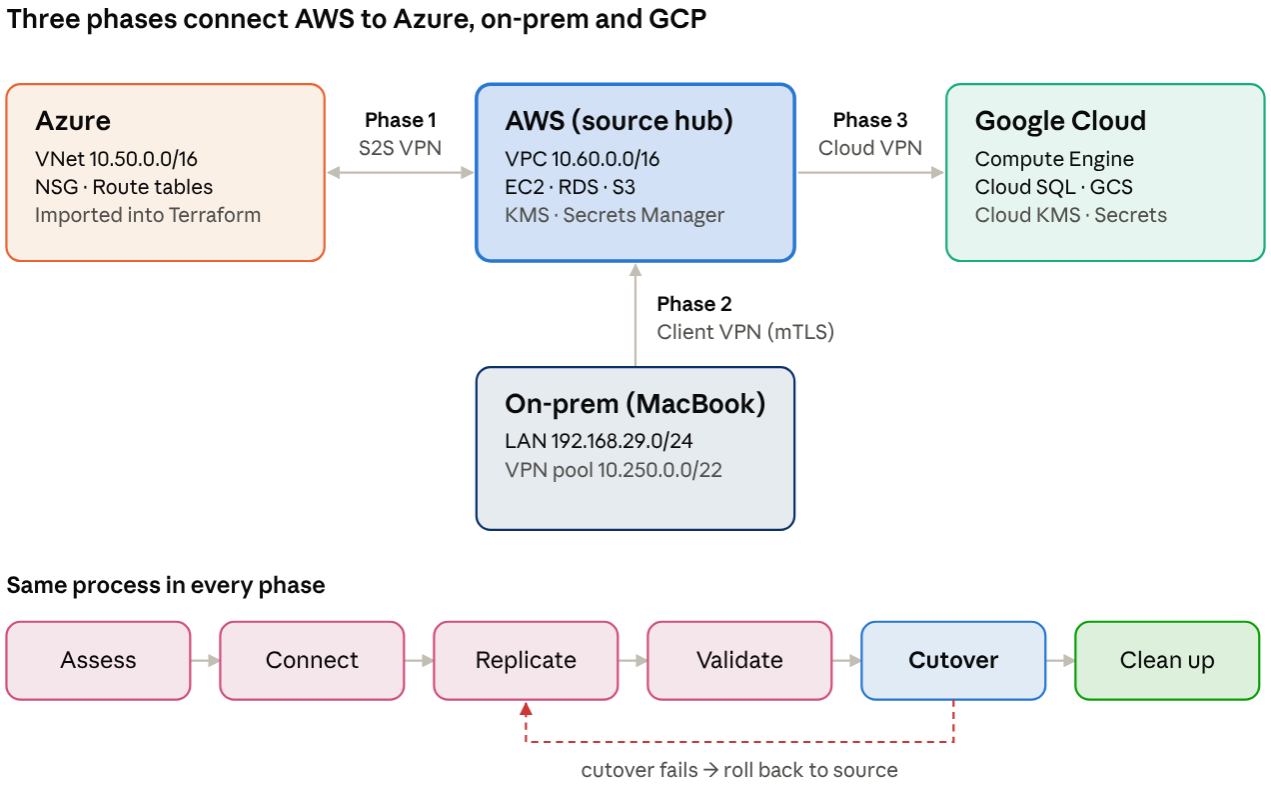
Enterprise Cloud Migration Project (Phase 1–3)

Oct 15, 2026 · @Dipesh Pawar

A hands-on, low-cost lab that migrates and connects workloads across AWS, Azure, GCP and on-prem, built small but run the way an enterprise migration team works.



Project at a glance · 3 phases, 1 process

AWS is the hub: Azure links in Phase 1, a laptop (on-prem) in Phase 2, and workloads move to GCP in Phase 3. Every phase follows the same six steps, with rollback ready at cutover.

Contents

1. Program overview and network plan
2. Phase 1 — AWS ↔ Azure: Azure foundation, Terraform import, AWS source environment, Site-to-Site VPN, migration roadmap
3. Phase 2 — On-prem (MacBook) → AWS: certificate authority, AWS Client VPN, user onboarding, AWS → Azure data moves, Entra SSO
4. Phase 3 — AWS → GCP: governance, AWS source build, GCP foundation, HA VPN + BGP, VM / database / storage / security migration, cutover and cleanup

5. Tech stack, learnings and LinkedIn post

> Security note: every password, private key, VPN pre-shared key and secret value has been removed from this document and from the screenshots. Values shown as `********` were redacted on purpose.

1. Program overview

The goal is a small, low-cost but enterprise-grade migration lab. The infrastructure is intentionally tiny; the architecture, method and documentation are enterprise-grade.

Golden rule: build small infrastructure, but think like an enterprise migration team. Create only the minimum resources needed to show the real engineering problem, document the production-scale approach, test the important pieces, and destroy the lab afterward.

1.1 What the lab covers

Area	What is practised
Cloud-to-cloud migration	AWS → Azure and AWS → GCP
Cloud-to-on-prem	Laptop (on-prem stand-in) → AWS over Client VPN
Network connectivity	VPC/VNet, subnets, route tables, SG/NSG, Site-to-Site and Client VPN, DNS, IP plan
Identity and federation	Microsoft Entra ID → AWS IAM Identity Center (SAML/SCIM), GCP IAM, service accounts
Hardcoded IP handling	Preserve IP, NAT/proxy, DNS abstraction, config change
Data migration	RDS PostgreSQL → Azure PostgreSQL and Cloud SQL; S3 → Azure Blob and Cloud Storage
VM migration	EC2 → Azure VM; EC2 → Compute Engine (Migrate to VMs)
Kubernetes migration	EKS → AKS / GKE pattern (Namespace, Deployment, Service, ConfigMap, Secret)
Secrets and keys	Secrets Manager → Key Vault / Secret Manager; KMS → Cloud KMS
Backup / DR	AWS Backup, vaults, restore tests, replication

Area	What is practised
Cutover	Minimal downtime, RPO/RTO, replication lag, freeze window, smoke tests, rollback
Infrastructure as Code	Console first, then Terraform import into modules

1.2 The three parts

1. **Part 1 — AWS ↔ Azure:** build Azure, import into Terraform, build the AWS source, connect over Site-to-Site VPN, then migrate.
2. **Part 2 — AWS ↔ On-prem:** connect a simulated on-prem site (a MacBook) to AWS with Client VPN; real data centres would use Site-to-Site VPN or Direct Connect.
3. **Part 3 — AWS ↔ GCP:** repeat the method for GCP with managed migration services (Migrate to VMs, Database Migration Service, Storage Transfer Service).

Order followed: Azure first → Terraform import → AWS minimal environment → connectivity → identity → migration → hardcoded-IP handling → backup/DR → cutover → rollback → Terraform → cleanup. The same method is then repeated for GCP and on-prem.

1.3 Network plan (no overlapping ranges)

Network	CIDR	Region	Purpose
AWS VPC (vpc-migration)	10.60.0.0/16	us-east-1	Source estate: EC2, RDS, S3 access
Azure VNet (vnet-migration)	10.50.0.0/16	Central India	Azure target network
GCP VPC (aws-gcp-migration-vpc)	10.70.0.0/16	us-central1	GCP target network
MacBook LAN	192.168.29.0/24	—	On-prem stand-in
Client VPN pool	10.250.0.0/22	—	Addresses for VPN clients

Planning non-overlapping ranges first is what makes every later VPN, route and firewall rule work without NAT tricks.

1.4 Cost control

- Target AWS spend for the lab: about ₹200 or less (a target, not a guarantee).
- Budgets and alerts in AWS and GCP; alerts are not spending caps, so usage is checked manually.

- Small instance classes, no NAT Gateway unless needed, stop EC2/RDS when idle, destroy everything after the rollback window.

2. Phase 1 — AWS ↔ Azure migration

Phase 1 builds Azure from scratch, brings it under Terraform, builds a minimal AWS source estate, and links the two clouds with a Site-to-Site IPsec VPN. Each part was planned to take about 5 hours.

2.1 Service mapping (AWS → Azure)

AWS	Azure
EC2	Virtual Machine
RDS (PostgreSQL)	Azure Database for PostgreSQL
EKS	AKS
S3	Blob Storage
Secrets Manager	Key Vault
IAM / Identity	Microsoft Entra ID
VPC	VNet
Security Group	Network Security Group (NSG)
Virtual Private Gateway + Customer Gateway	VPN Gateway + Local Network Gateway

2.2 The 12 internal sub-phases

#	Sub-phase	What happens
1	Azure foundation	Resource group, VNet, subnets, NSGs, route tables, storage, Key Vault, Entra ID — created in the console, no AWS dependency
2	Azure → Terraform	Console create → validate → `terraform import` → code → `plan` shows no unexpected changes
3	AWS minimal foundation	VPC, subnets, route tables, SGs, IGW, small EC2, small RDS, S3, Secrets Manager, backup config

#	Sub-phase	What happens
4	AWS ↔ Azure connectivity	AWS VPN → internet → Azure VPN Gateway; validate both directions, routing, rules, ports, DNS
5	Identity and access	Entra ID → SAML/SCIM → AWS IAM Identity Center → accounts; users, groups, permission sets, SSO, least privilege
6	Workload migration	EC2 → VM, RDS → Azure DB, S3 → Blob, Secrets → Key Vault, EKS → AKS pattern
7	Hardcoded IP and legacy apps	Find hardcoded IPs, hostnames, ports, allow-lists, certificates; choose a strategy
8	Backup, DR, data protection	Backup → verify → replicate → migrate; AWS Backup, vaults, restore, validation
9	Live application cutover	Discovery → backup + replication → validation → controlled cutover → continue or roll back
10	Troubleshooting and validation	Network, application and data checks done on purpose
11	Terraform finalization	Console → import → Terraform → plan → apply → validate, clean module layout
12	Cleanup	Destroy lab resources after the rollback window, verify billing

2.3 Phase 1 target design

1. Azure: `rg-migration-lab` → `vnet-migration` 10.50.0.0/16 → GatewaySubnet 10.50.0.0/24, ApplicationSubnet 10.50.1.0/24 (NSG + route table), DatabaseSubnet 10.50.2.0/24 (NSG + route table).
2. AWS: `vpc-migration` 10.60.0.0/16 → PublicSubnet 10.60.0.0/24, AppSubnet 10.60.1.0/24, DatabaseSubnet 10.60.2.0/24 (us-east-1b), DatabaseSubnet2 10.60.3.0/24 (us-east-1a).
3. Link: AWS Virtual Private Gateway + Customer Gateway ⇄ IPsec ⇄ Azure VPN Gateway + Local Network Gateway, with private routing both ways.

2.4 Azure foundation — step by step

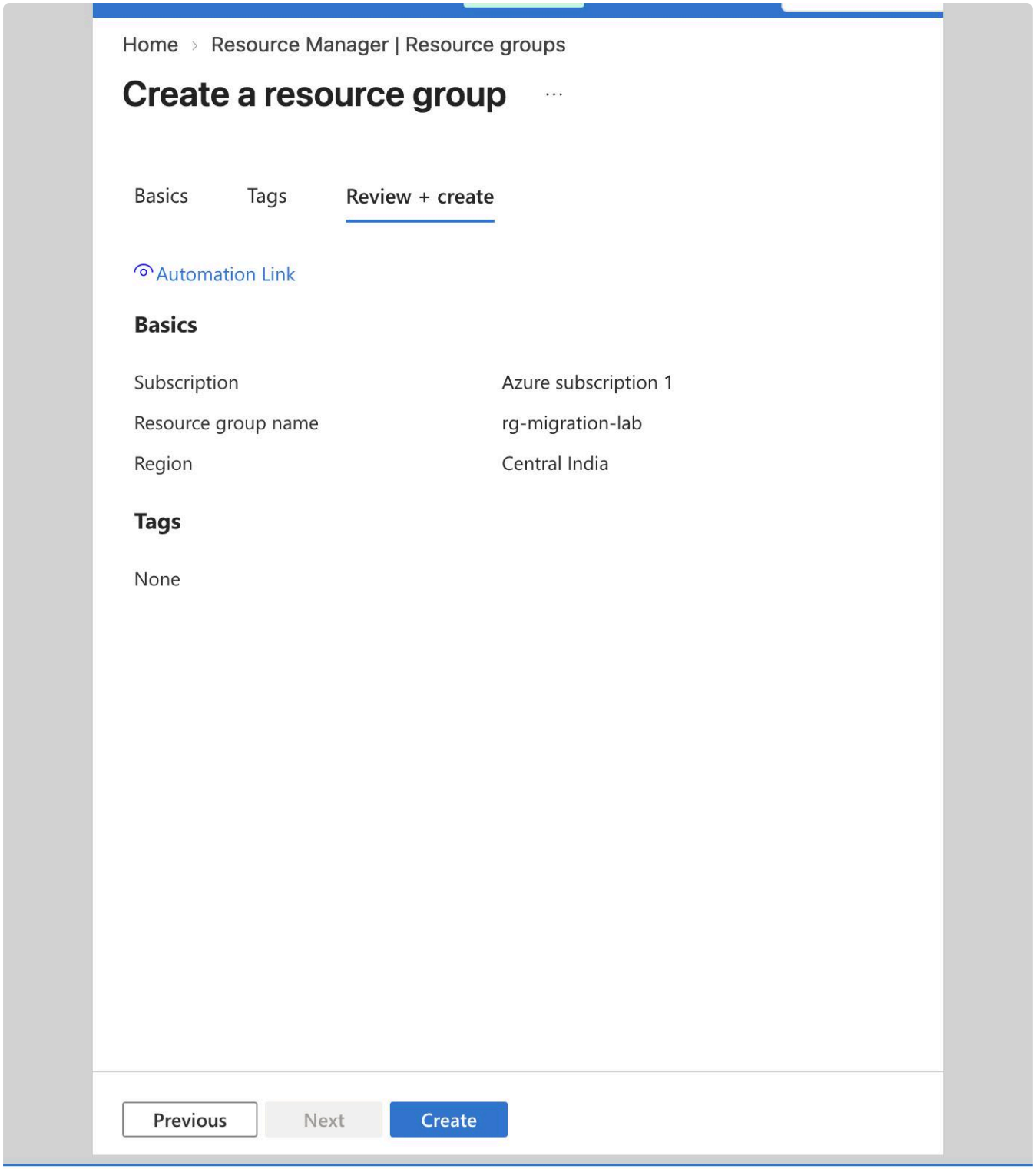
Everything below was created in the Azure Portal first, in region Central India, with no dependency on AWS. The Terraform import comes after, in 2.5.

Step 1 — Create the resource group

1. Azure Portal → Resource groups → Create.
2. Subscription: Azure subscription 1. Name: `rg-migration-lab`. Region: Central India.
3. Review + create → Create.

Step 2 — Create the virtual network

1. Virtual networks → Create.
2. Resource group `rg-migration-lab`, name `vnet-migration`, region Central India.
3. IP addresses tab: address space 10.50.0.0/16 (65,536 addresses).
4. Edit the default subnet to become the first planned subnet; leave Bastion, Firewall and DDoS disabled to keep cost near zero.
5. Review + create → Create.



Home › Network foundation | Virtual networks

Create virtual network

Basics Security Address space Tags Review + create

networks. VNet is similar to a traditional network that you'd operate in your own data center, but brings with it additional benefits of Azure's infrastructure such as scale, availability, and isolation.

Learn more.

Project details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription * : Azure subscription 1

Resource group * : rg-migration-lab

Create new

Instance details

Virtual network name * : vnet-migration

Region * : (Asia Pacific) Central India

Deploy to an Azure Extended Zone

Previous | Next | Review + create

Home › Network foundation | Virtual networks

Create virtual network

Basics | Security | Address space | Tags | Review + create

Define the address space of your virtual network with one or more IPv4 or IPv6 address ranges. Create subnets to segment the virtual network address space into smaller ranges for use by your applications. When you deploy resources into a subnet, Azure assigns the resource an IP address from the subnet. Learn more

☐ Allocate using IP address pools. Learn more

+ Add a subnet

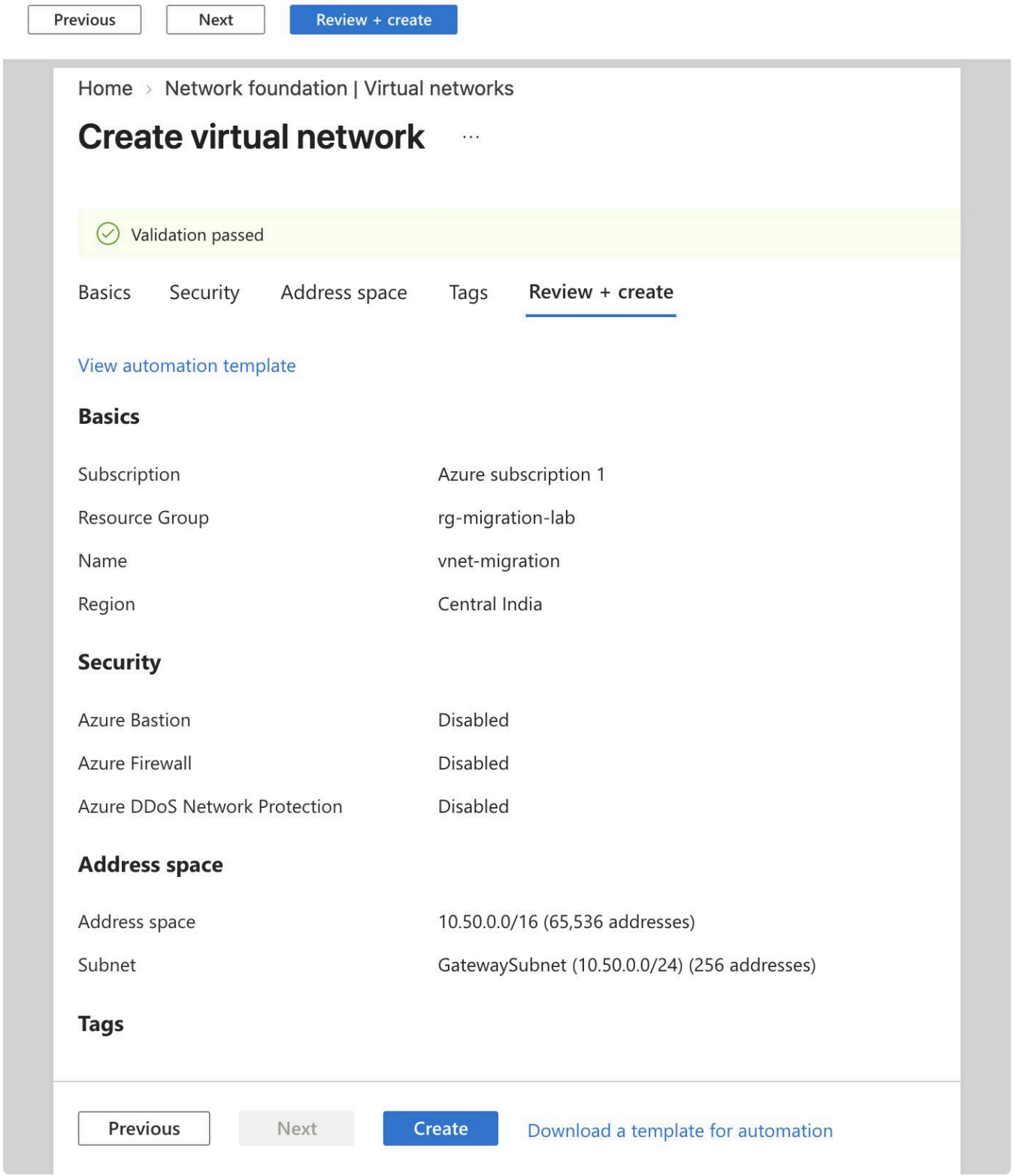
10.50.0.0/16 — Delete address space

Starting address	Size
10.50.0.0	/16
10.50.0.0 - 10.50.255.255	65,536 addresses

Subnets	IP address range	Size	NAT gateway
default	10.50.0.0 - 10.50.0.255	/24 (256 addresses)	-

Add IPv4 address space

Advertised address prefixes

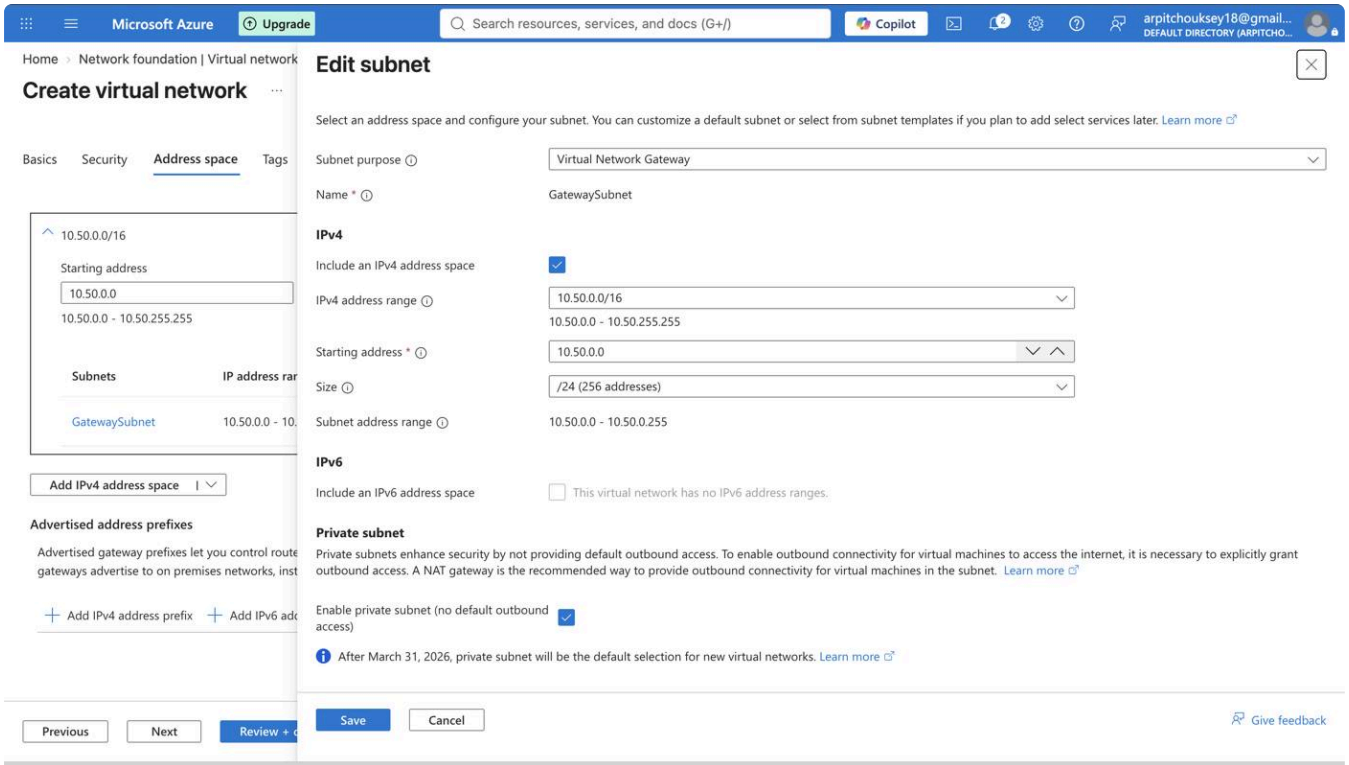


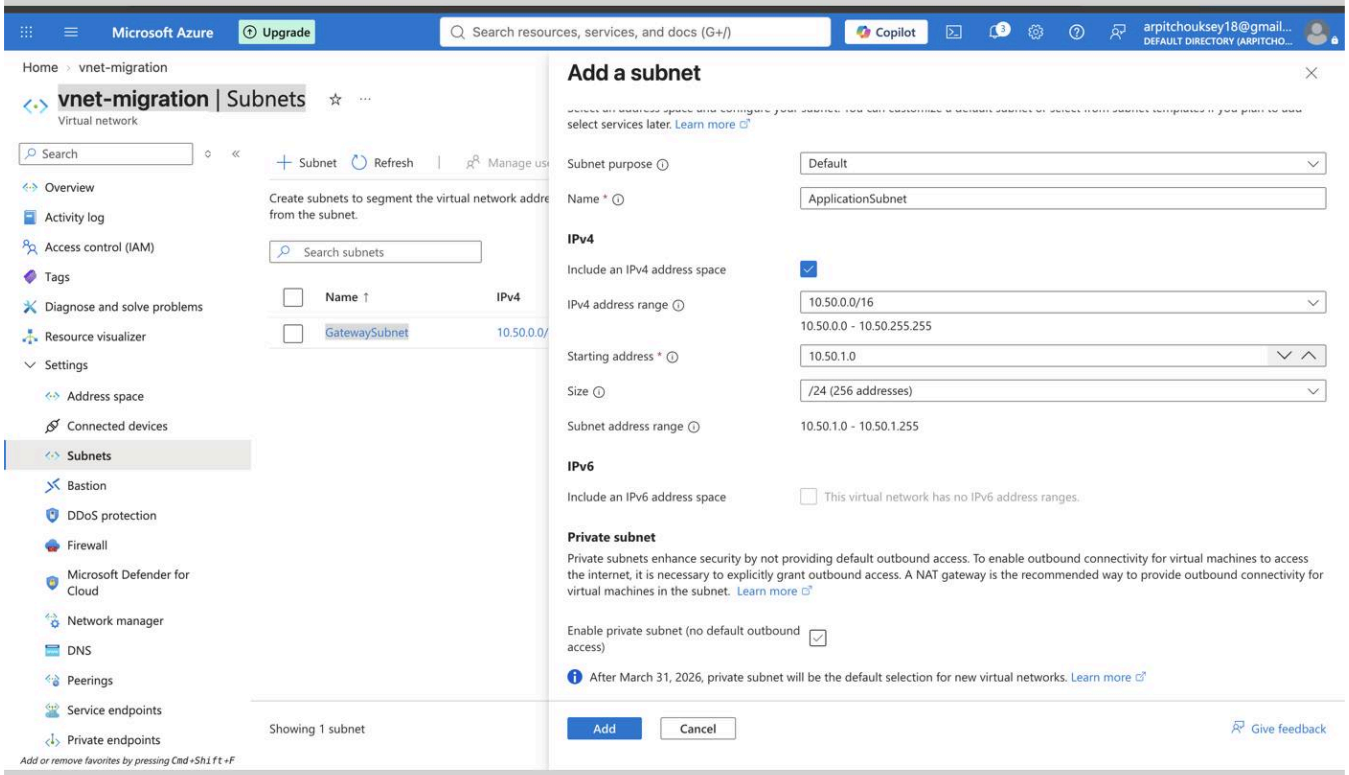
Step 3 — Create the three subnets

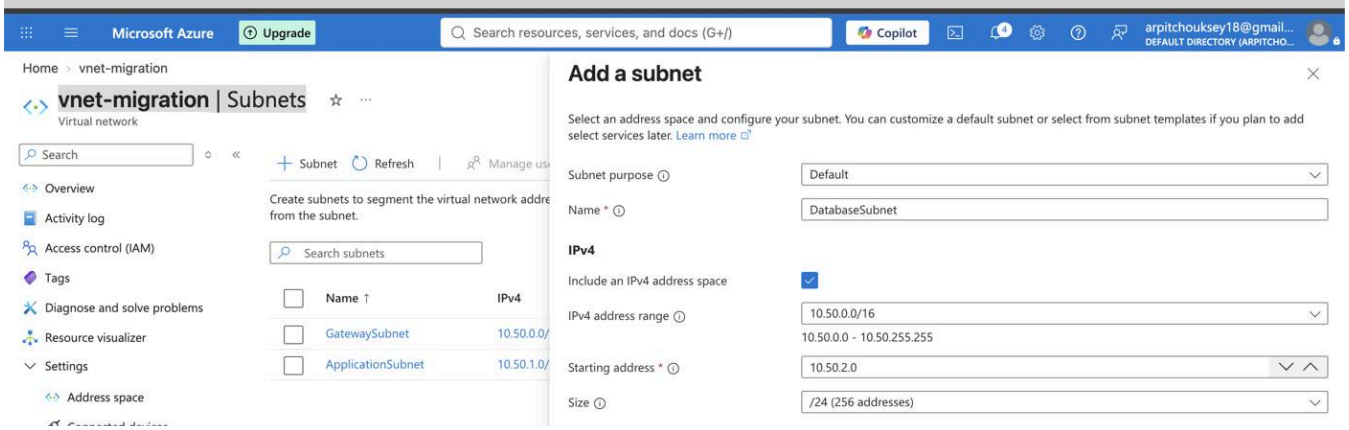
Subnet	CIDR	Purpose
GatewaySubnet	10.50.0.0/24	Reserved for the Azure VPN Gateway (name must be exactly `GatewaySubnet`)
ApplicationSubnet	10.50.1.0/24	Application VMs

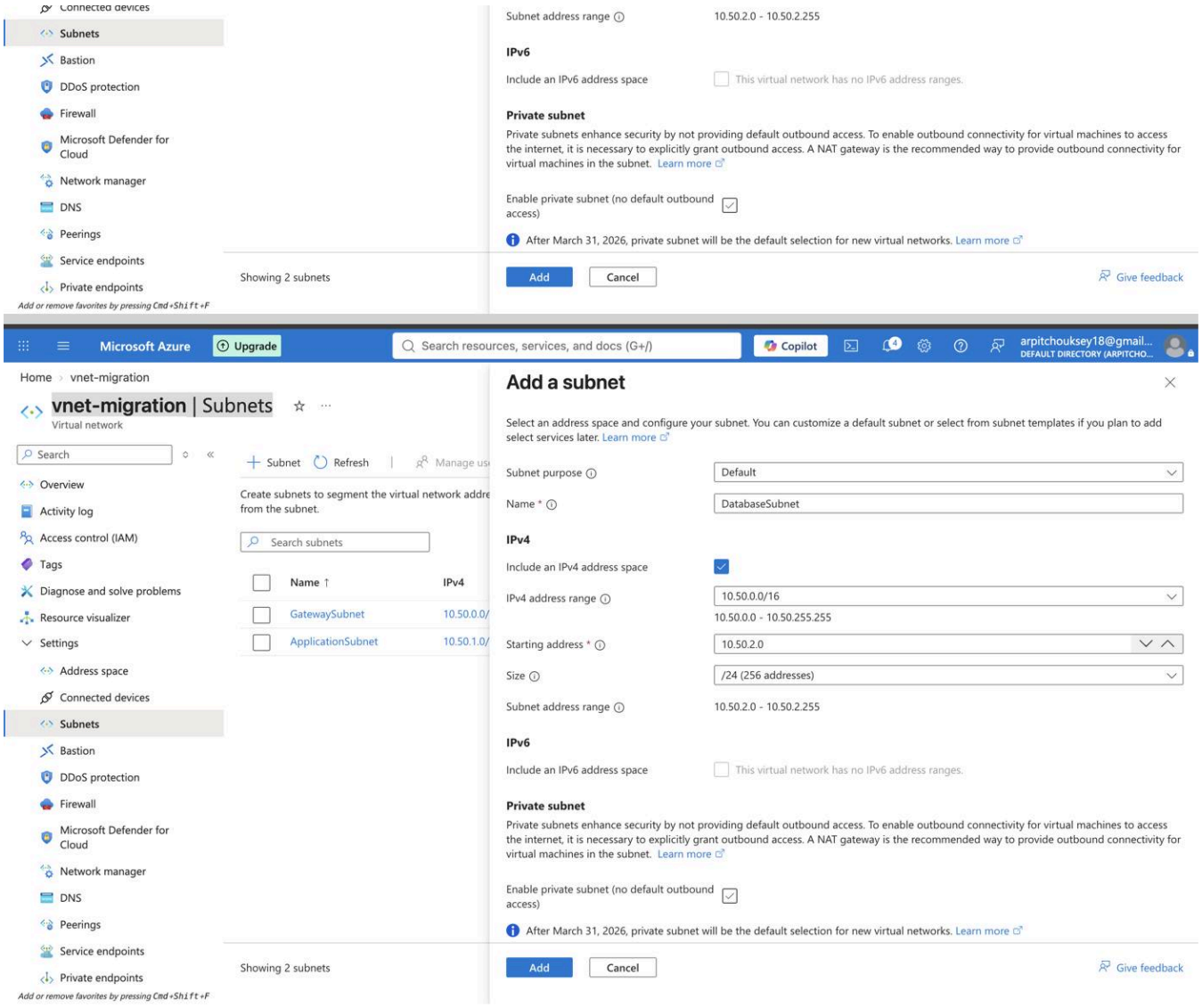
Subnet	CIDR	Purpose
DatabaseSubnet	10.50.2.0/24	Database tier

1. vnet-migration → Subnets → + Subnet.
2. Enter name and range, leave NAT gateway and service endpoints empty.
3. Private subnet (no default outbound access) enabled for app and DB subnets.
4. Add. Repeat for all three.


Microsoft Azure
Upgrade
Search resources, services, and docs (G+/)
Copilot
arpitchouksey18@gmail...
DEFAULT DIRECTORY (ARPITCHO...
Home › Network foundation | Virtual network
Create virtual network
Basics
Security
Address space
Tags
10.50.0.0/16
Starting address
10.50.0.0
10.50.0.0 - 10.50.255.255
Subnets
IP address rar
GatewaySubnet
10.50.0.0 - 10.
Add IPv4 address space
Advertised address prefixes
Advertised gateway prefixes let you control route
gateways advertise to on premises networks, inst
Add IPv4 address prefix
Add IPv6 ad
Previous
Next
Review +
Edit subnet
Select an address space and configure your subnet. You can customize a default subnet or select from subnet templates if you plan to add select services later. Learn more
Subnet purpose
Virtual Network Gateway
Name *
GatewaySubnet
IPv4
Include an IPv4 address space
IPv4 address range
10.50.0.0/16
10.50.0.0 - 10.50.255.255
Starting address *
10.50.0.0
Size
/24 (256 addresses)
Subnet address range
10.50.0.0 - 10.50.0.255
IPv6
Include an IPv6 address space
This virtual network has no IPv6 address ranges.
Private subnet
Private subnets enhance security by not providing default outbound access. To enable outbound connectivity for virtual machines to access the internet, it is necessary to explicitly grant outbound access. A NAT gateway is the recommended way to provide outbound connectivity for virtual machines in the subnet. Learn more
Enable private subnet (no default outbound access)
After March 31, 2026, private subnet will be the default selection for new virtual networks. Learn more
Save
Cancel
Give feedback

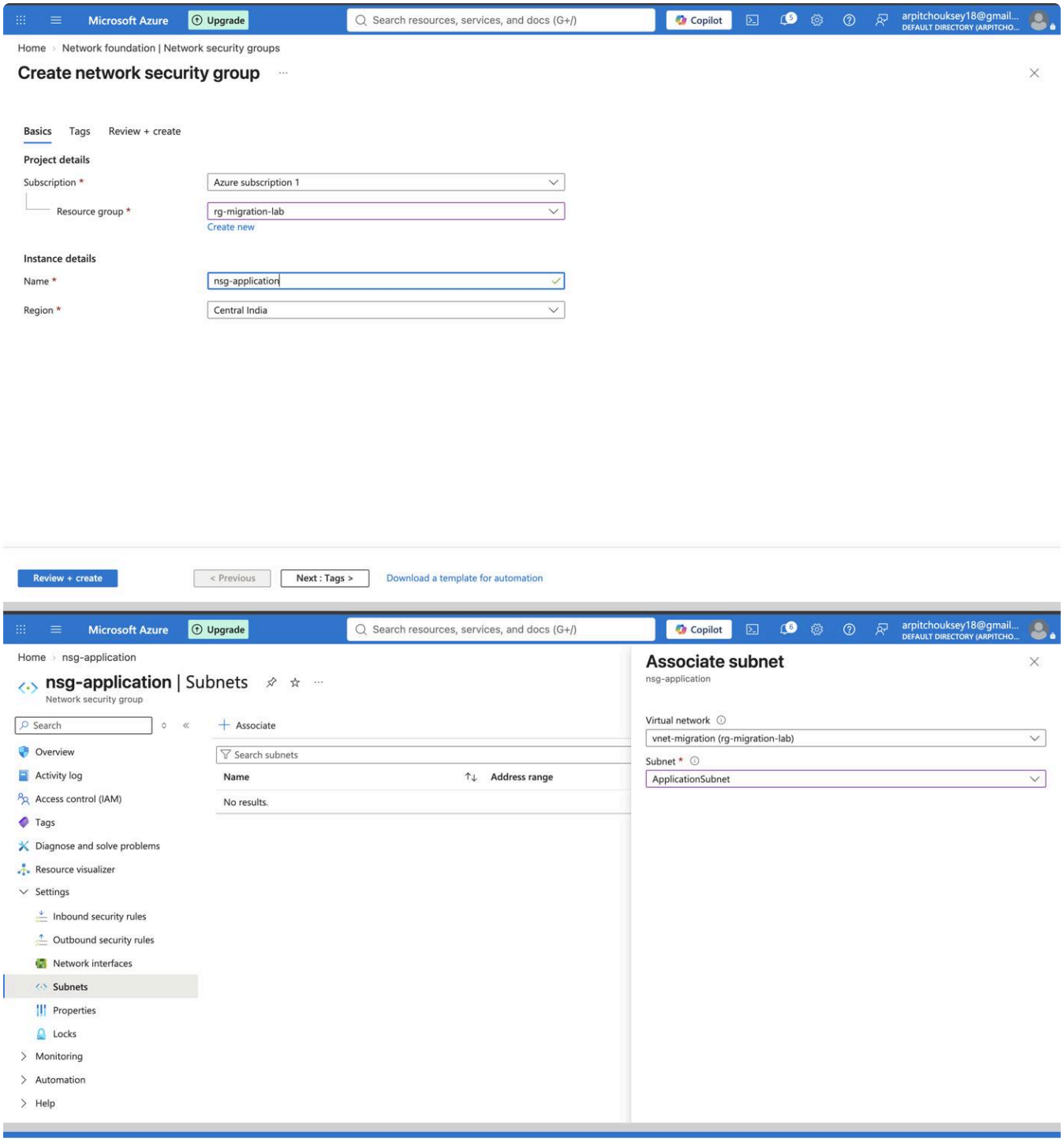

Microsoft Azure
Upgrade
Search resources, services, and docs (G+/)
Copilot
arpitchouksey18@gmail...
DEFAULT DIRECTORY (ARPITCHO...
Home › vnet-migration
vnet-migration | Subnets
Virtual network
Search
Overview
Activity log
Access control (IAM)
Tags
Diagnose and solve problems
Resource visualizer
Settings
Address space
Connected devices
Subnets
Bastion
DDoS protection
Firewall
Microsoft Defender for Cloud
Network manager
DNS
Peerings
Service endpoints
Private endpoints
Add or remove favorites by pressing Cmd+Shift+F
Subnet
Refresh
Create subnets to segment the virtual network addre
from the subnet.
Search subnets
Name
IPv4
GatewaySubnet
10.50.0.0/
Showing 1 subnet
Add a subnet
select services later. Learn more
Subnet purpose
Default
Name *
ApplicationSubnet
IPv4
Include an IPv4 address space
IPv4 address range
10.50.0.0/16
10.50.0.0 - 10.50.255.255
Starting address *
10.50.1.0
Size
/24 (256 addresses)
Subnet address range
10.50.1.0 - 10.50.1.255
IPv6
Include an IPv6 address space
This virtual network has no IPv6 address ranges.
Private subnet
Private subnets enhance security by not providing default outbound access. To enable outbound connectivity for virtual machines to access the internet, it is necessary to explicitly grant outbound access. A NAT gateway is the recommended way to provide outbound connectivity for virtual machines in the subnet. Learn more
Enable private subnet (no default outbound access)
After March 31, 2026, private subnet will be the default selection for new virtual networks. Learn more
Add
Cancel
Give feedback

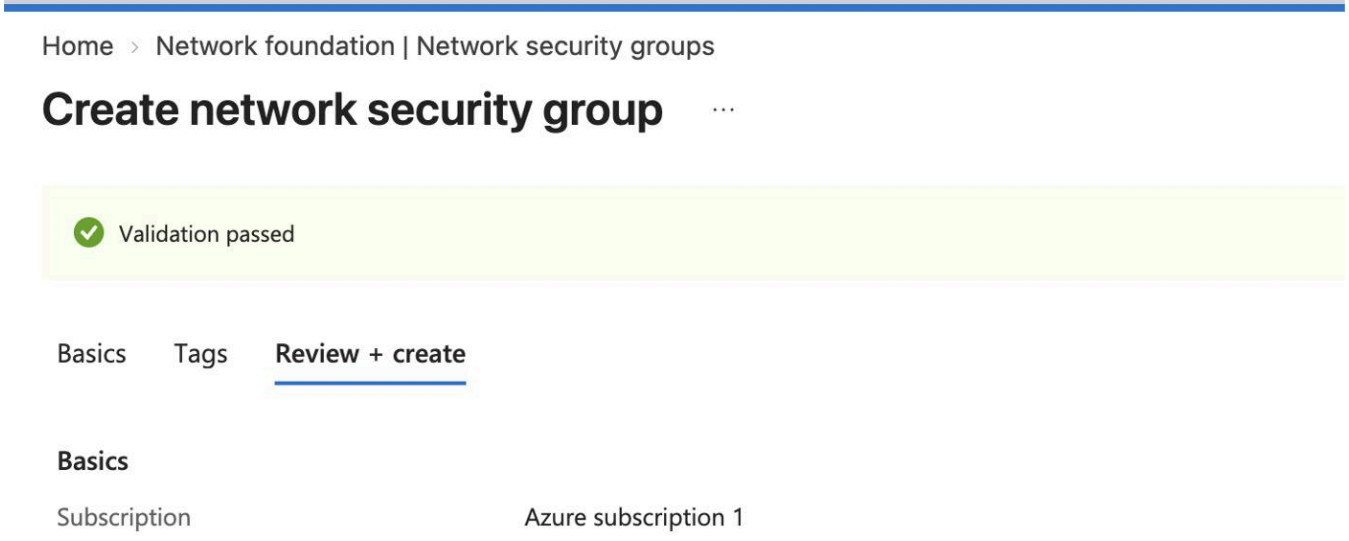

Microsoft Azure
Upgrade
Search resources, services, and docs (G+/)
Copilot
arpitchouksey18@gmail...
DEFAULT DIRECTORY (ARPITCHO...
Home › vnet-migration
vnet-migration | Subnets
Virtual network
Search
Overview
Activity log
Access control (IAM)
Tags
Diagnose and solve problems
Resource visualizer
Settings
Address space
Subnet
Refresh
Create subnets to segment the virtual network addre
from the subnet.
Search subnets
Name
IPv4
GatewaySubnet
10.50.0.0/
ApplicationSubnet
10.50.1.0/
Add a subnet
Select an address space and configure your subnet. You can customize a default subnet or select from subnet templates if you plan to add select services later. Learn more
Subnet purpose
Default
Name *
DatabaseSubnet
IPv4
Include an IPv4 address space
IPv4 address range
10.50.0.0/16
10.50.0.0 - 10.50.255.255
Starting address *
10.50.2.0
Size
/24 (256 addresses)



Step 4 — Create the Network Security Groups

1. Network security groups → Create → `nsg-application` in `rg-migration-lab`, Central India → Create.
2. Repeat for `nsg-database`.
3. Open each NSG → Subnets → Associate: `nsg-application` → ApplicationSubnet, `nsg-database` → DatabaseSubnet.
4. GatewaySubnet gets no NSG (Azure does not support NSGs on the gateway subnet).


Microsoft Azure
Upgrade
Search resources, services, and docs (G+/)
Copilot
arpitchouksey18@gmail...
DEFAULT DIRECTORY (ARPITCHO...
Home › Network foundation | Network security groups
Create network security group
Basics
Tags
Review + create
Project details
Subscription *
Azure subscription 1
Resource group *
rg-migration-lab
Create new
Instance details
Name *
nsg-application
Region *
Central India
Review + create
< Previous
Next : Tags >
Download a template for automation
Home › nsg-application
nsg-application | Subnets
Network security group
Search
Overview
Activity log
Access control (IAM)
Tags
Diagnose and solve problems
Resource visualizer
Settings
Inbound security rules
Outbound security rules
Network interfaces
Subnets
Properties
Locks
Monitoring
Automation
Help
Associate
Search subnets
Name
Address range
No results.
Associate subnet
nsg-application
Virtual network
vnet-migration (rg-migration-lab)
Subnet *
ApplicationSubnet

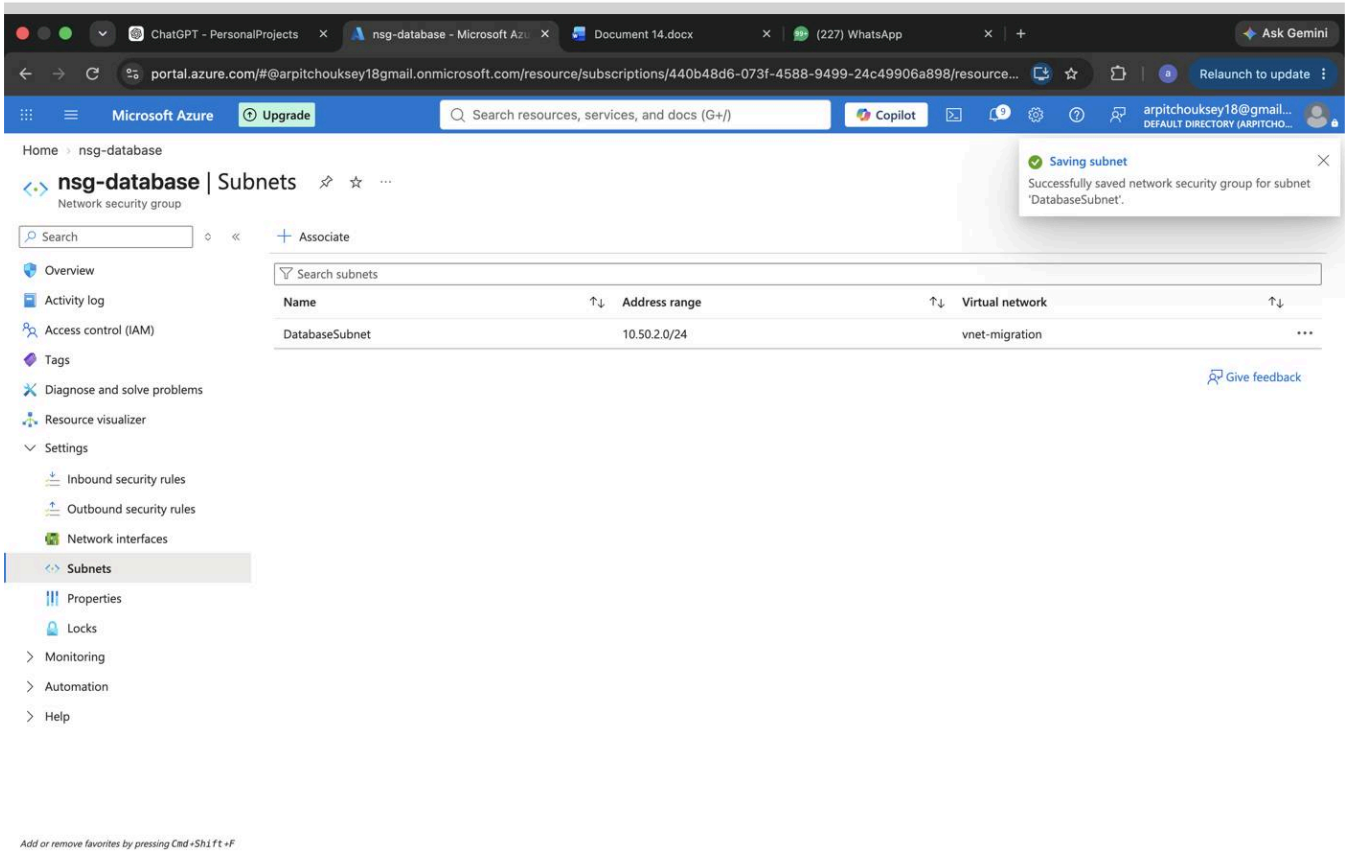

Home › Network foundation | Network security groups
Create network security group
Validation passed
Basics
Tags
Review + create
Basics
Subscription
Azure subscription 1

Resource group	rg-migration-lab
Region	Central India
name	nsg-database

Tags

None

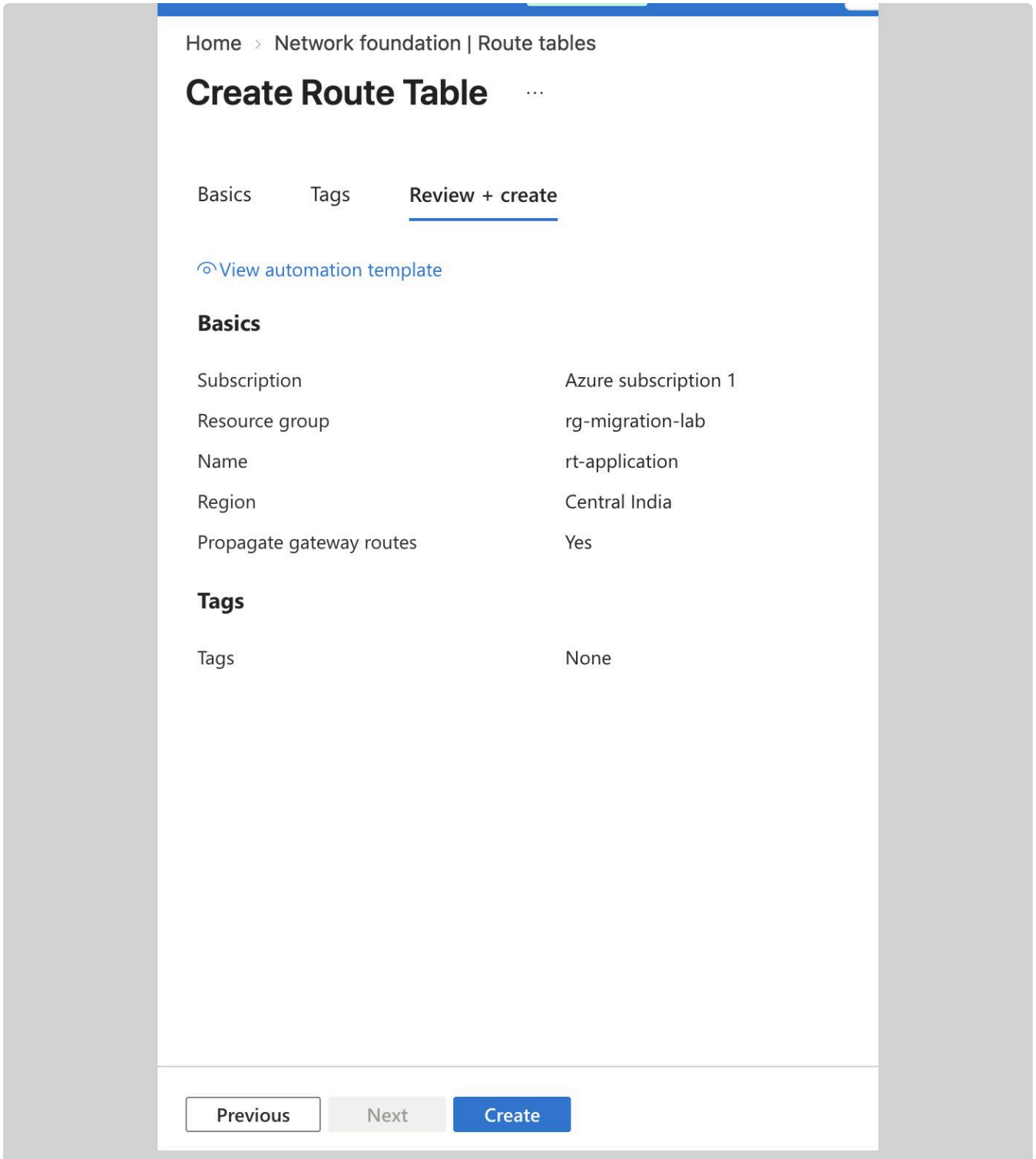
Create < Previous Next > Download a template for automation

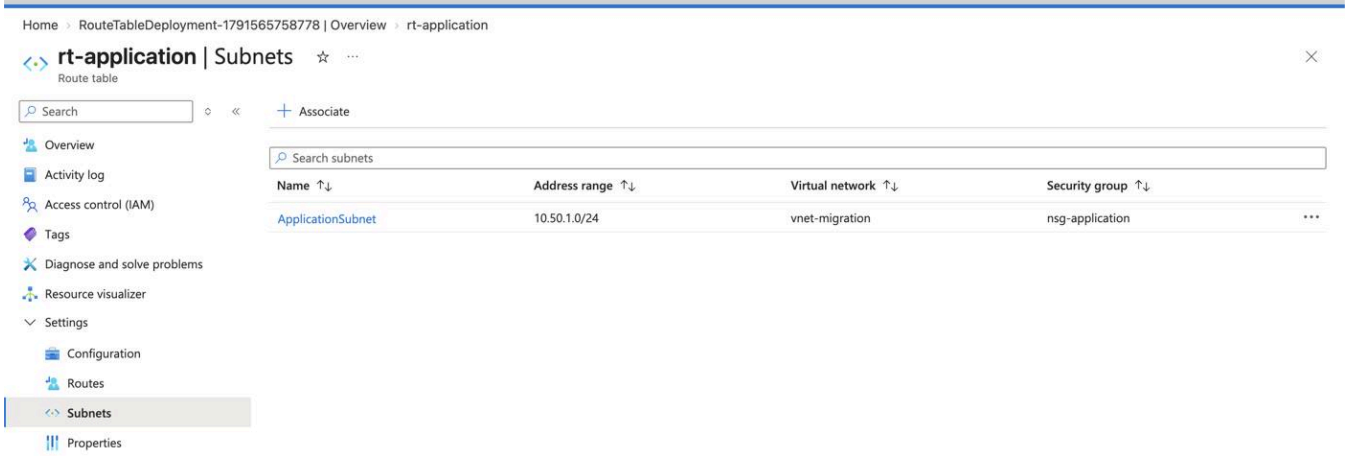


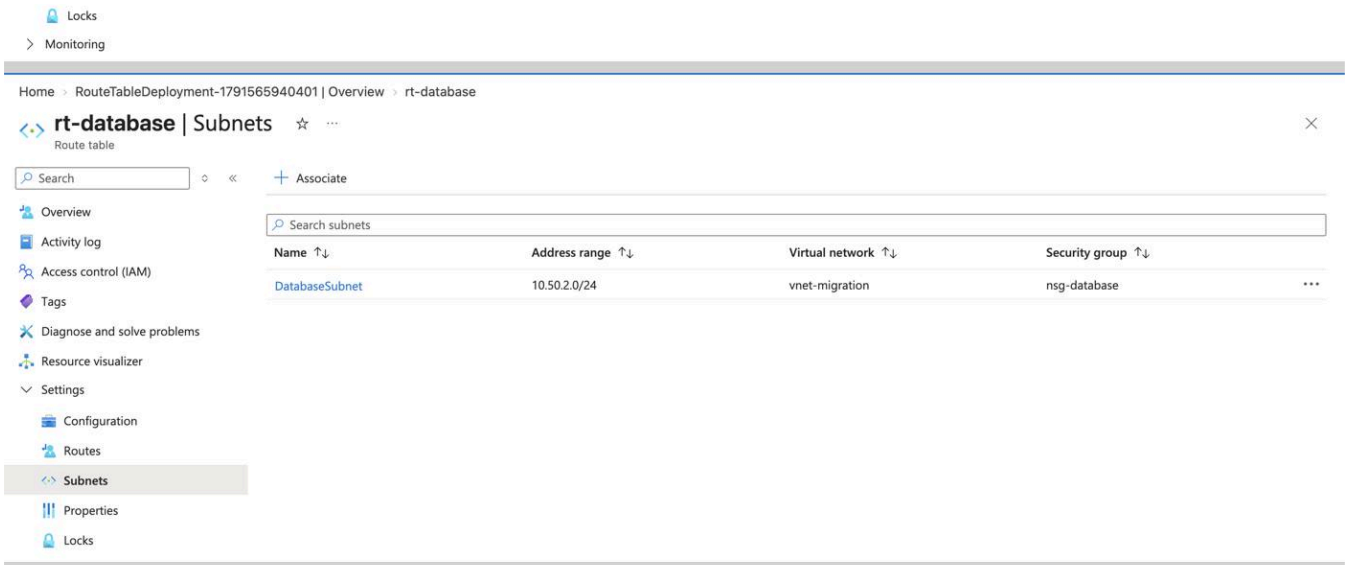
Step 5 — Create the route tables (UDRs)

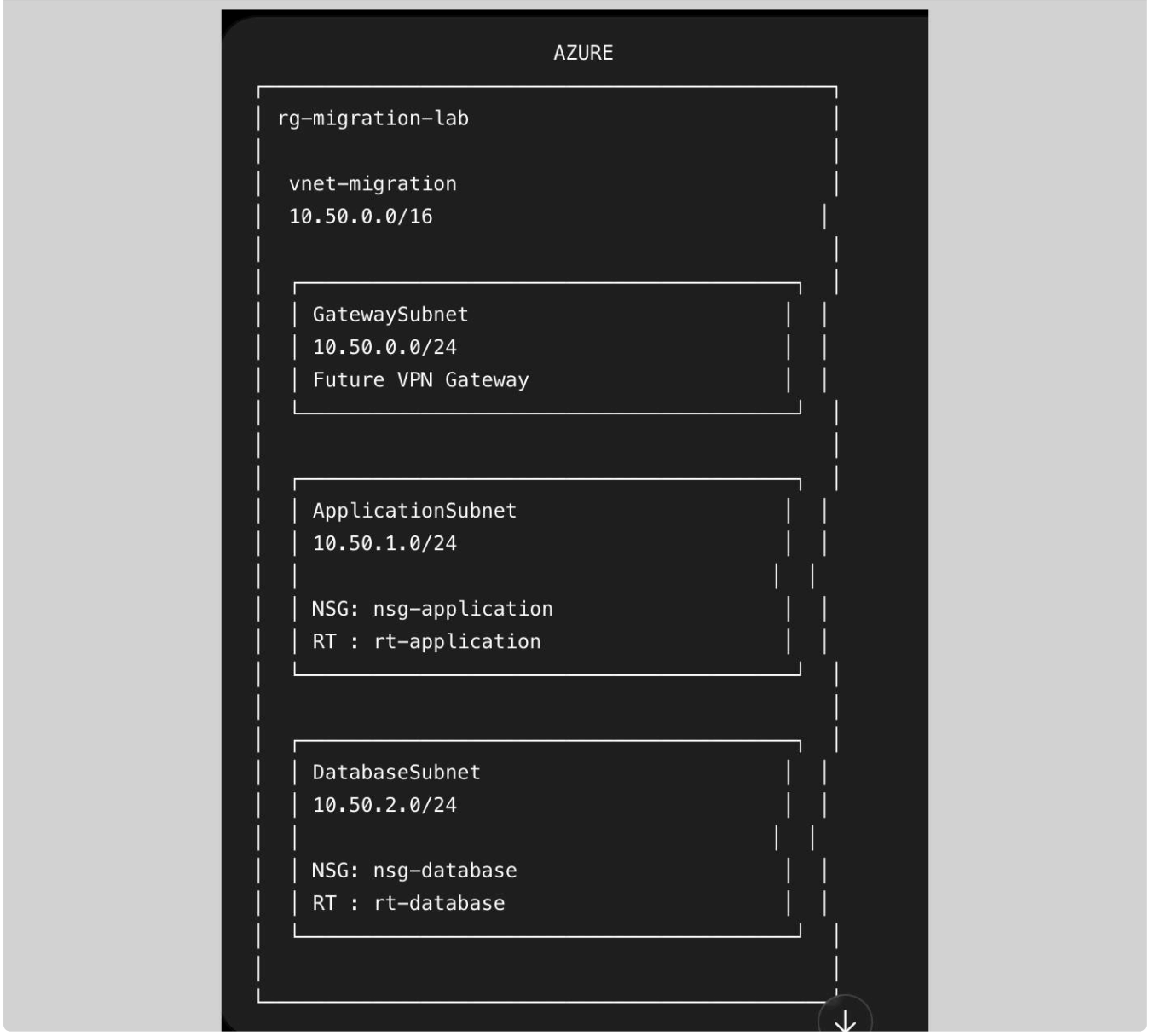
1. Route tables → Create → `rt-application`, propagate gateway routes = Yes → Create.
2. Repeat for `rt-database`.

3. Route table → Subnets → Associate: `rt-application` → ApplicationSubnet, `rt-database` → DatabaseSubnet.
4. Gateway route propagation stays on so routes learned over the VPN later appear automatically.


Home › Network foundation | Route tables
Create Route Table
Basics
Tags
Review + create
View automation template
Basics
Subscription
Azure subscription 1
Resource group
rg-migration-lab
Name
rt-application
Region
Central India
Propagate gateway routes
Yes
Tags
Tags
None
Previous
Next
Create


Home › RouteTableDeployment-1791565758778 | Overview › rt-application
rt-application | Subnets
Route table
Search
Associate
Overview
Activity log
Access control (IAM)
Tags
Diagnose and solve problems
Resource visualizer
Settings
Configuration
Routes
Subnets
Properties
Search subnets
Name
Address range
Virtual network
Security group
ApplicationSubnet
10.50.1.0/24
vnet-migration
nsg-application





Result — Azure layout

```
rg-migration-lab (Central India)
└── vnet-migration  10.50.0.0/16
    ├── GatewaySubnet      10.50.0.0/24  [Azure VPN Gateway]
    ├── ApplicationSubnet  10.50.1.0/24  NSG: nsg-application · RT: rt-
application
    └── DatabaseSubnet     10.50.2.0/24  NSG: nsg-database    · RT: rt-
database
```

Also in scope for the Azure foundation: DNS configuration, network monitoring/logging foundation, Storage account/Blob and Key Vault as migration targets.

2.5 Terraform import of the Azure resources

The console-built Azure network was adopted by Terraform without recreating anything: the first plan showed 13 to import, 0 to add, 2 to change, 0 to destroy, and the final plan showed no changes.

Why import instead of recreate: real brownfield estates already exist. Importing teaches how to bring live infrastructure under code safely, then manage it through Terraform from then on.

Step 1 — Module layout

```
migration/
├── modules/
|   ├── resource_group/          # azurerm_resource_group.this
|   ├── networking/              # azurerm_virtual_network.this,
azurerm_subnet.this[for_each],
|   |                            # subnet ↔ NSG and subnet ↔ route table
associations
|   ├── network_security_group/  #
azurerm_network_security_group.this[for_each]
|   └── route_table/             # azurerm_route_table.this[for_each]
├── environments/
├── providers.tf
├── variables.tf
├── locals.tf
├── main.tf
├── outputs.tf
└── terraform.tfvars
```

Values such as names, CIDRs and region live in `terraform.tfvars` and `locals.tf`; nothing is hardcoded inside modules.

Step 2 — Map every live resource to a Terraform address

#	Terraform address	Azure resource
1	module.resource_group.azurerm_resource_group.this	rg-migration-lab
2	module.networking.azurerm_virtual_network.this	vnet-migration
3	module.networking.azurerm_subnet.this["GatewaySubnet"]	GatewaySubnet
4	module.networking.azurerm_subnet.this["ApplicationSubnet"]	ApplicationSubnet
5	module.networking.azurerm_subnet.this["DatabaseSubnet"]	DatabaseSubnet
6	module.network_security_group.azurerm_network_security_group.this["nsg-application"]	nsg-application
7	module.network_security_group.azurerm_network_security_group.this["nsg-database"]	nsg-database
8	module.route_table.azurerm_route_table.this["rt-application"]	rt-application
9	module.route_table.azurerm_route_table.this["rt-database"]	rt-database
10	module.networking.azurerm_subnet_network_security_group_association.this["ApplicationSubnet"]	App subnet ↔ NSG
11	module.networking.azurerm_subnet_network_security_group_association.this["DatabaseSubnet"]	DB subnet ↔ NSG
12	module.networking.azurerm_subnet_route_table_association.this["ApplicationSubnet"]	App subnet ↔ RT
13	module.networking.azurerm_subnet_route_table_association.this["DatabaseSubnet"]	DB subnet ↔ RT

Step 3 — Write import blocks (pattern; the subscription ID is replaced with a placeholder)

```
import {
  to = module.resource_group.azurerm_resource_group.this
  id = "/subscriptions/<subscription-id>/resourceGroups/rg-migration-lab"
}

import {
  to = module.networking.azurerm_virtual_network.this
  id = "/subscriptions/<subscription-id>/resourceGroups/rg-migration-
lab/providers/Microsoft.Network/virtualNetworks/vnet-migration"
}
```

Step 4 — Format, validate, plan

```
terraform fmt -recursive
terraform validate      # Success! The configuration is valid.
terraform plan
```

Plan result:

```
Plan: 13 to import, 0 to add, 2 to change, 0 to destroy.

# module.networking.azurerm_subnet.this["ApplicationSubnet"] will be updated
in-place
  ~ default_outbound_access_enabled = true -> false
# module.networking.azurerm_subnet.this["DatabaseSubnet"] will be updated in-
place
  ~ default_outbound_access_enabled = true -> false
```

Step 5 — Review the 2 changes

1. Both changes switch default outbound internet access off on the app and DB subnets.
2. This matches the private-subnet intent, so the change is accepted rather than suppressed.
3. Nothing is destroyed or recreated, so the import is safe.

Step 6 — Apply and confirm zero drift

```
terraform apply
terraform plan
# No changes. Your infrastructure matches the configuration.
```

Outcome: the Azure network is now fully managed as code. Any later console change shows up as drift in `terraform plan`.

Tip: save plans with `terraform plan -out=tfplan` and apply that file, so Terraform applies exactly what was reviewed.

2.6 AWS source environment — step by step

The AWS side represents the company's live production estate that will be migrated. Region us-east-1, account kept private. Only the minimum resources were created.

Target source architecture

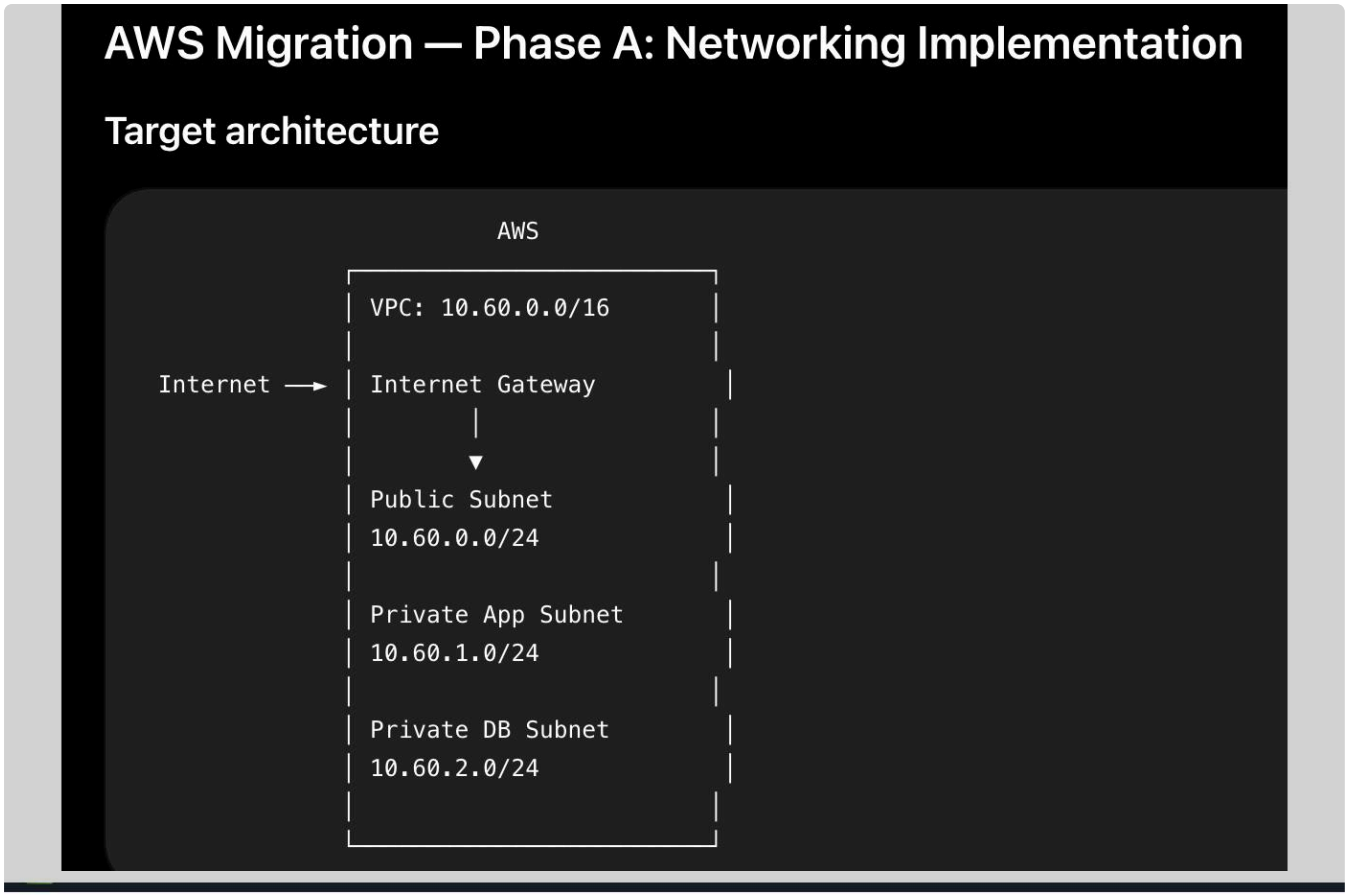
```
AWS SOURCE ENVIRONMENT (us-east-1)
vpc-migration 10.60.0.0/16
├── PublicSubnet     10.60.0.0/24  → Internet Gateway
├── AppSubnet        10.60.1.0/24  → EC2 migration-app-server
├── DatabaseSubnet   10.60.2.0/24  (us-east-1b) → RDS PostgreSQL
└── DatabaseSubnet2  10.60.3.0/24  (us-east-1a)
S3 migration-source-v1          → application data
Secrets Manager migration/app/database → application secrets
```

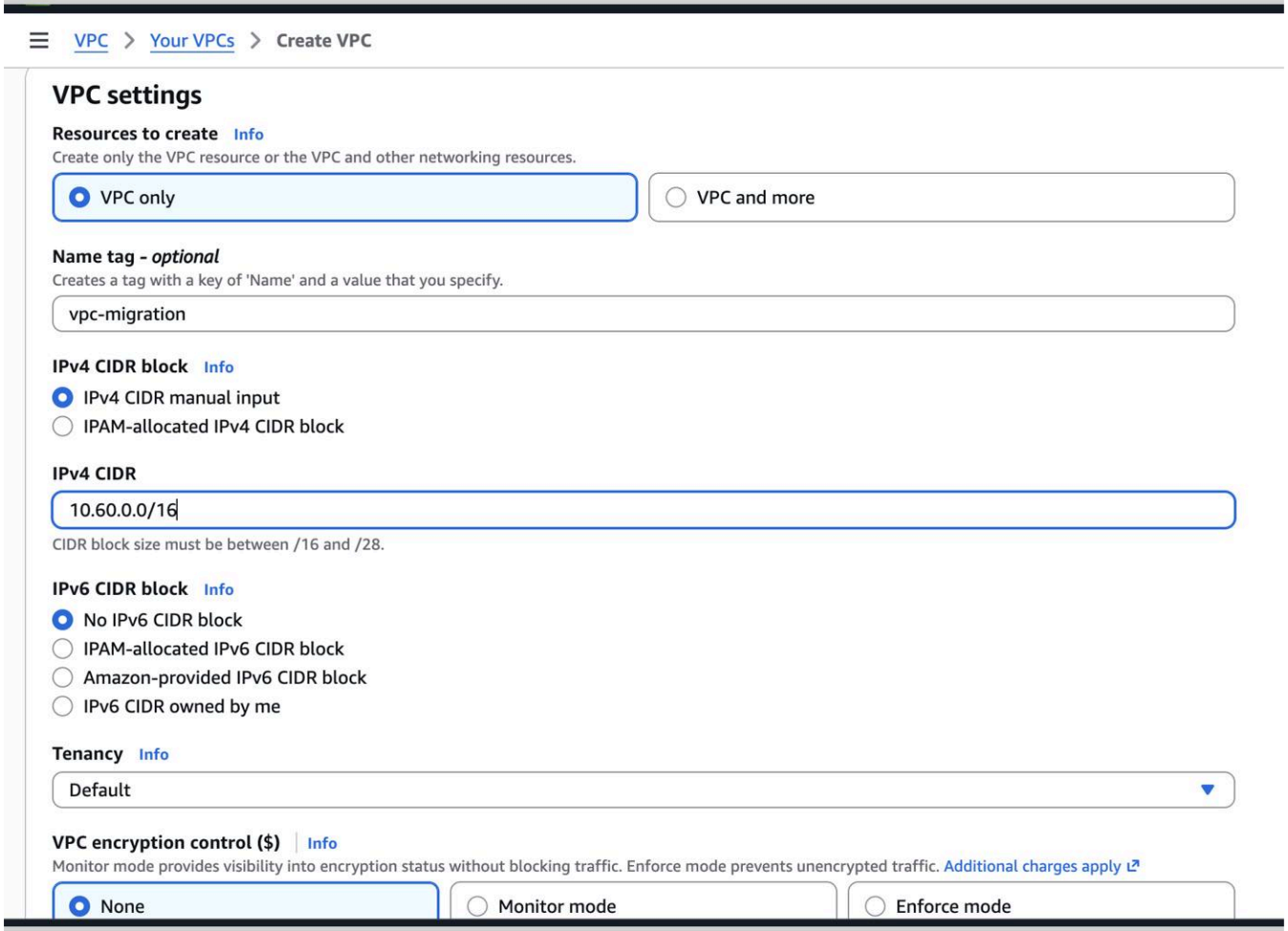
Step 1 — Create the VPC and subnets

1. VPC → Your VPCs → Create VPC → *VPC only*.
2. Name `vpc-migration`, IPv4 CIDR 10.60.0.0/16, no IPv6, tenancy Default.
3. Subnets → Create subnet → select `vpc-migration`.
4. Create `PublicSubnet` 10.60.0.0/24, `AppSubnet` 10.60.1.0/24, `DatabaseSubnet` 10.60.2.0/24 in us-east-1b.

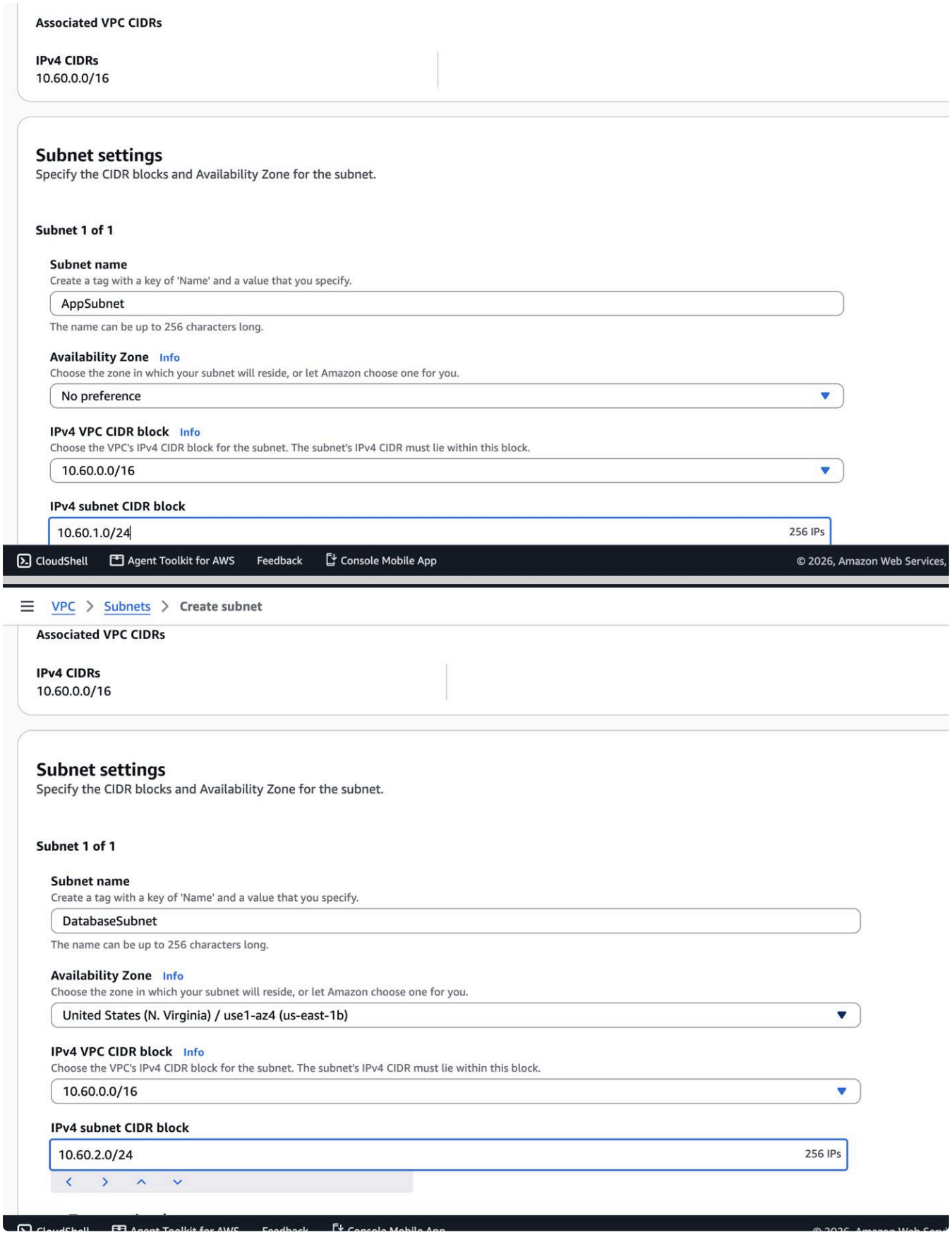
AWS Migration — Phase A: Networking Implementation

Target architecture





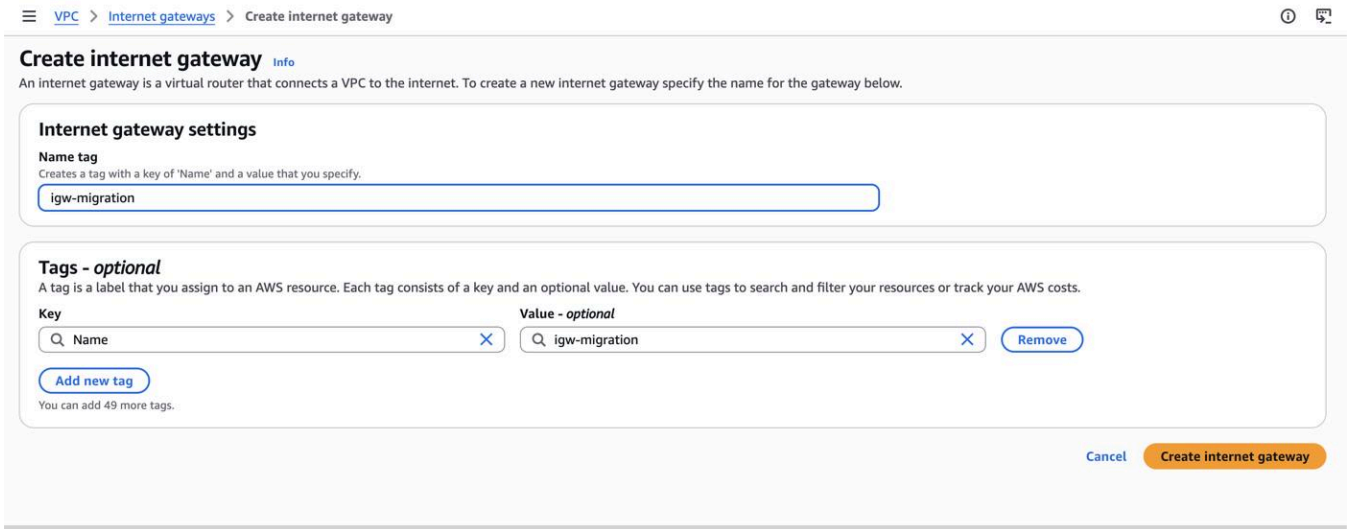


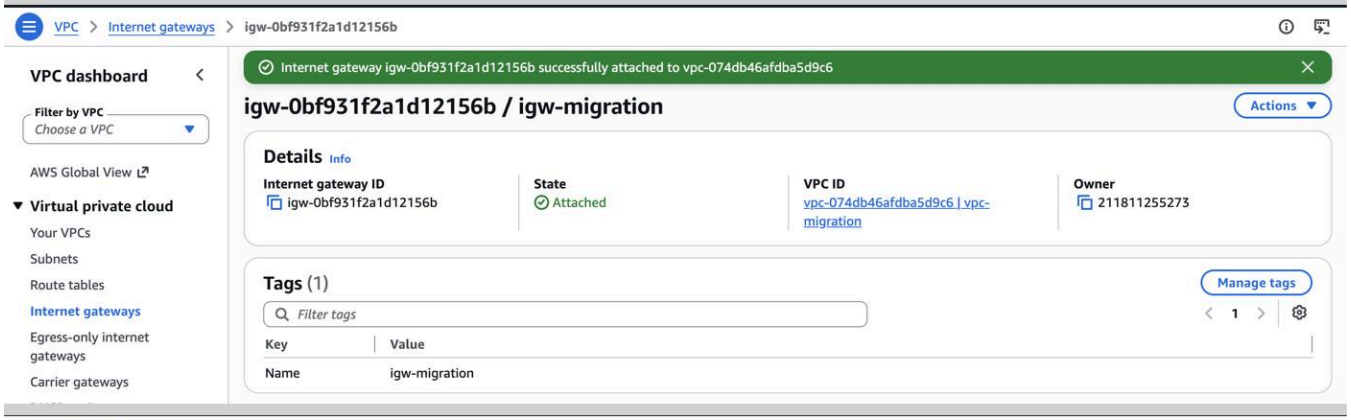


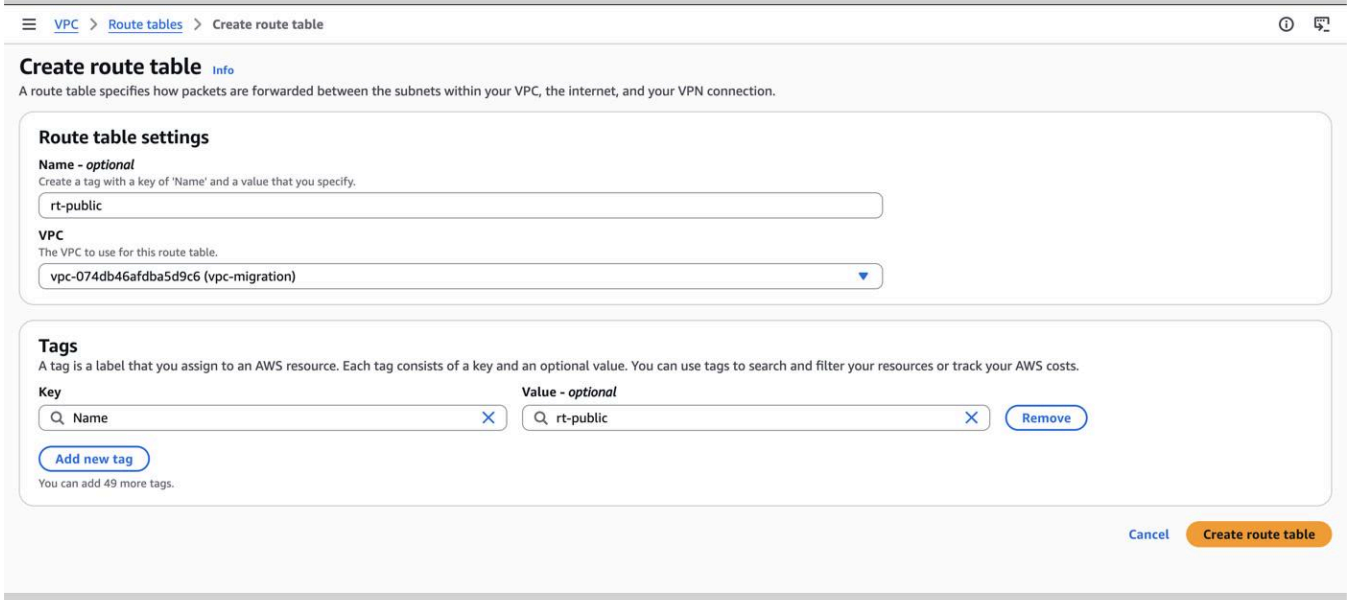
Step 2 — Internet gateway and route tables

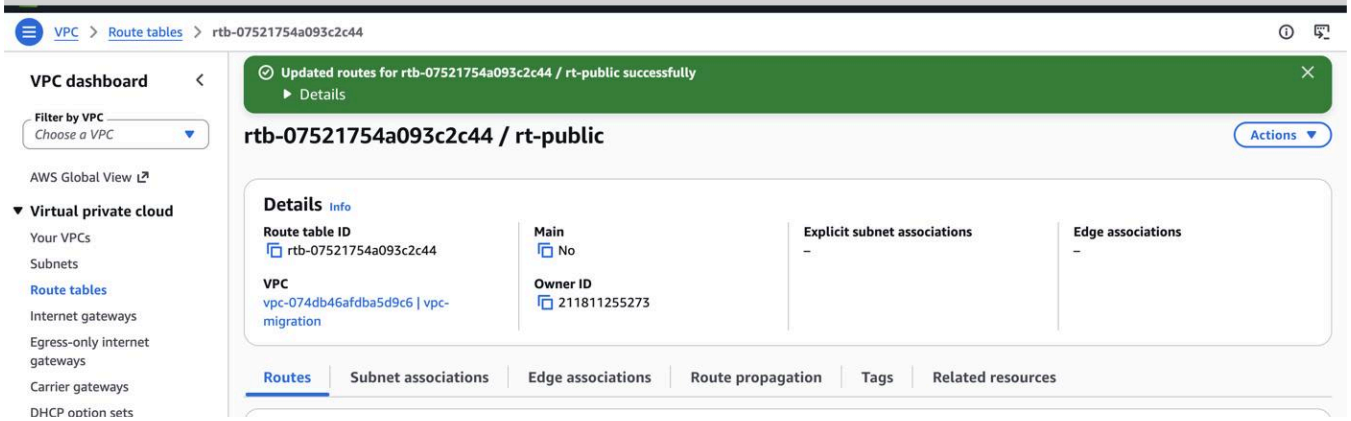
1. Internet gateways → Create → `igw-migration` → Actions → Attach to VPC → `vpc-migration`.

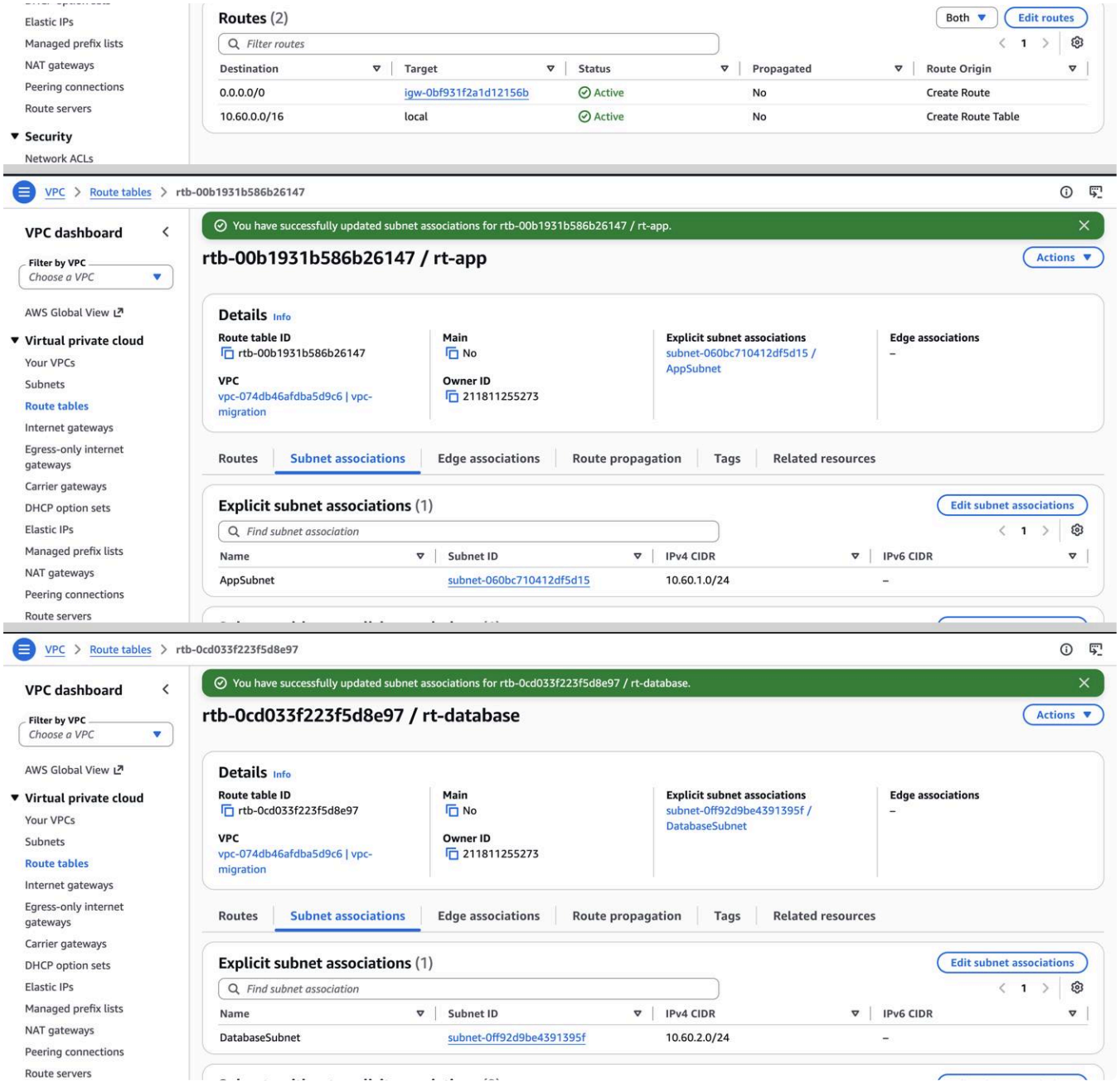
2. Route tables → Create `rt-public` → routes: 10.60.0.0/16 local, 0.0.0.0/0 → igw-migration → associate PublicSubnet.
3. Create `rt-app` → associate AppSubnet.
4. Create `rt-database` → local route only (database stays private) → associate DatabaseSubnet.


VPC > Internet gateways > Create internet gateway
Create internet gateway Info
An internet gateway is a virtual router that connects a VPC to the internet. To create a new internet gateway specify the name for the gateway below.
Internet gateway settings
Name tag
Creates a tag with a key of 'Name' and a value that you specify.
igw-migration
Tags - optional
A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.
Key
Name
Value - optional
igw-migration
Remove
Add new tag
You can add 49 more tags.
Cancel
Create internet gateway


VPC > Internet gateways > igw-0bf931f2a1d12156b
Internet gateway igw-0bf931f2a1d12156b successfully attached to vpc-074db46afdba5d9c6
VPC dashboard
Filter by VPC
Choose a VPC
AWS Global View
Virtual private cloud
Your VPCs
Subnets
Route tables
Internet gateways
Egress-only internet gateways
Carrier gateways
igw-0bf931f2a1d12156b / igw-migration
Actions
Details Info
Internet gateway ID
igw-0bf931f2a1d12156b
State
Attached
VPC ID
vpc-074db46afdba5d9c6 | vpc-migration
Owner
211811255273
Tags (1)
Manage tags
Filter tags
Key
Value
Name
igw-migration


VPC > Route tables > Create route table
Create route table Info
A route table specifies how packets are forwarded between the subnets within your VPC, the internet, and your VPN connection.
Route table settings
Name - optional
Create a tag with a key of 'Name' and a value that you specify.
rt-public
VPC
The VPC to use for this route table.
vpc-074db46afdba5d9c6 (vpc-migration)
Tags
A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.
Key
Name
Value - optional
rt-public
Remove
Add new tag
You can add 49 more tags.
Cancel
Create route table

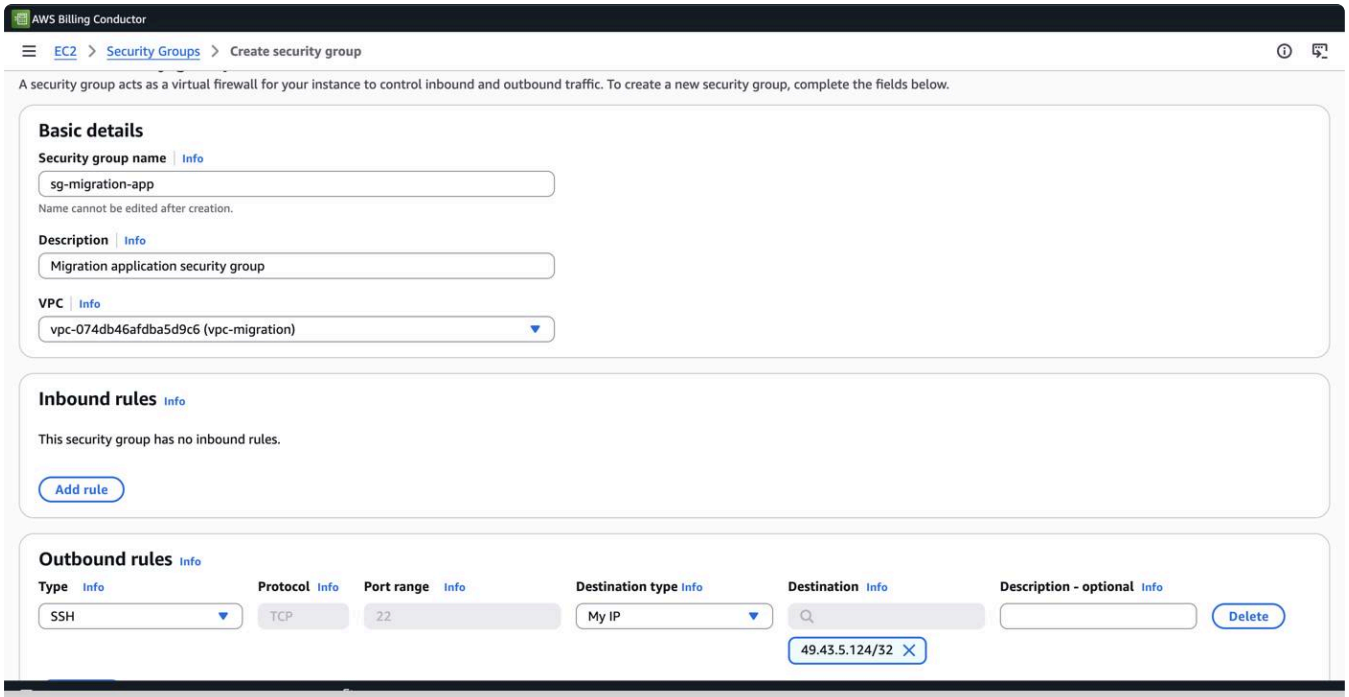

VPC > Route tables > rtb-07521754a093c2c44
Updated routes for rtb-07521754a093c2c44 / rt-public successfully
Details
VPC dashboard
Filter by VPC
Choose a VPC
AWS Global View
Virtual private cloud
Your VPCs
Subnets
Route tables
Internet gateways
Egress-only internet gateways
Carrier gateways
DHCP option sets
rtb-07521754a093c2c44 / rt-public
Actions
Details Info
Route table ID
rtb-07521754a093c2c44
Main
No
Explicit subnet associations
–
Edge associations
–
VPC
vpc-074db46afdba5d9c6 | vpc-migration
Owner ID
211811255273
Routes
Subnet associations
Edge associations
Route propagation
Tags
Related resources

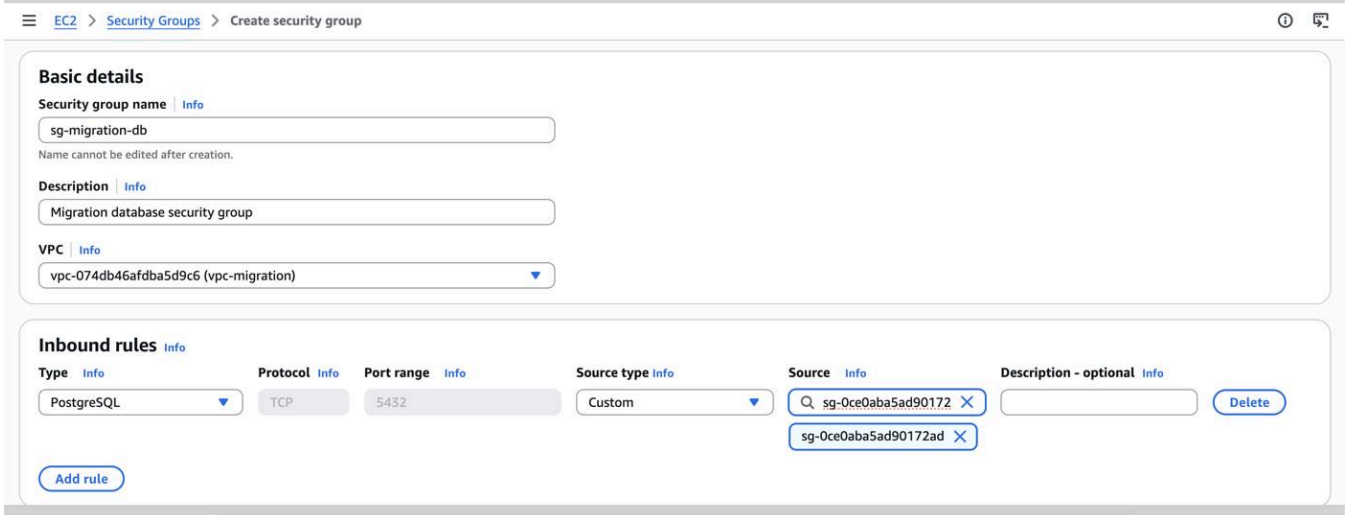


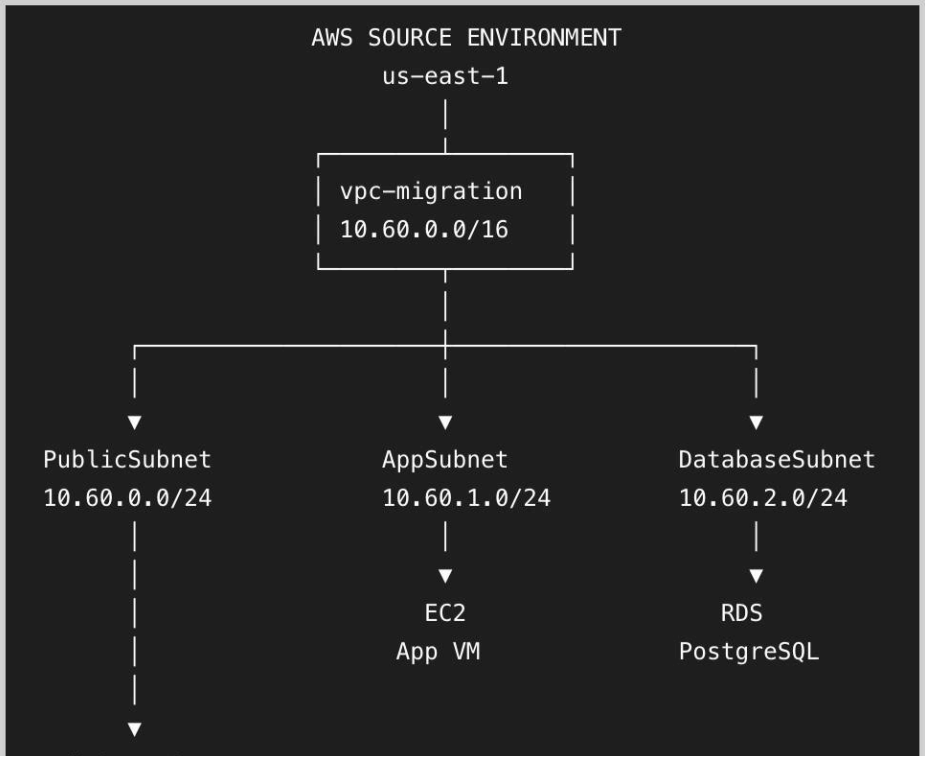
Step 3 — Security groups

AWS does not allow security group names starting with `sg-`, so the names are `migration-app` and `migration-db`.

Security group	Inbound rule	Source
migration-app	Only required admin/app ports (Session Manager used, so no open SSH to the internet)	Restricted
migration-db	TCP 5432 (PostgreSQL)	migration-app security group only


AWS Billing Conductor
EC2 > Security Groups > Create security group
A security group acts as a virtual firewall for your instance to control inbound and outbound traffic. To create a new security group, complete the fields below.
Basic details
Security group name | Info
sg-migration-app
Name cannot be edited after creation.
Description | Info
Migration application security group
VPC | Info
vpc-074db46afdba5d9c6 (vpc-migration)
Inbound rules Info
This security group has no inbound rules.
Add rule
Outbound rules Info
Type Info
Protocol Info
Port range Info
Destination type Info
Destination Info
Description - optional Info
SSH
TCP
22
My IP
49.43.5.124/32
Delete


EC2 > Security Groups > Create security group
Basic details
Security group name | Info
sg-migration-db
Name cannot be edited after creation.
Description | Info
Migration database security group
VPC | Info
vpc-074db46afdba5d9c6 (vpc-migration)
Inbound rules Info
Type Info
Protocol Info
Port range Info
Source type Info
Source Info
Description - optional Info
PostgreSQL
TCP
5432
Custom
sg-0ce0aba5ad90172
sg-0ce0aba5ad90172ad
Delete
Add rule


AWS SOURCE ENVIRONMENT
us-east-1
vpc-migration
10.60.0.0/16
PublicSubnet
10.60.0.0/24
AppSubnet
10.60.1.0/24
DatabaseSubnet
10.60.2.0/24
EC2
App VM
RDS
PostgreSQL



Step 4 — IAM role for the EC2 instance

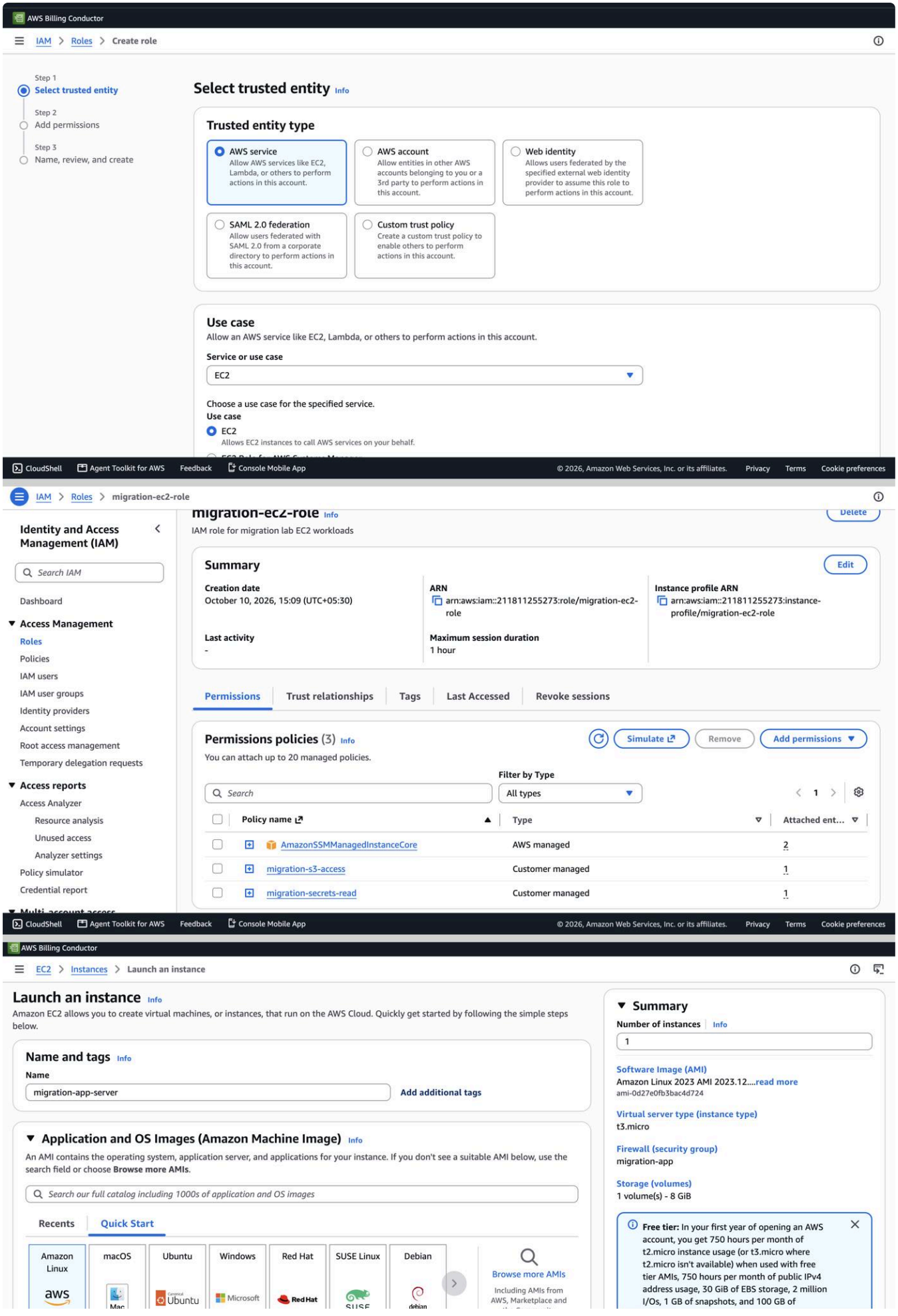
1. IAM → Roles → Create role → trusted entity *AWS service* → use case *EC2*.
2. Attach `AmazonSSMManagedInstanceCore` plus two customer-managed policies for the required S3 bucket and the required secret.
3. Name the role `migration-ec2-role`.

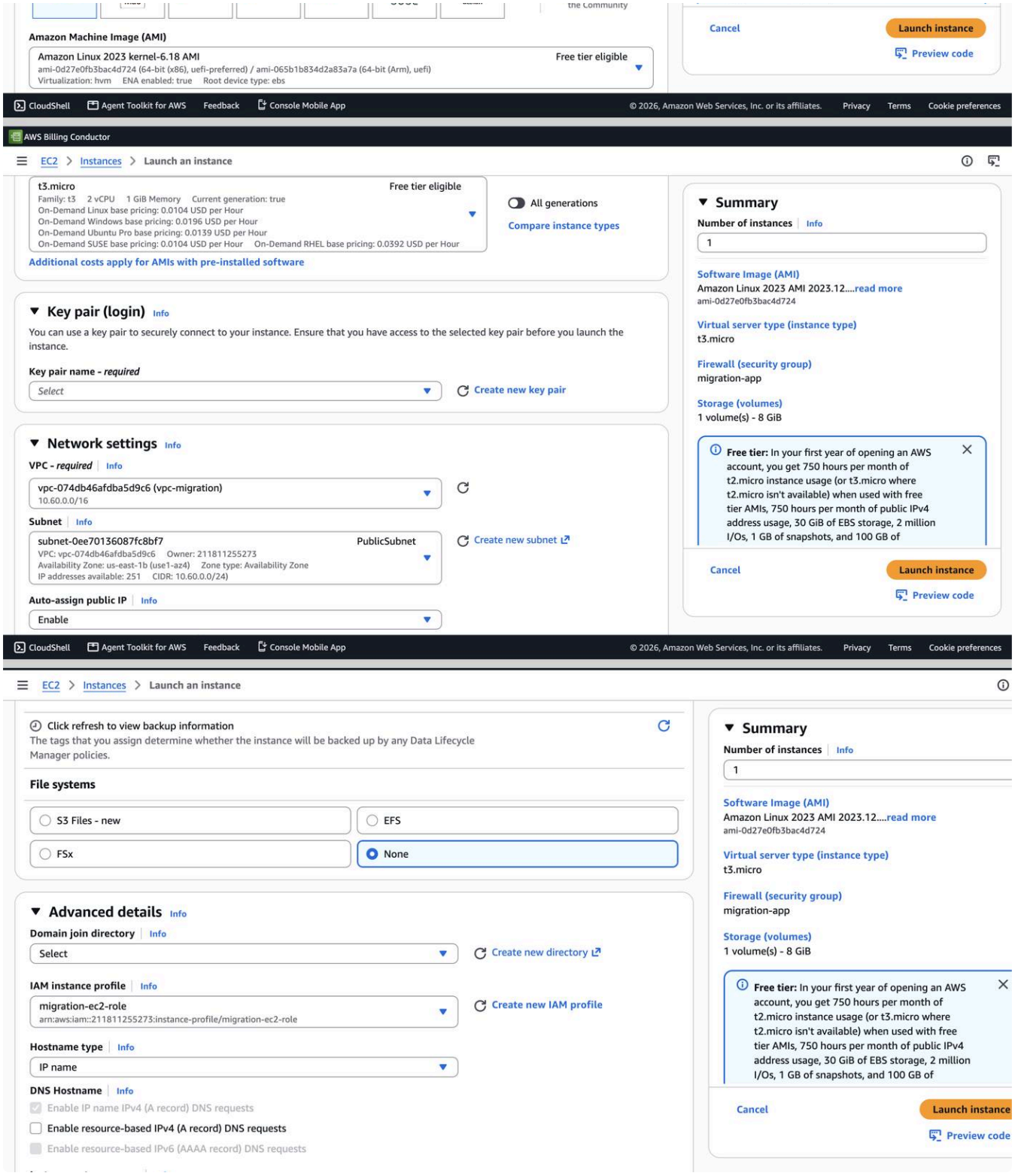
Least-privilege model:

```
migration-ec2-role
├── SSM              → allowed
├── Required S3      → allowed
├── Required Secret  → allowed
└── Everything else  → denied
```

Step 5 — Launch the EC2 application server

1. EC2 → Launch instances → name `migration-app-server`.
2. AMI Amazon Linux 2023, small free-tier-eligible instance type.
3. Network: `vpc-migration`, AppSubnet, security group `migration-app`.
4. Advanced details → IAM instance profile `migration-ec2-role`.
5. Launch. Private IP assigned: 10.60.0.117. Access is through Session Manager.


AWS Billing Conductor
IAM > Roles > Create role
Step 1
Select trusted entity
Step 2
Add permissions
Step 3
Name, review, and create
Select trusted entity Info
Trusted entity type
AWS service
Allow AWS services like EC2, Lambda, or others to perform actions in this account.
AWS account
Allow entities in other AWS accounts belonging to you or a 3rd party to perform actions in this account.
Web identity
Allows users federated by the specified external web identity provider to assume this role to perform actions in this account.
SAML 2.0 federation
Allow users federated with SAML 2.0 from a corporate directory to perform actions in this account.
Custom trust policy
Create a custom trust policy to enable others to perform actions in this account.
Use case
Allow an AWS service like EC2, Lambda, or others to perform actions in this account.
Service or use case
EC2
Choose a use case for the specified service.
Use case
EC2
Allows EC2 instances to call AWS services on your behalf.
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
IAM > Roles > migration-ec2-role
Identity and Access Management (IAM)
Search IAM
Dashboard
Access Management
Roles
Policies
IAM users
IAM user groups
Identity providers
Account settings
Root access management
Temporary delegation requests
Access reports
Access Analyzer
Resource analysis
Unused access
Analyzer settings
Policy simulator
Credential report
migration-ec2-role Info
IAM role for migration lab EC2 workloads
Delete
Summary
Edit
Creation date
October 10, 2026, 15:09 (UTC+05:30)
ARN
arn:aws:iam::211811255273:role/migration-ec2-role
Instance profile ARN
arn:aws:iam::211811255273:instance-profile/migration-ec2-role
Last activity
-
Maximum session duration
1 hour
Permissions
Trust relationships
Tags
Last Accessed
Revoke sessions
Permissions policies (3) Info
You can attach up to 20 managed policies.
Simulate
Remove
Add permissions
Filter by Type
Search
All types
Policy name
Type
Attached ent...
AmazonSSMManagedInstanceCore
AWS managed
2
migration-s3-access
Customer managed
1
migration-secrets-read
Customer managed
1
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
AWS Billing Conductor
EC2 > Instances > Launch an instance
Launch an instance Info
Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.
Name and tags Info
Name
migration-app-server
Add additional tags
Application and OS Images (Amazon Machine Image) Info
An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose Browse more AMIs.
Search our full catalog including 1000s of application and OS images
Recents
Quick Start
Amazon Linux
macOS
Ubuntu
Windows
Red Hat
SUSE Linux
Debian
Browse more AMIs
Including AMIs from AWS, Marketplace and
Summary
Number of instances Info
1
Software Image (AMI)
Amazon Linux 2023 AMI 2023.12....read more
ami-0d27e0fb3bac4d724
Virtual server type (instance type)
t3.micro
Firewall (security group)
migration-app
Storage (volumes)
1 volume(s) - 8 GiB
Free tier: In your first year of opening an AWS account, you get 750 hours per month of t2.micro instance usage (or t3.micro where t2.micro isn't available) when used with free tier AMIs, 750 hours per month of public IPv4 address usage, 30 GiB of EBS storage, 2 million I/Os, 1 GB of snapshots, and 100 GB of

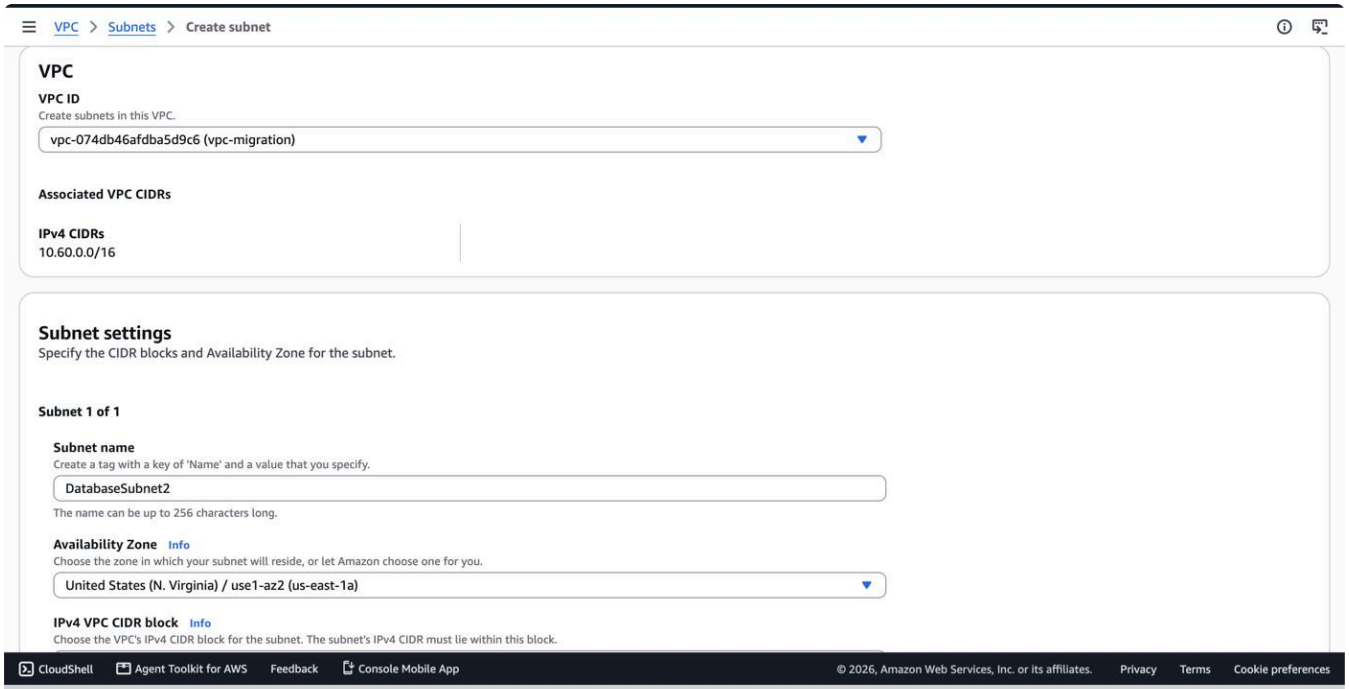


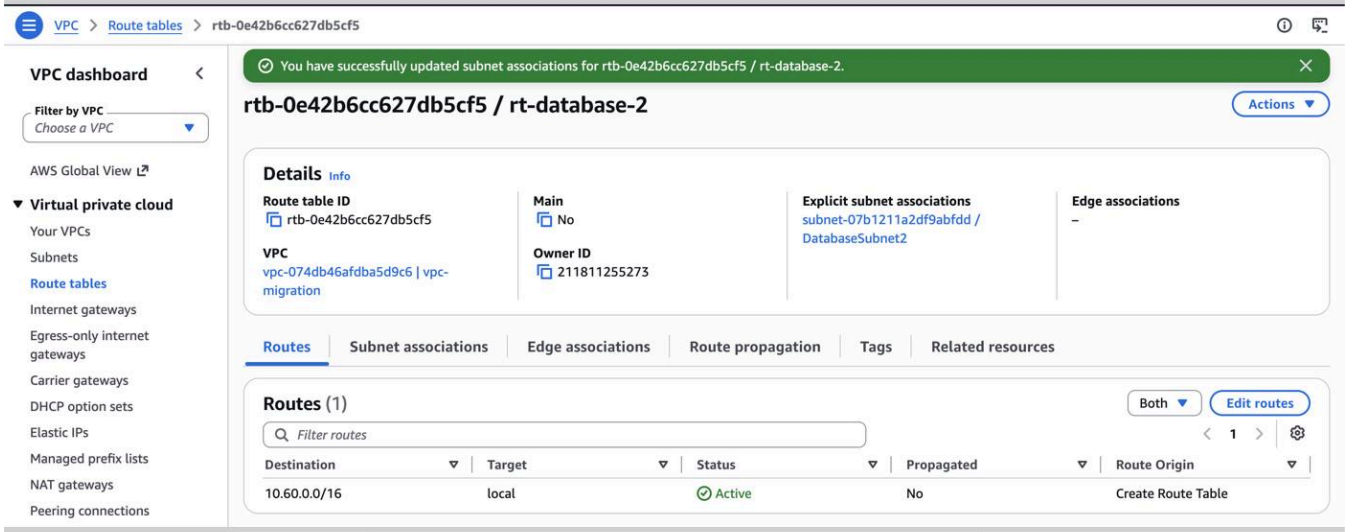
Step 6 — Second DB subnet and DB subnet group

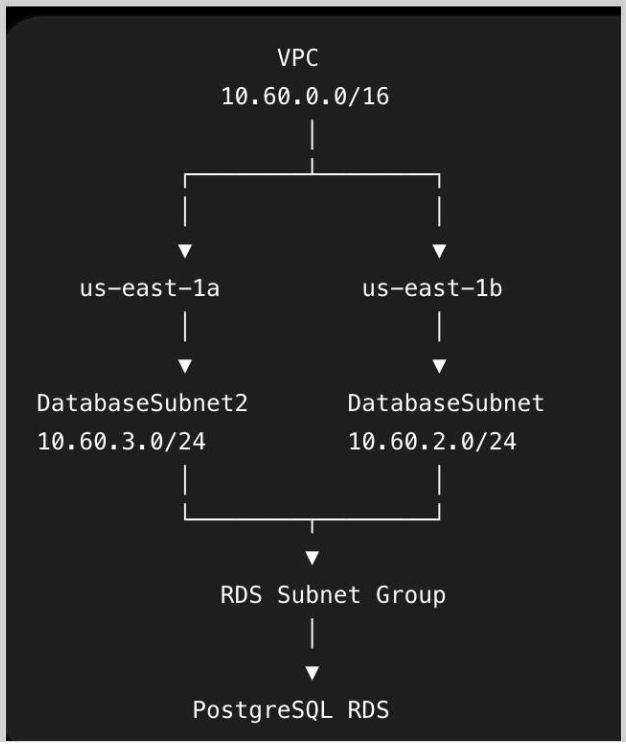
RDS needs subnets in at least two Availability Zones.

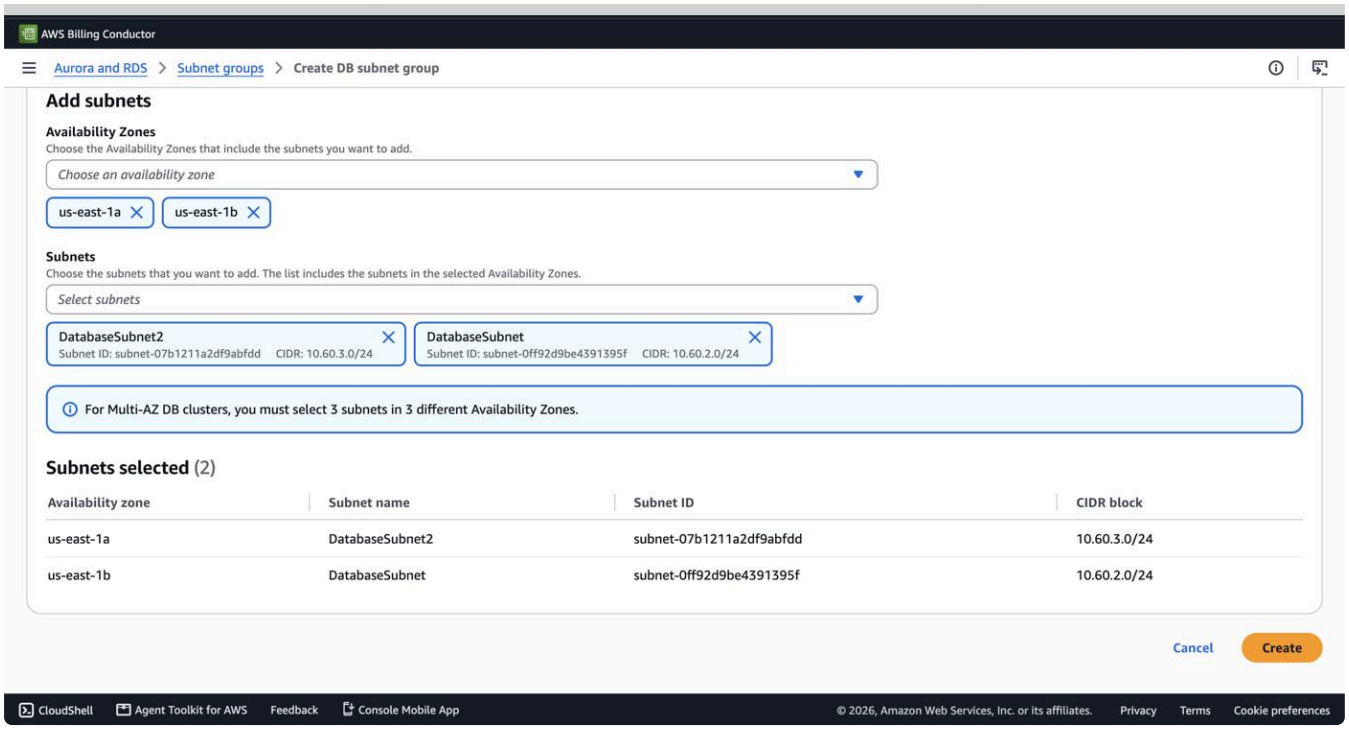
1. Create `DatabaseSubnet2` 10.60.3.0/24 in us-east-1a and associate it with `rt-database`.
2. RDS → Subnet groups → Create `migration-db-subnet-group` with DatabaseSubnet (us-east-1b) and DatabaseSubnet2 (us-east-1a).

```
migration-db-subnet-group
├── DatabaseSubnet   us-east-1b  10.60.2.0/24
└── DatabaseSubnet2  us-east-1a  10.60.3.0/24
```


VPC > Subnets > Create subnet
VPC
VPC ID
Create subnets in this VPC.
vpc-074db46afdba5d9c6 (vpc-migration)
Associated VPC CIDRs
IPv4 CIDRs
10.60.0.0/16
Subnet settings
Specify the CIDR blocks and Availability Zone for the subnet.
Subnet 1 of 1
Subnet name
Create a tag with a key of 'Name' and a value that you specify.
DatabaseSubnet2
The name can be up to 256 characters long.
Availability Zone Info
Choose the zone in which your subnet will reside, or let Amazon choose one for you.
United States (N. Virginia) / use1-az2 (us-east-1a)
IPv4 VPC CIDR block Info
Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

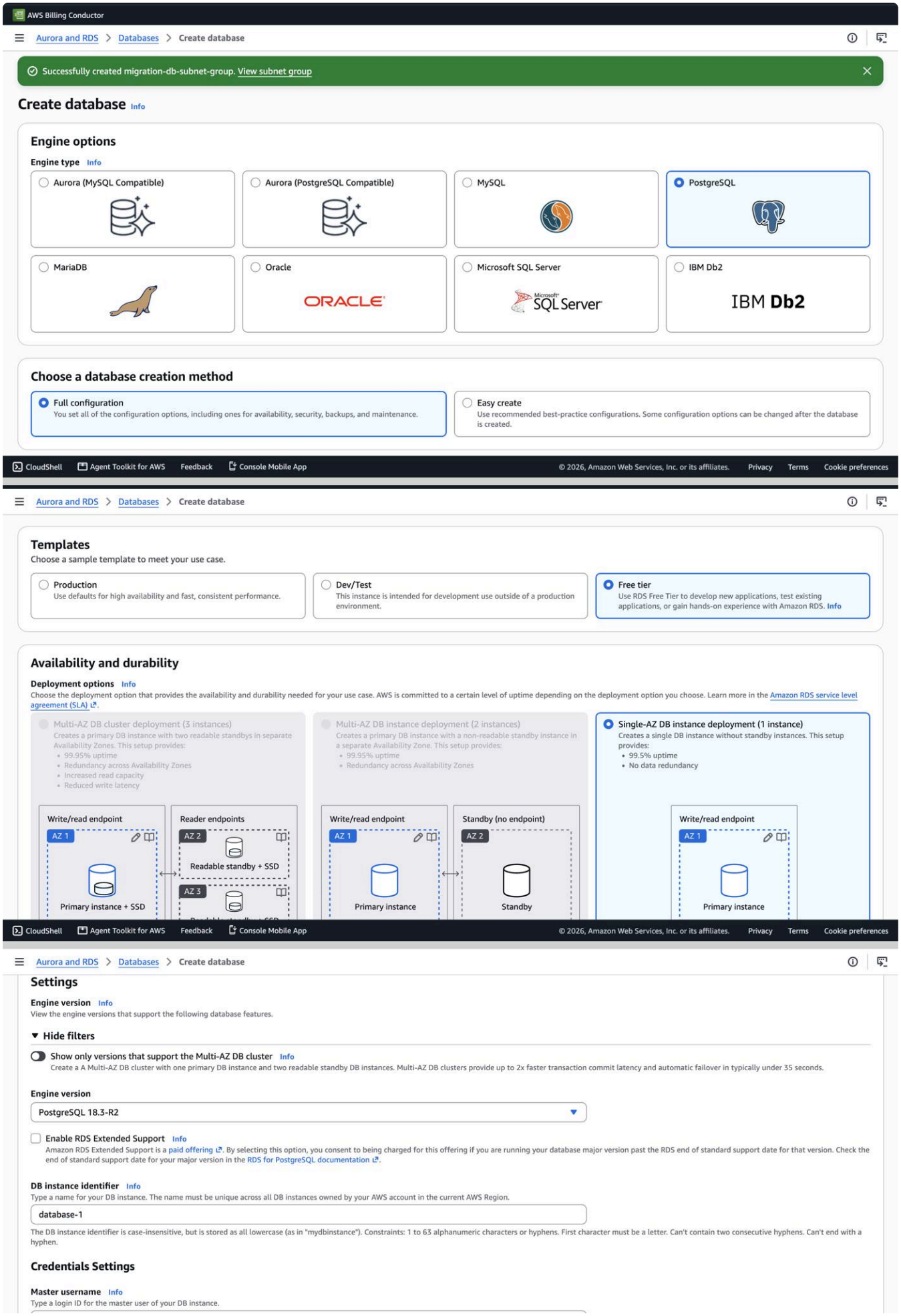

VPC > Route tables > rtb-0e42b6cc627db5cf5
VPC dashboard
Filter by VPC
Choose a VPC
AWS Global View
Virtual private cloud
Your VPCs
Subnets
Route tables
Internet gateways
Egress-only internet gateways
Carrier gateways
DHCP option sets
Elastic IPs
Managed prefix lists
NAT gateways
Peering connections
You have successfully updated subnet associations for rtb-0e42b6cc627db5cf5 / rt-database-2.
rtb-0e42b6cc627db5cf5 / rt-database-2
Actions
Details Info
Route table ID
rtb-0e42b6cc627db5cf5
Main
No
Explicit subnet associations
subnet-07b1211a2df9abfdd / DatabaseSubnet2
Edge associations
–
VPC
vpc-074db46afdba5d9c6 | vpc-migration
Owner ID
211811255273
Routes
Subnet associations
Edge associations
Route propagation
Tags
Related resources
Routes (1)
Both
Edit routes
Filter routes
Destination
Target
Status
Propagated
Route Origin
10.60.0.0/16
local
Active
No
Create Route Table

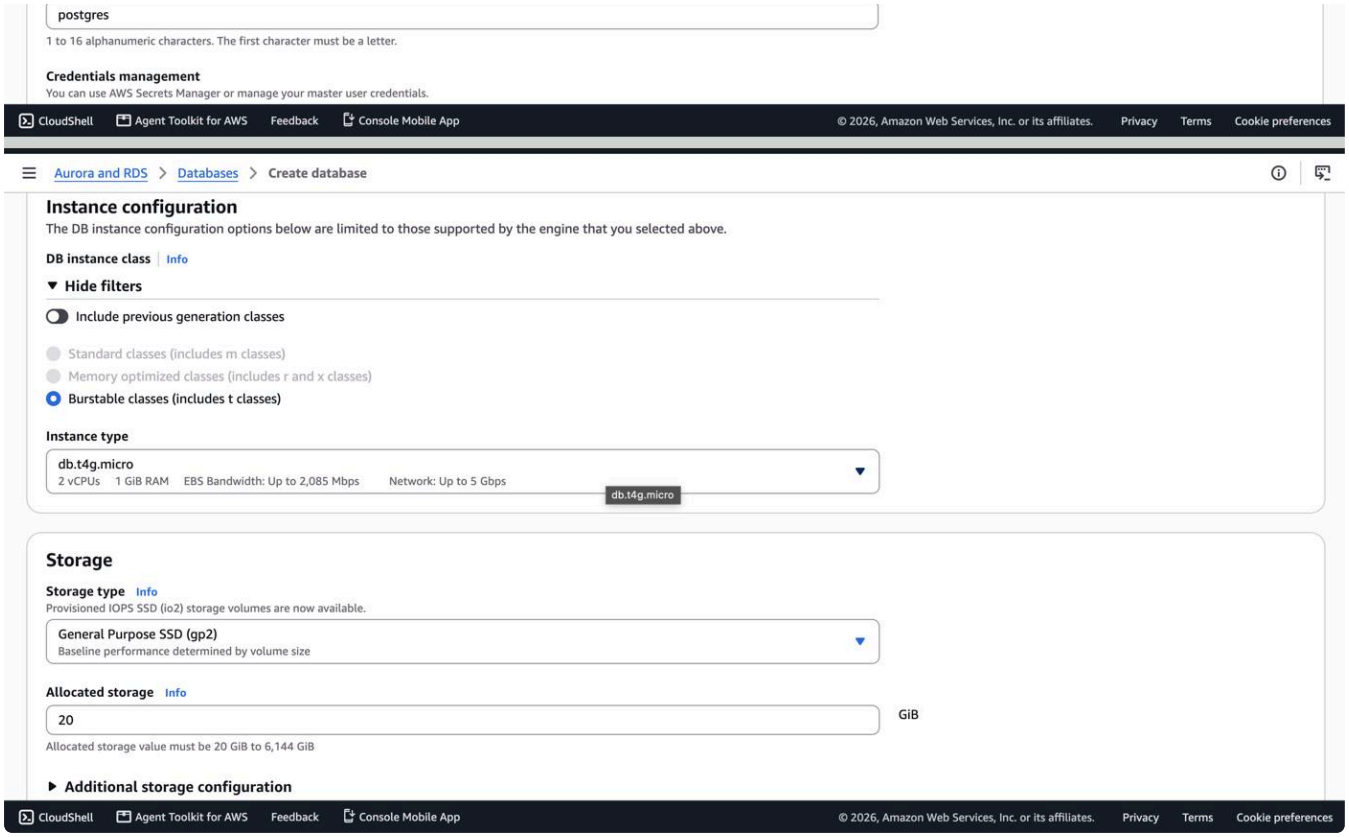

VPC
10.60.0.0/16
us-east-1a
us-east-1b
DatabaseSubnet2
10.60.3.0/24
DatabaseSubnet
10.60.2.0/24
RDS Subnet Group
PostgreSQL RDS



Step 7 — Create the RDS PostgreSQL database

1. RDS → Create database → Full configuration → PostgreSQL.
2. Template Free tier, deployment single-AZ, identifier `database-1`.
3. Instance class db.t4g.micro, 20 GiB gp storage.
4. Connectivity: `vpc-migration`, subnet group `migration-db-subnet-group`, Public access = No, security group `migration-db`.
5. Create. Engine version PostgreSQL 18.3, port 5432.


AWS Billing Conductor
Aurora and RDS > Databases > Create database
Successfully created migration-db-subnet-group. View subnet group
Create database Info
Engine options
Engine type Info
Aurora (MySQL Compatible)
Aurora (PostgreSQL Compatible)
MySQL
PostgreSQL
MariaDB
Oracle
Microsoft SQL Server
IBM Db2
Choose a database creation method
Full configuration
You set all of the configuration options, including ones for availability, security, backups, and maintenance.
Easy create
Use recommended best-practice configurations. Some configuration options can be changed after the database is created.
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
Templates
Choose a sample template to meet your use case.
Production
Use defaults for high availability and fast, consistent performance.
Dev/Test
This instance is intended for development use outside of a production environment.
Free tier
Use RDS Free Tier to develop new applications, test existing applications, or gain hands-on experience with Amazon RDS. Info
Availability and durability
Deployment options Info
Choose the deployment option that provides the availability and durability needed for your use case. AWS is committed to a certain level of uptime depending on the deployment option you choose. Learn more in the Amazon RDS service level agreement (SLA).
Multi-AZ DB cluster deployment (3 instances)
Creates a primary DB instance with two readable standbys in separate Availability Zones. This setup provides:
99.95% uptime
Redundancy across Availability Zones
Increased read capacity
Reduced write latency
Write/read endpoint
Reader endpoints
AZ 1
AZ 2
AZ 3
Readable standby + SSD
Primary instance + SSD
Multi-AZ DB instance deployment (2 instances)
Creates a primary DB instance with a non-readable standby instance in a separate Availability Zone. This setup provides:
99.95% uptime
Redundancy across Availability Zones
Write/read endpoint
Standby (no endpoint)
Primary instance
Standby
Single-AZ DB instance deployment (1 instance)
Creates a single DB instance without standby instances. This setup provides:
99.5% uptime
No data redundancy
Write/read endpoint
Primary instance
Settings
Engine version Info
View the engine versions that support the following database features.
Hide filters
Show only versions that support the Multi-AZ DB cluster Info
Create a A Multi-AZ DB cluster with one primary DB instance and two readable standby DB instances. Multi-AZ DB clusters provide up to 2x faster transaction commit latency and automatic failover in typically under 35 seconds.
Engine version
PostgreSQL 18.3-R2
Enable RDS Extended Support Info
Amazon RDS Extended Support is a paid offering. By selecting this option, you consent to being charged for this offering if you are running your database major version past the RDS end of standard support date for that version. Check the end of standard support date for your major version in the RDS for PostgreSQL documentation.
DB instance identifier Info
Type a name for your DB instance. The name must be unique across all DB instances owned by your AWS account in the current AWS Region.
database-1
The DB instance identifier is case-insensitive, but is stored as all lowercase (as in "mydbinstance"). Constraints: 1 to 63 alphanumeric characters or hyphens. First character must be a letter. Can't contain two consecutive hyphens. Can't end with a hyphen.
Credentials Settings
Master username Info
Type a login ID for the master user of your DB instance.



Step 8 — Database, application user and secret

From the EC2 instance (Session Manager), as the master user:

```
CREATE ROLE migrationadmin LOGIN PASSWORD '********';
GRANT migrationadmin TO postgres;
CREATE DATABASE migration_db OWNER migrationadmin;
```

Then store the credentials in Secrets Manager as `migration/app/database` (keys: username, password, engine, host, port). The application reads the secret through its IAM role; the password is never hardcoded.

Step 9 — S3 source bucket

1. S3 → Create bucket `migration-source-v1` in us-east-1.
2. ACLs disabled, Block all public access on, default encryption on.
3. Upload test data under `application/`, `documents/` and `images/` prefixes.

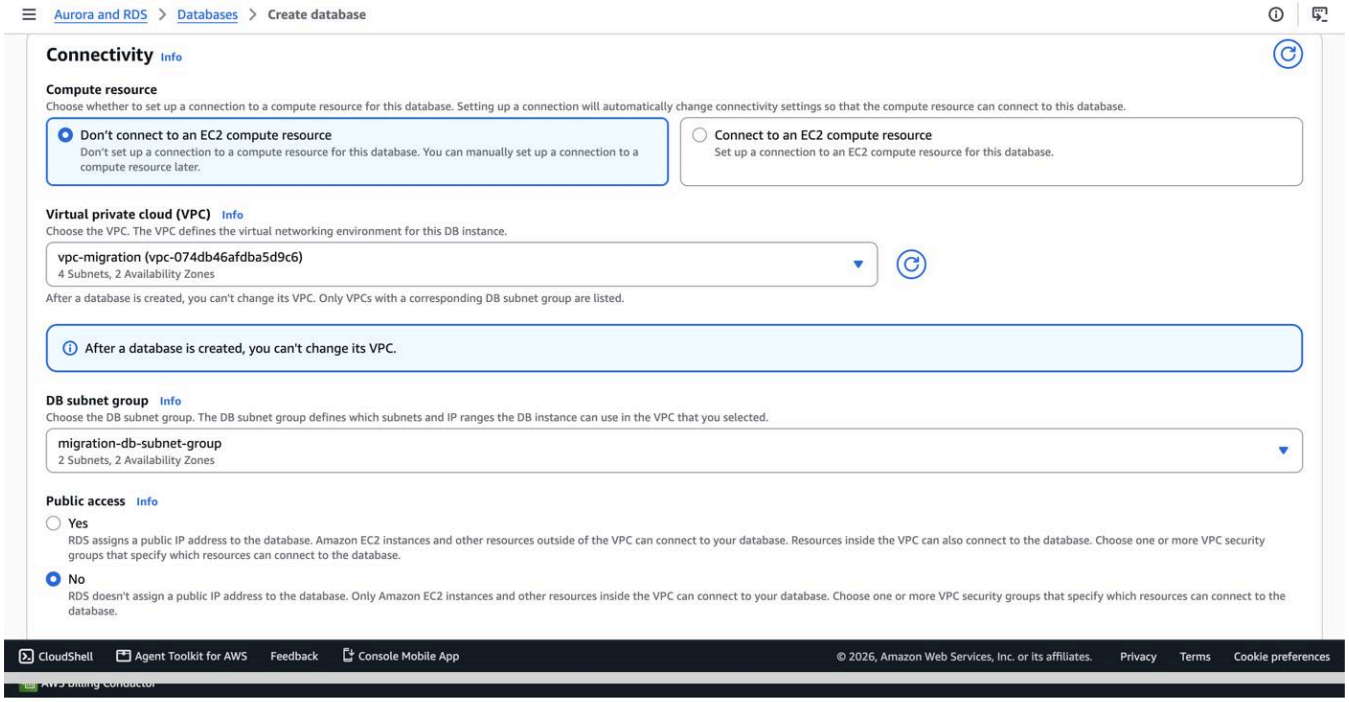
Step 10 — Validate the source environment from EC2

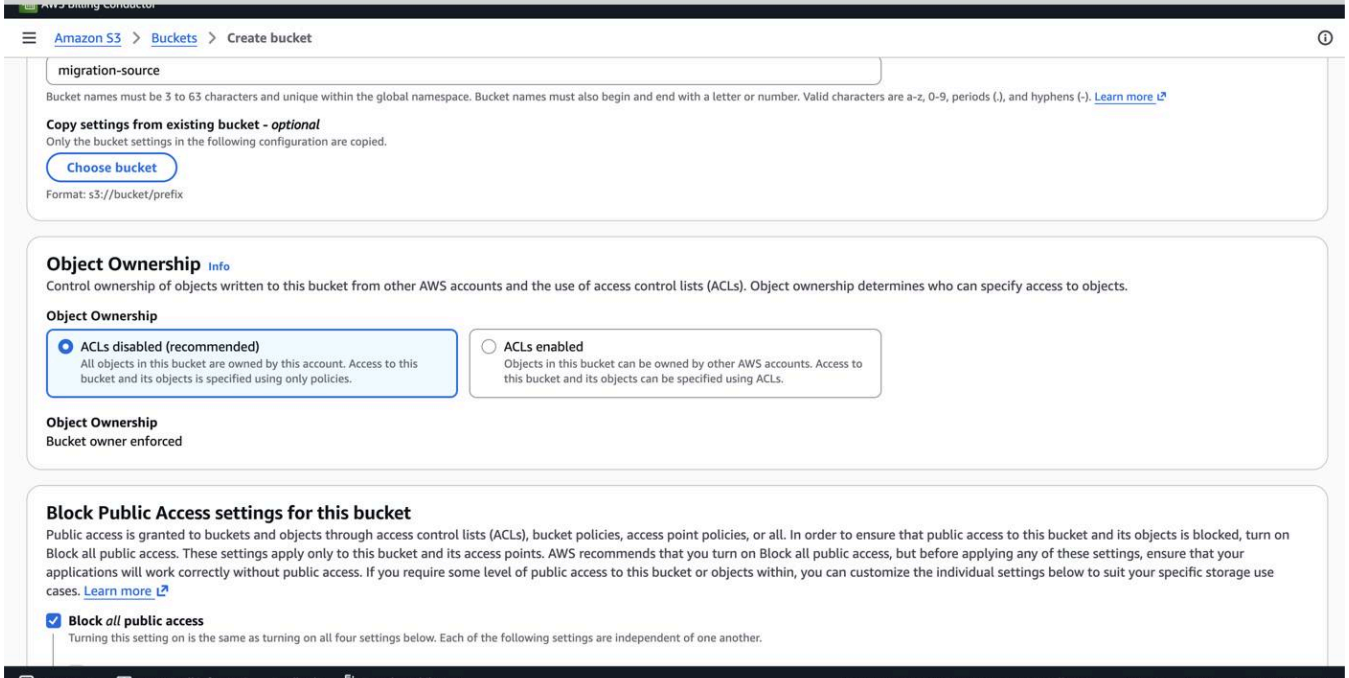
```
# network path EC2 → RDS
nc -zv database-1.<id>.us-east-1.rds.amazonaws.com 5432   # succeeded

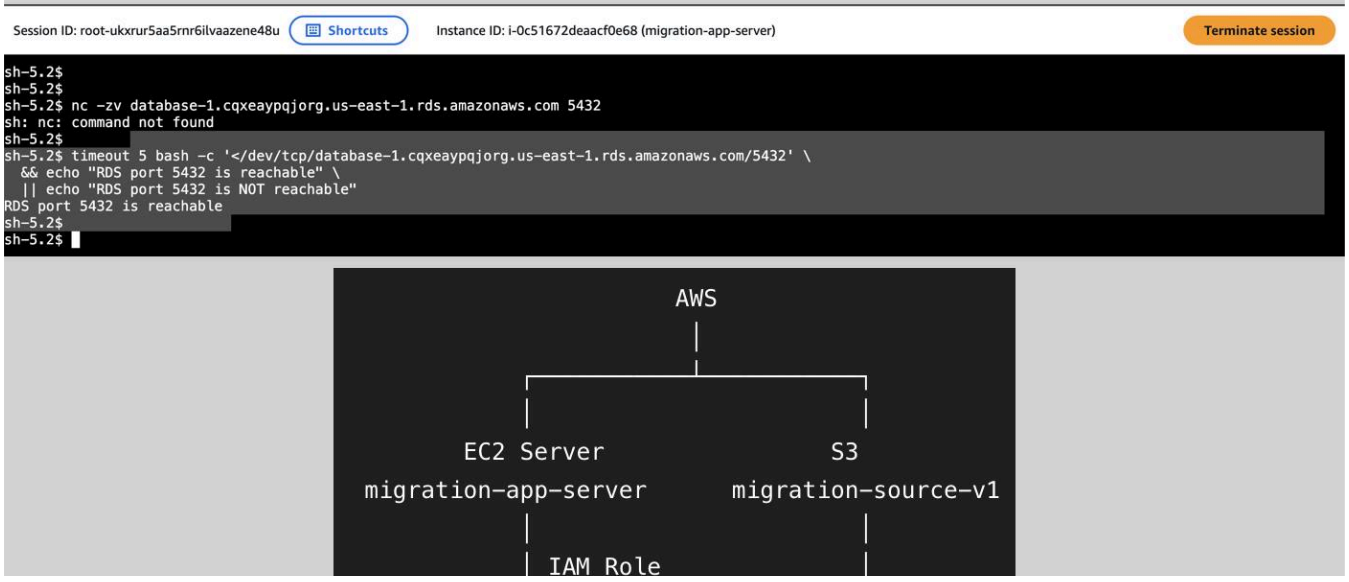
# database login as the app user
psql -h database-1.<id>.us-east-1.rds.amazonaws.com -p 5432 -U migrationadmin
-d migration_db
SELECT current_user;      -- migrationadmin
SELECT current_database(); -- migration_db

# instance identity comes from the IAM role
aws sts get-caller-identity   # assumed-role/migration-ec2-role

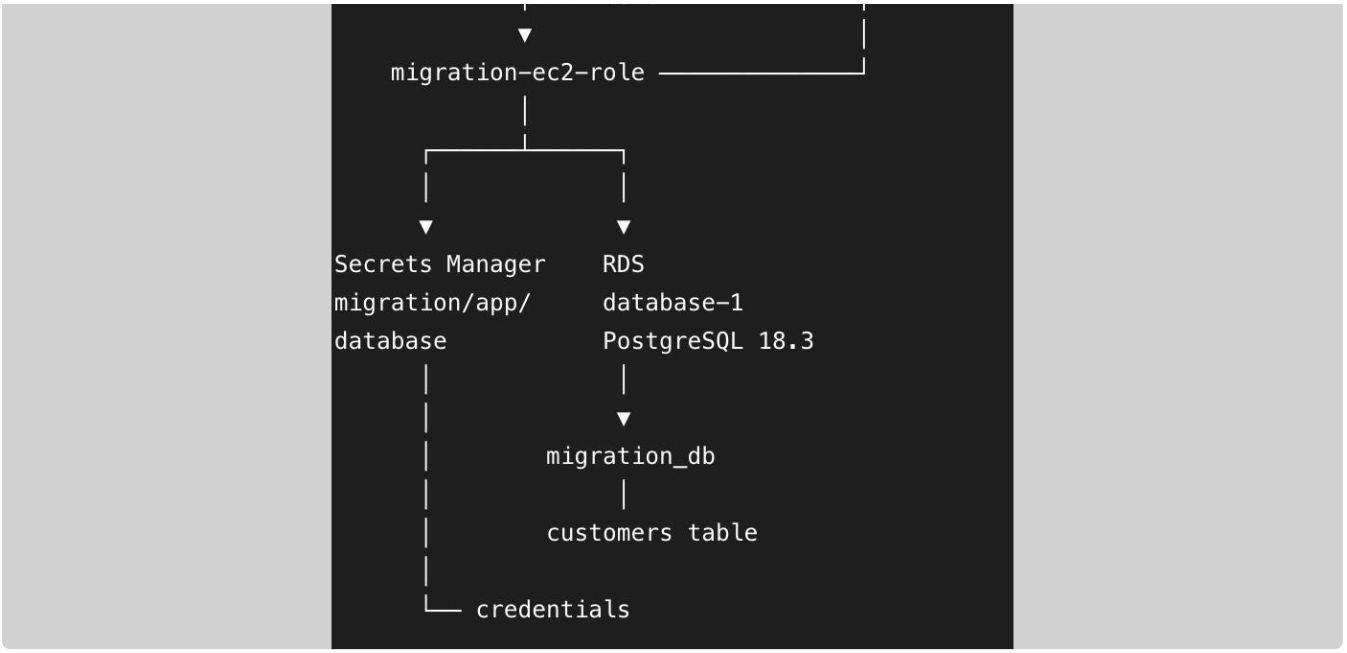
# secret and bucket access through the role
aws secretsmanager get-secret-value --secret-id migration/app/database --
query SecretString --output text
aws s3 ls s3://migration-source-v1   # PRE application/ documents/ images/
```


Aurora and RDS > Databases > Create database
Connectivity Info
Compute resource
Choose whether to set up a connection to a compute resource for this database. Setting up a connection will automatically change connectivity settings so that the compute resource can connect to this database.
Don't connect to an EC2 compute resource
Don't set up a connection to a compute resource for this database. You can manually set up a connection to a compute resource later.
Connect to an EC2 compute resource
Set up a connection to an EC2 compute resource for this database.
Virtual private cloud (VPC) Info
Choose the VPC. The VPC defines the virtual networking environment for this DB instance.
vpc-migration (vpc-074db46afdba5d9c6)
4 Subnets, 2 Availability Zones
After a database is created, you can't change its VPC. Only VPCs with a corresponding DB subnet group are listed.
After a database is created, you can't change its VPC.
DB subnet group Info
Choose the DB subnet group. The DB subnet group defines which subnets and IP ranges the DB instance can use in the VPC that you selected.
migration-db-subnet-group
2 Subnets, 2 Availability Zones
Public access Info
Yes
RDS assigns a public IP address to the database. Amazon EC2 instances and other resources outside of the VPC can connect to your database. Resources inside the VPC can also connect to the database. Choose one or more VPC security groups that specify which resources can connect to the database.
No
RDS doesn't assign a public IP address to the database. Only Amazon EC2 instances and other resources inside the VPC can connect to your database. Choose one or more VPC security groups that specify which resources can connect to the database.
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Amazon S3 > Buckets > Create bucket
migration-source
Bucket names must be 3 to 63 characters and unique within the global namespace. Bucket names must also begin and end with a letter or number. Valid characters are a-z, 0-9, periods (.), and hyphens (-). Learn more
Copy settings from existing bucket - optional
Only the bucket settings in the following configuration are copied.
Choose bucket
Format: s3://bucket/prefix
Object Ownership Info
Control ownership of objects written to this bucket from other AWS accounts and the use of access control lists (ACLs). Object ownership determines who can specify access to objects.
Object Ownership
ACLs disabled (recommended)
All objects in this bucket are owned by this account. Access to this bucket and its objects is specified using only policies.
ACLs enabled
Objects in this bucket can be owned by other AWS accounts. Access to this bucket and its objects can be specified using ACLs.
Object Ownership
Bucket owner enforced
Block Public Access settings for this bucket
Public access is granted to buckets and objects through access control lists (ACLs), bucket policies, access point policies, or all. In order to ensure that public access to this bucket and its objects is blocked, turn on Block all public access. These settings apply only to this bucket and its access points. AWS recommends that you turn on Block all public access, but before applying any of these settings, ensure that your applications will work correctly without public access. If you require some level of public access to this bucket or objects within, you can customize the individual settings below to suit your specific storage use cases. Learn more
Block all public access
Turning this setting on is the same as turning on all four settings below. Each of the following settings are independent of one another.
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Session ID: root-ukxrur5aa5rnr6ilvaazene48u
Shortcuts
Instance ID: i-0c51672deaacf0e68 (migration-app-server)
Terminate session
sh-5.2$
sh-5.2$
sh-5.2$ nc -zv database-1.cqxeaypqjorg.us-east-1.rds.amazonaws.com 5432
sh: nc: command not found
sh-5.2$
sh-5.2$ timeout 5 bash -c '</dev/tcp/database-1.cqxeaypqjorg.us-east-1.rds.amazonaws.com/5432' \
&& echo "RDS port 5432 is reachable" \
|| echo "RDS port 5432 is NOT reachable"
RDS port 5432 is reachable
sh-5.2$
sh-5.2$

AWS
EC2 Server
migration-app-server
S3
migration-source-v1
IAM Role



Validation result:

```
EC2 migration-app-server
| TCP 5432 ✅
RDS database-1
├── PostgreSQL 18.3 ✅
├── User: migrationadmin ✅
├── Database: migration_db ✅
└── SSL/TLS connection ✅
EC2 → IAM role → Secrets Manager ✅ · S3 ✅
```

Step 11 — Bring the AWS side under Terraform

The AWS resources were imported the same way as Azure. A final `terraform apply` changed one setting on the secret (recovery window 30 days) and nothing else:

```
Plan: 0 to add, 1 to change, 0 to destroy.
Apply complete! Resources: 0 added, 1 changed, 0 destroyed.
```

Output	Value
vpc_cidr	10.60.0.0/16
subnet_ids	AppSubnet, DatabaseSubnet, PublicSubnet
route_table_ids	rt-app, rt-database, rt-public
security_group_ids	migration-app, migration-db
ec2_private_ip	10.60.0.117
iam_role_name	migration-ec2-role

Output	Value
rds_port	5432
s3_bucket_name	migration-source-v1
secret_name	migration/app/database

2.7 AWS ↔ Azure Site-to-Site VPN — step by step

The two private networks, AWS 10.60.0.0/16 and Azure 10.50.0.0/16, were joined with an IPsec tunnel over the internet. Traffic was then tested in both directions.

VPN design

```
AWS                                          Azure
vpc-migration 10.60.0.0/16                   vnet-migration 10.50.0.0/16
      |                                             |
Virtual Private Gateway                      VPN Gateway (in GatewaySubnet)
      |                                             |
Site-to-Site VPN connection  == IPsec ==     Connection (IKEv2, shared key)
      |                                             |
Customer Gateway                             Local Network Gateway
(= Azure VPN Gateway public IP)              (= AWS tunnel outside IP +
10.60.0.0/16)
```

Each side needs an object describing the *other* side: AWS models Azure as a **Customer Gateway**; Azure models AWS as a **Local Network Gateway**.

Step 1 — Azure public IP for the gateway

1. Public IP addresses → Create → Standard SKU, static, Central India, resource group `rg-migration-lab`.
2. This becomes the Azure end of the tunnel.

Step 2 — Azure virtual network gateway

1. Virtual network gateways → Create.
2. Gateway type VPN, route-based, region Central India.
3. Virtual network `vnet-migration` (it uses GatewaySubnet 10.50.0.0/24 automatically).
4. Public IP: the one from Step 1. Active-active and BGP disabled for the lab.
5. Create. Provisioning takes around 30–45 minutes.

Step 3 — AWS customer gateway

1. VPC → Customer gateways → Create.

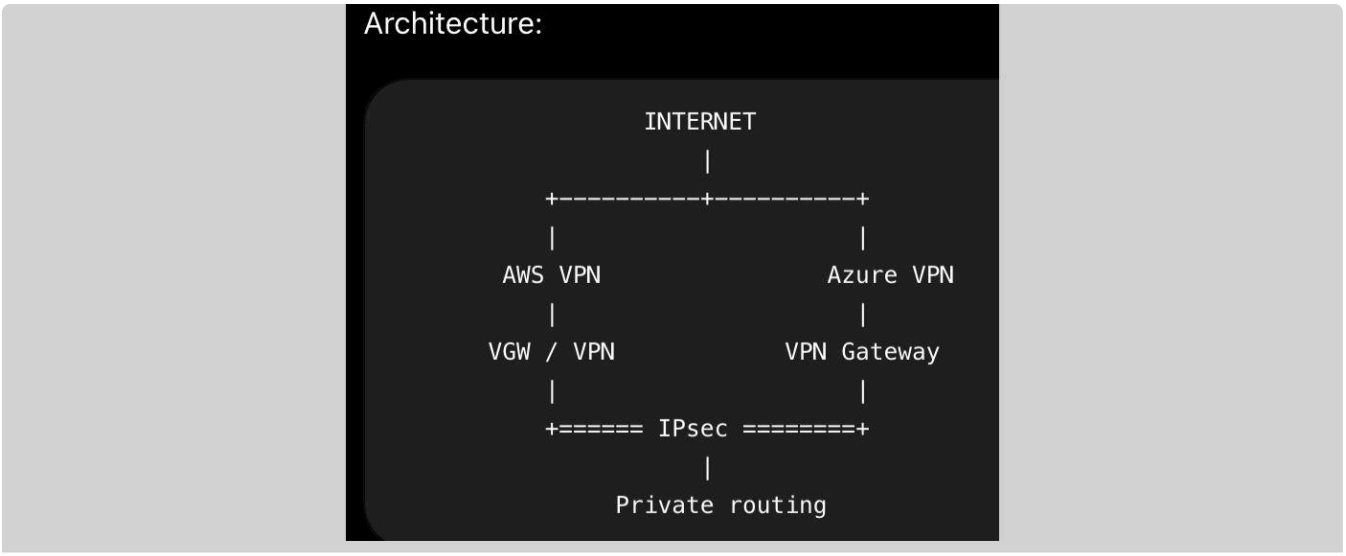
2. Name `cgw-azure-migration`, BGP ASN 65000 (default, static routing), IP address = Azure gateway public IP.

Step 4 — AWS virtual private gateway

1. VPC → Virtual private gateways → Create (Amazon default ASN).
2. Actions → Attach to VPC → `vpc-migration`.

Step 5 — AWS Site-to-Site VPN connection

1. VPC → Site-to-Site VPN connections → Create.
2. Target gateway type: Virtual private gateway; customer gateway: existing `cgw-azure-migration`.
3. Routing: Static; static IP prefix 10.50.0.0/16 (the Azure VNet).
4. Create. AWS builds two tunnels, each with its own outside IP.
5. Download configuration (Generic) to read tunnel outside IPs and pre-shared keys. Keep the keys out of documents and chat.



Home › Network foundation | Public IP addresses

Create public IP address ...

Basics | DDoS Protection | Tags | Review + create

Create a public IP address. Associate it with a virtual machine or other Azure resources. Internet resources communicate to Azure resources through a public IP address. Learn more.

Project details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription * : Azure subscription 1

Resource group * : rg-migration-lab

Create new

Instance details

Region * : (Asia Pacific) Central India

Deploy to an Azure Extended Zone

Configuration details

Name * : pip-vpn-migration

Previous | Next | Review + create

Create virtual network gateway ...

Gateway type * : VPN | ExpressRoute

Gateway type * ⓘ VPN ExpressRoute

SKU * ⓘ VpnGw2AZ

Enable Advanced Connectivity ⓘ ○ Enabled ◉ Disabled

Virtual network * ⓘ vnet-migration

Create virtual network

Subnet ⓘ GatewaySubnet (10.50.0.0/24)

ⓘ Only virtual networks in the currently selected subscription and region are listed.

Public IP address

Public IP address * ⓘ ○ Create new ◉ Use existing

Choose public IP address * pip-vpn-migration (4.186.53.6)

Enable active-active mode * ○ Enabled ◉ Disabled

Configure BGP * ⓘ ○ Enabled ◉ Disabled

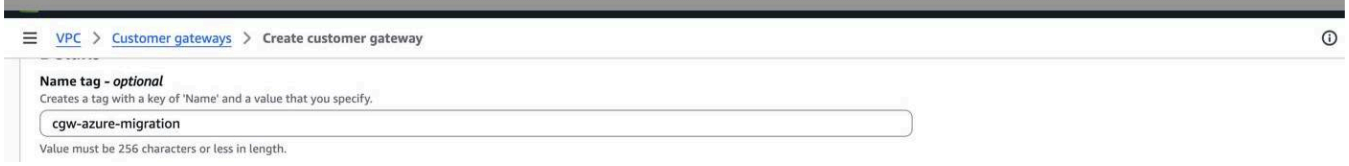
Authentication Information

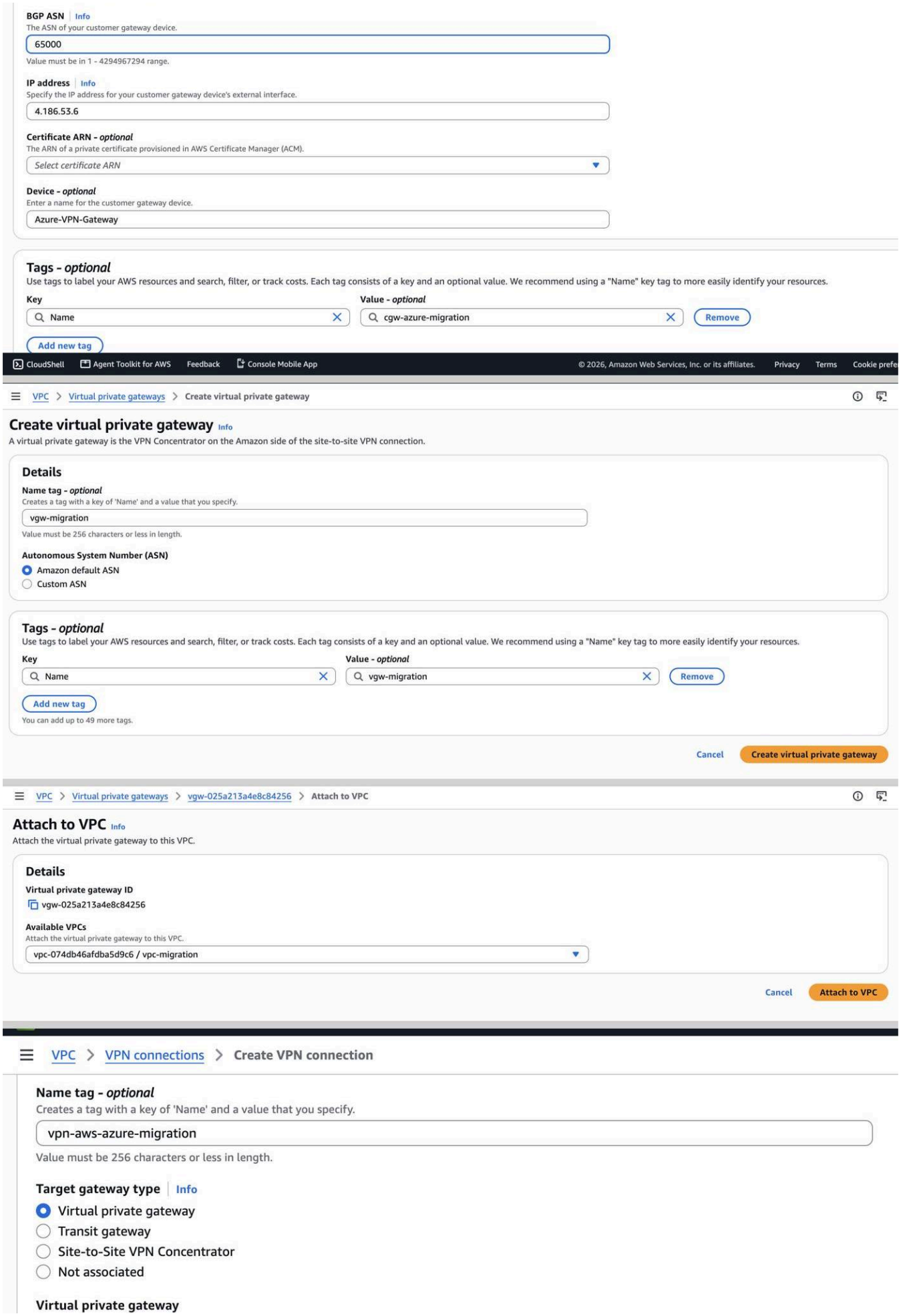
Enable Key Vault Access ⓘ ○ Enabled ◉ Disabled

Azure recommends using a validated VPN device with your virtual network gateway. To view a list of validated devices and instructions for configuration, refer to Azure's documentation regarding validated VPN devices.

Review + create | Previous | Next : Tags > | Download a template for automation

```
Azure
10.50.0.0/16
     |
Azure VPN Gateway
     |
     | IPsec
     |
AWS Customer Gateway
     |
AWS VPN
     |
Virtual Private Gateway
     |
vpc-migration
10.60.0.0/16
```




BGP ASN Info
The ASN of your customer gateway device.
65000
Value must be in 1 - 4294967294 range.
IP address Info
Specify the IP address for your customer gateway device's external interface.
4.186.53.6
Certificate ARN - optional
The ARN of a private certificate provisioned in AWS Certificate Manager (ACM).
Select certificate ARN
Device - optional
Enter a name for the customer gateway device.
Azure-VPN-Gateway
Tags - optional
Use tags to label your AWS resources and search, filter, or track costs. Each tag consists of a key and an optional value. We recommend using a "Name" key tag to more easily identify your resources.
Key
Name
Value - optional
cgw-azure-migration
Remove
Add new tag
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie prefe
VPC > Virtual private gateways > Create virtual private gateway
Create virtual private gateway Info
A virtual private gateway is the VPN Concentrator on the Amazon side of the site-to-site VPN connection.
Details
Name tag - optional
Creates a tag with a key of 'Name' and a value that you specify.
vgw-migration
Value must be 256 characters or less in length.
Autonomous System Number (ASN)
Amazon default ASN
Custom ASN
Tags - optional
Use tags to label your AWS resources and search, filter, or track costs. Each tag consists of a key and an optional value. We recommend using a "Name" key tag to more easily identify your resources.
Key
Name
Value - optional
vgw-migration
Remove
Add new tag
You can add up to 49 more tags.
Cancel
Create virtual private gateway
VPC > Virtual private gateways > vgw-025a213a4e8c84256 > Attach to VPC
Attach to VPC Info
Attach the virtual private gateway to this VPC.
Details
Virtual private gateway ID
vgw-025a213a4e8c84256
Available VPCs
Attach the virtual private gateway to this VPC.
vpc-074db46afdba5d9c6 / vpc-migration
Cancel
Attach to VPC
VPC > VPN connections > Create VPN connection
Name tag - optional
Creates a tag with a key of 'Name' and a value that you specify.
vpn-aws-azure-migration
Value must be 256 characters or less in length.
Target gateway type Info
Virtual private gateway
Transit gateway
Site-to-Site VPN Concentrator
Not associated
Virtual private gateway

vgw-025a213a4e8c84256

Customer gateway | Info

- (•) Existing
- ( ) New

Customer gateway ID

cgw-038acc8620e87150a

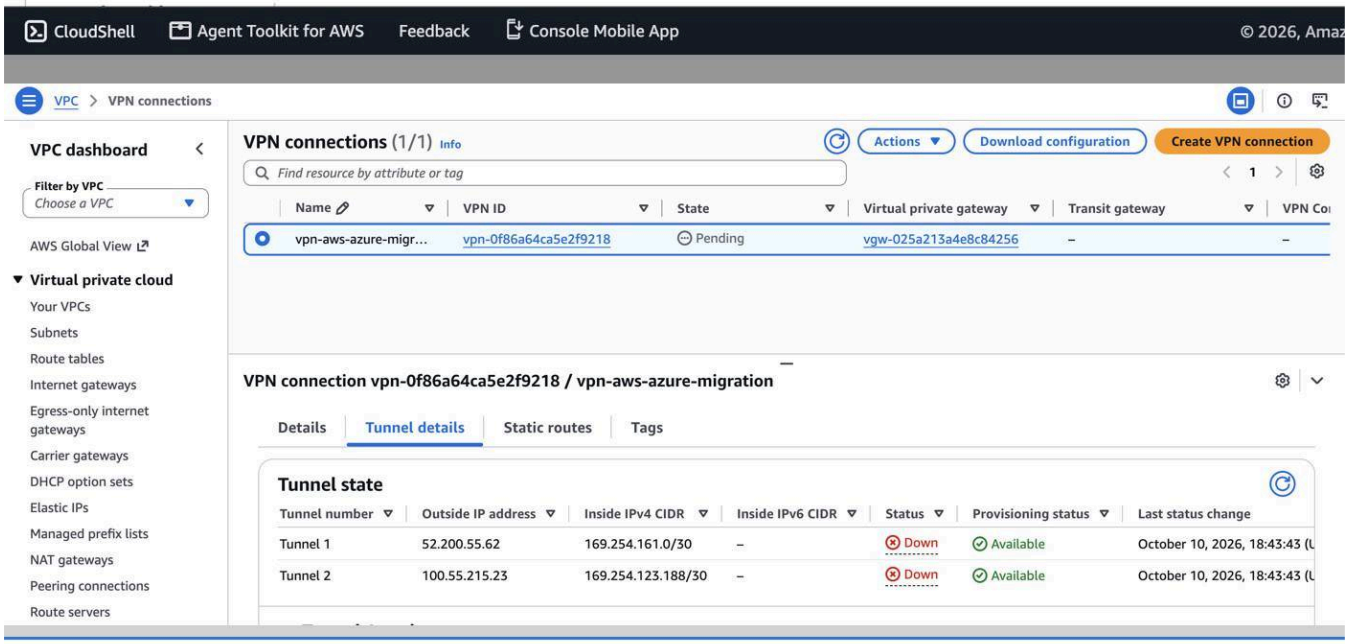
Routing options | Info

- ( ) Dynamic (requires BGP)
- (•) Static

Static IP prefixes | Info

Add static IP prefix

10.50.0.0/16 ×



Home › Hybrid connectivity | Local network gateways

Create local network gateway

Basics | Advanced | Review + create

A local network gateway is a specific object that represents an on-premises location (the site) for routing purposes. Learn more

Project details

Subscription * : Azure subscription 1

Resource group * : rg-migration-lab

Create new

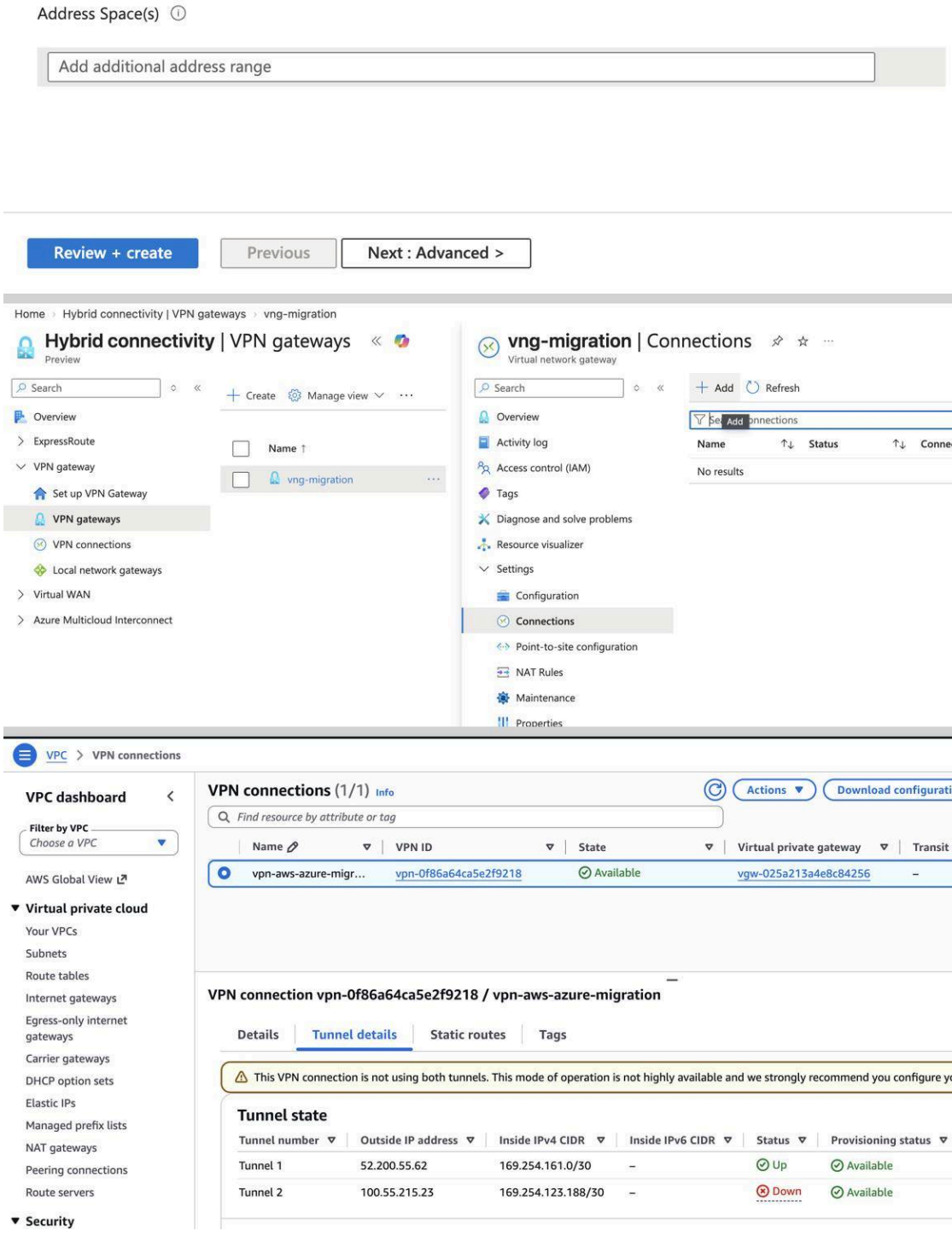
Instance details

Region * : Central India

Name * : lng-aws-migration-tunnel1

Endpoint : IP address | FQDN

IP address * : 52.200.55.62



Step 6 — Azure local network gateway (represents AWS)

1. Local network gateways → Create, Central India, `rg-migration-lab`.
2. Endpoint: IP address = AWS tunnel 1 outside IP.
3. Address space: 10.60.0.0/16.

Step 7 — Azure connection

1. Open the virtual network gateway → Connections → Add.
2. Type Site-to-site (IPsec), name `connection-aws-migration-tunnel1`.
3. Local network gateway from Step 6, shared key = AWS tunnel 1 pre-shared key, IKEv2.

4. Create, then wait until status shows Connected.

Step 8 — Check tunnel status on AWS

VPN connection → Tunnel details: Tunnel 1 status **Up**. Tunnel 2 stays Down because only one Azure connection was created in the lab; production would configure both tunnels (or active-active) for redundancy.

Step 9 — Routing on AWS

1. Route tables → `rt-public` / `rt-app` → Edit routes.
2. Add 10.50.0.0/16 → target Virtual Private Gateway (or enable route propagation from the VGW).
3. Azure needs no manual route: gateway route propagation on `rt-application` / `rt-database` learns 10.60.0.0/16.

Step 10 — Azure test VM

1. Virtual machines → Create `vm-azure-connectivity-test`, small Ubuntu size.
2. Network: `vnet-migration` / ApplicationSubnet, no public IP needed.
3. NSG rule on the VM/subnet: allow inbound from 10.60.0.0/16 on the test ports.

Step 11 — Allow Azure traffic on AWS

Edit inbound rules of `migration-app`: allow SSH (22) and TCP 8000 from 10.50.0.0/16.

Step 12 — Test both directions

```
# AWS EC2 → Azure VM (private IPs only, over the tunnel)
ping <azure-vm-private-ip>
nc -zv <azure-vm-private-ip> 22

# Azure VM → AWS EC2: start a test web server on EC2
python3 -m http.server 8000 --bind 10.60.0.117
# from Azure Portal → VM → Run command
curl http://10.60.0.117:8000      # directory listing returned
```

Home › Hybrid connectivity | VPN gateways › vng-migration | Connections

Create connection ...

1 Basics 2 Settings 3 Monitoring 4 Tags 5 Review + create

Create a secure connection to your virtual network by using VPN Gateway or ExpressRoute.
Learn more about VPN Gateway
Learn more about ExpressRoute

Project details

Subscription * : Azure subscription 1

Resource group * : rg-migration-lab
Create new

Instance details

Connection type * : Site-to-site (IPsec)

Name * : connection-aws-migration-tunnel1

Region * : Central India

Home › Hybrid connectivity | VPN gateways › vng-migration | Connections

Create connection ...

✓ Basics 2 Settings 3 Monitoring 4 Tags 5 Review + create

Virtual network gateway

To use a virtual network with a connection, it must be associated to a virtual network gateway.

Virtual network gateway * : vng-migration

Local network gateway * : lng-aws-migration-tunnel1

Authentication Method : (•) Shared Key(PSK) ( ) Key Vault Certificate

Shared Key(PSK) * : ••••••••••••••••••••••••••••••••

IKE Protocol : ( ) IKEv1 (•) IKEv2

Use Azure Private IP Address : [ ]

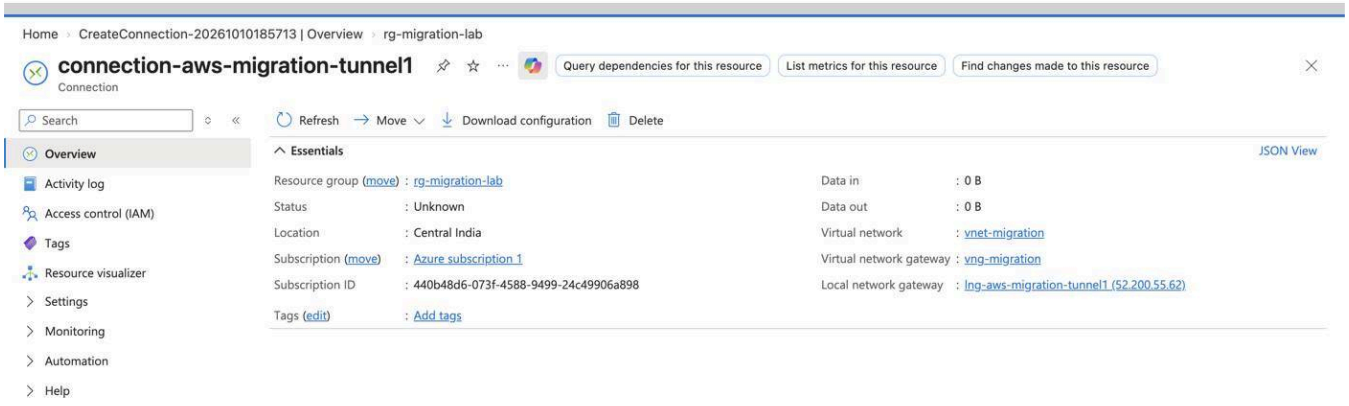
Enable BGP : [ ]

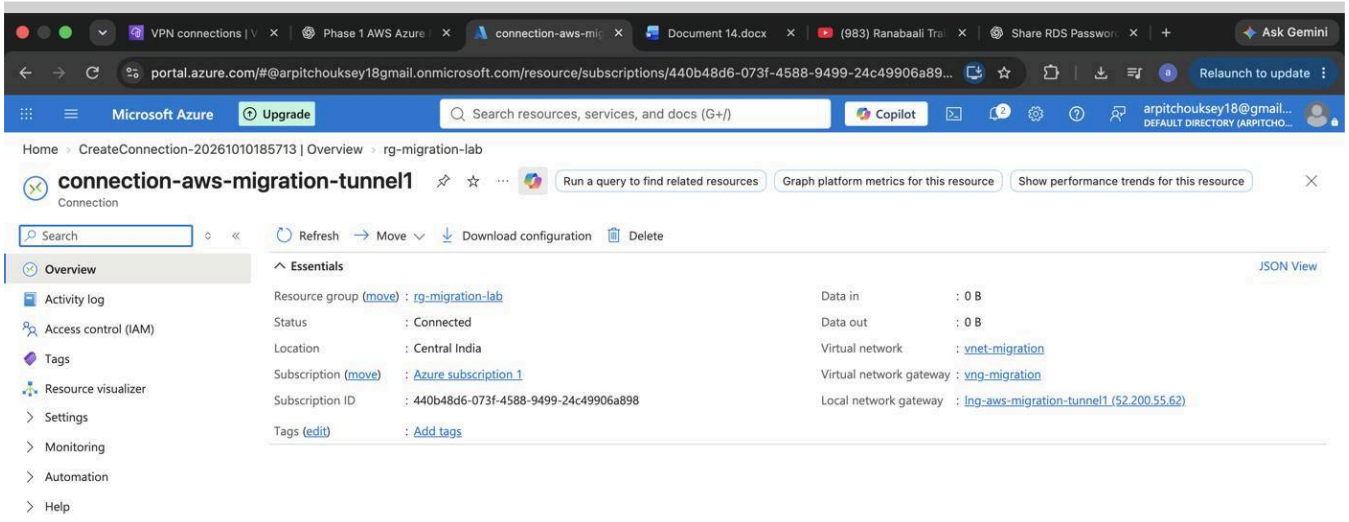
IPsec / IKE policy : Default | Custom

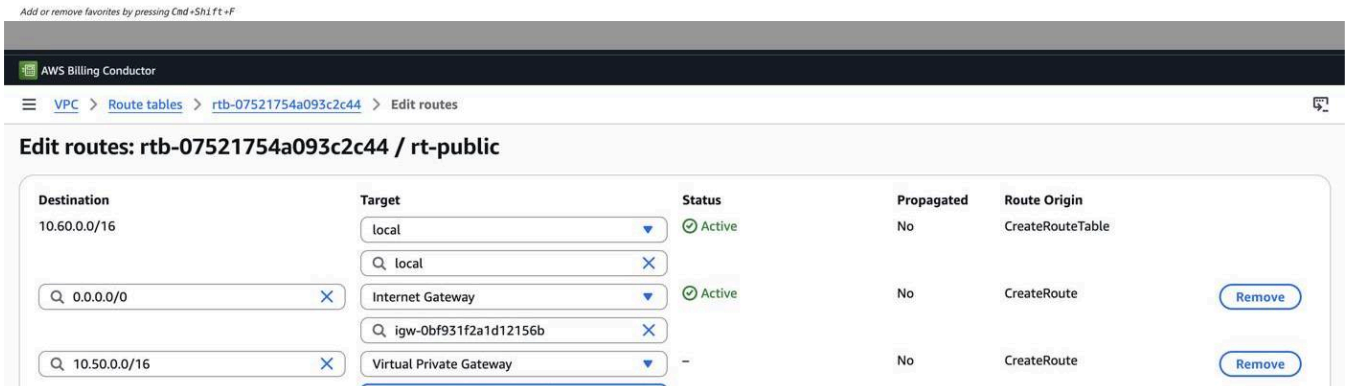
Use policy based traffic selector : Enable | Disable

DPD timeout in seconds * : 45


Connection Mode
Default
InitiatorOnly
ResponderOnly
NAT Rules Associations
Previous
Next : Monitoring >


Home › CreateConnection-20261010185713 | Overview › rg-migration-lab
connection-aws-migration-tunnel1
Connection
Query dependencies for this resource
List metrics for this resource
Find changes made to this resource
Search
Refresh
Move
Download configuration
Delete
Overview
Activity log
Access control (IAM)
Tags
Resource visualizer
Settings
Monitoring
Automation
Help
Essentials
JSON View
Resource group (move) : rg-migration-lab
Status : Unknown
Location : Central India
Subscription (move) : Azure subscription 1
Subscription ID : 440b48d6-073f-4588-9499-24c49906a898
Tags (edit) : Add tags
Data in : 0 B
Data out : 0 B
Virtual network : vnet-migration
Virtual network gateway : vng-migration
Local network gateway : lng-aws-migration-tunnel1 (52.200.55.62)


Ask Gemini
portal.azure.com/#@arpitchouksey18gmail.onmicrosoft.com/resource/subscriptions/440b48d6-073f-4588-9499-24c49906a89...
Relaunch to update
Microsoft Azure
Upgrade
Search resources, services, and docs (G+/)
Copilot
arpitchouksey18@gmail...
DEFAULT DIRECTORY (ARPITCHO...
Home › CreateConnection-20261010185713 | Overview › rg-migration-lab
connection-aws-migration-tunnel1
Connection
Run a query to find related resources
Graph platform metrics for this resource
Show performance trends for this resource
Search
Refresh
Move
Download configuration
Delete
Overview
Activity log
Access control (IAM)
Tags
Resource visualizer
Settings
Monitoring
Automation
Help
Essentials
JSON View
Resource group (move) : rg-migration-lab
Status : Connected
Location : Central India
Subscription (move) : Azure subscription 1
Subscription ID : 440b48d6-073f-4588-9499-24c49906a898
Tags (edit) : Add tags
Data in : 0 B
Data out : 0 B
Virtual network : vnet-migration
Virtual network gateway : vng-migration
Local network gateway : lng-aws-migration-tunnel1 (52.200.55.62)


Add or remove favorites by pressing Cmd+Shift+F
AWS Billing Conductor
VPC › Route tables › rtb-07521754a093c2c44 › Edit routes
Edit routes: rtb-07521754a093c2c44 / rt-public
Destination
Target
Status
Propagated
Route Origin
10.60.0.0/16
local
Active
No
CreateRouteTable
local
0.0.0.0/0
Internet Gateway
Active
No
CreateRoute
Remove
igw-0bf931f2a1d12156b
10.50.0.0/16
Virtual Private Gateway
–
No
CreateRoute
Remove



Home › Compute infrastructure | Virtual machines

Create a virtual machine ...

Basics | Disks | Networking | Management | Monitoring | Advanced | Tags | Review + create

ⓘ This is a preview of a new Create-VM experience. Features are limited in this experience and all feedback is appreciated. Click here to access the previous experience.

Create a virtual machine that runs Linux or Windows. Select an image from Azure marketplace or use your own customized image. Complete the Basics tab then Review + create to provision a virtual machine with default parameters or review each tab for full customization. Learn more

Project details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription * ⓘ: Azure subscription 1

Resource group * ⓘ: rg-migration-lab
Create new

Instance details

Virtual machine name * ⓘ: vm-azure-connectivity-test

Region * ⓘ: (Asia Pacific) Central India
Deploy to an Azure Extended Zone

Availability options ⓘ: Availability zone

Previous | Next | Review + create

Home › Compute infrastructure | Virtual machines

Create a virtual machine ...

Basics | Disks | **Networking** | Management | Monitoring | Advanced | Tags | Review + create

Virtual network

Azure Virtual Network (VNet) enables many types of Azure resources to securely communicate with each other, the internet, and on-premises networks. Learn more about Vnets

Virtual network *: vnet-migration *

Subnet *: ApplicationSubnet - 10.50.1.0/24

Network interface

A network interface enables an Azure virtual machine to communicate with internet, Azure, and on-premises resources. A VM can have one or more network interfaces.

Public IP ⓘ: (new) vm-azure-connectivity-test-ip

Create new

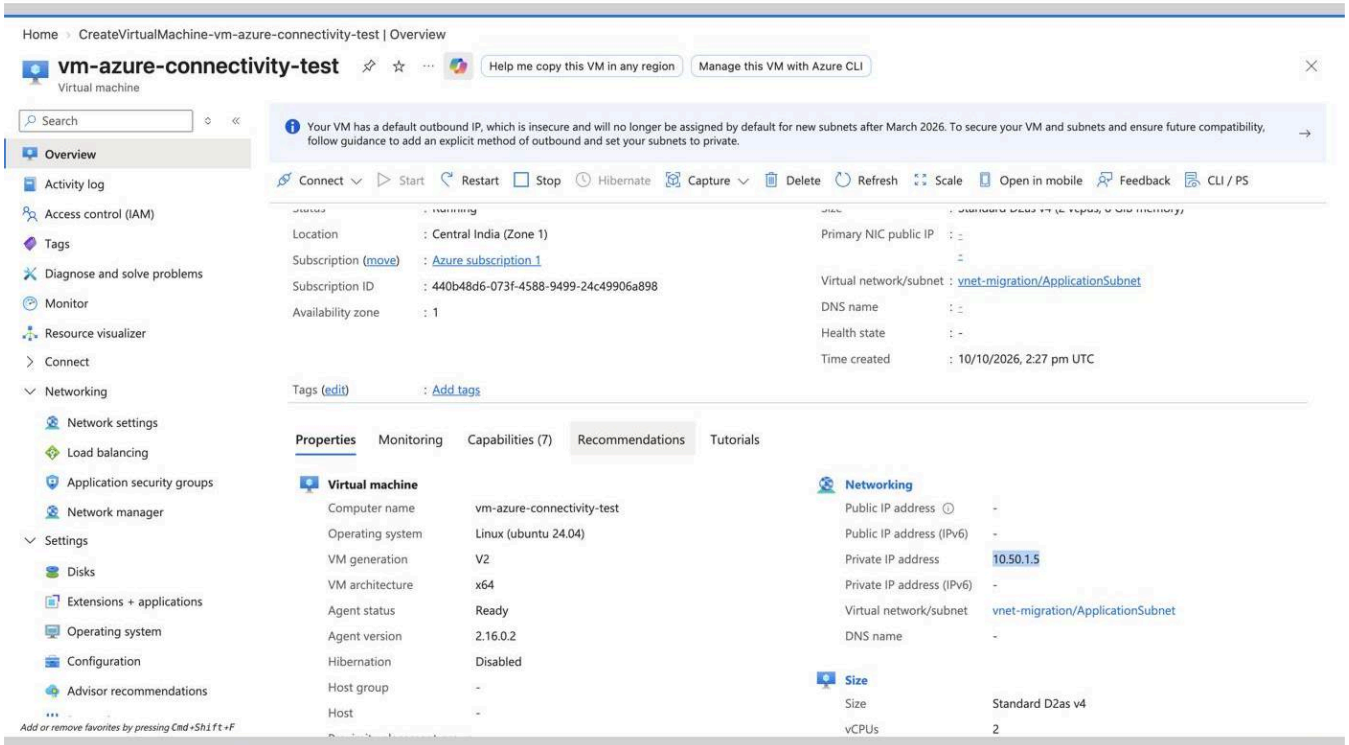
ⓘ Public IP addresses have a nominal charge. Learn more

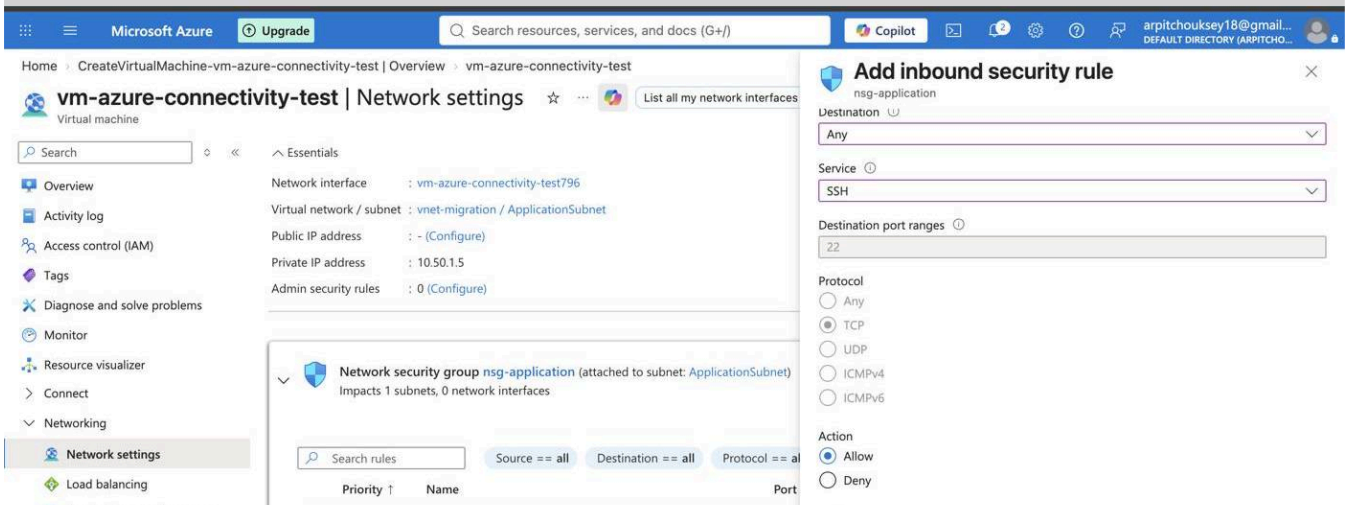
NIC network security group ⓘ

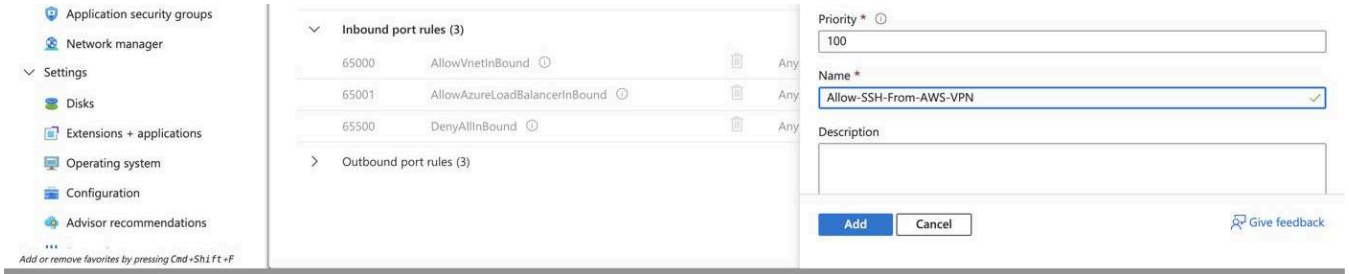
- (●) None
- ( ) Basic
- ( ) Advanced

ⓘ The selected subnet 'ApplicationSubnet - 10.50.1.0/24' is already associated to a network security group 'nsg-application'. We recommend

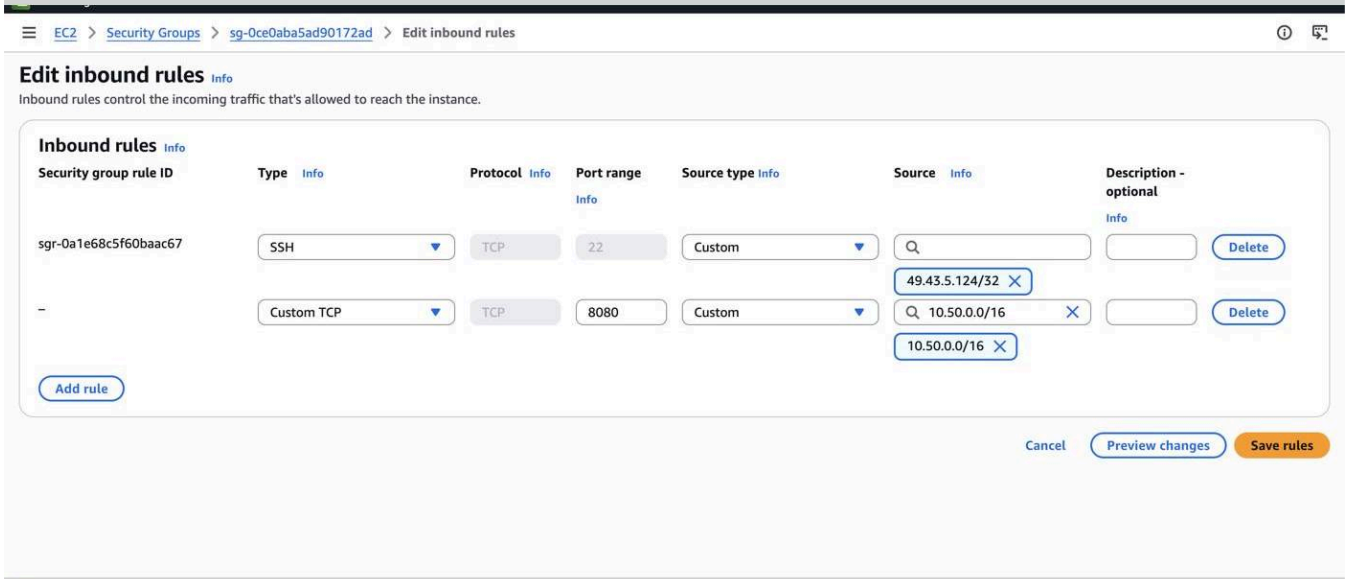
Previous | Next | Review + create

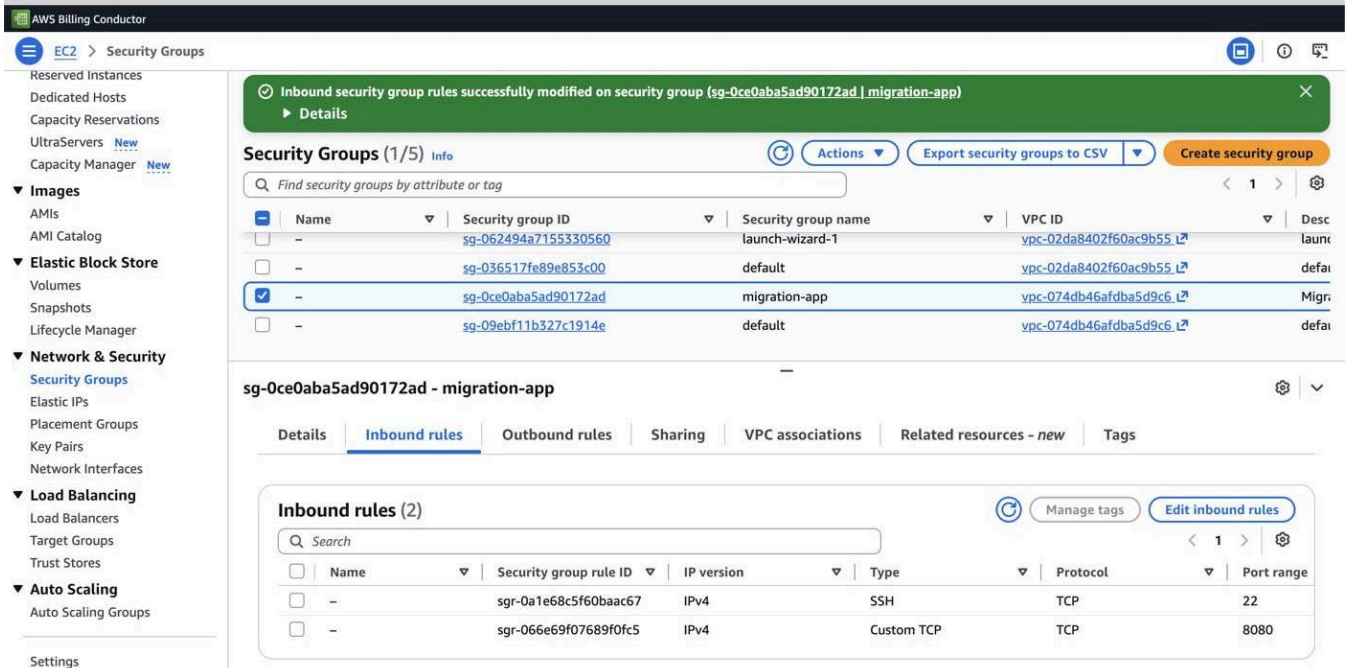


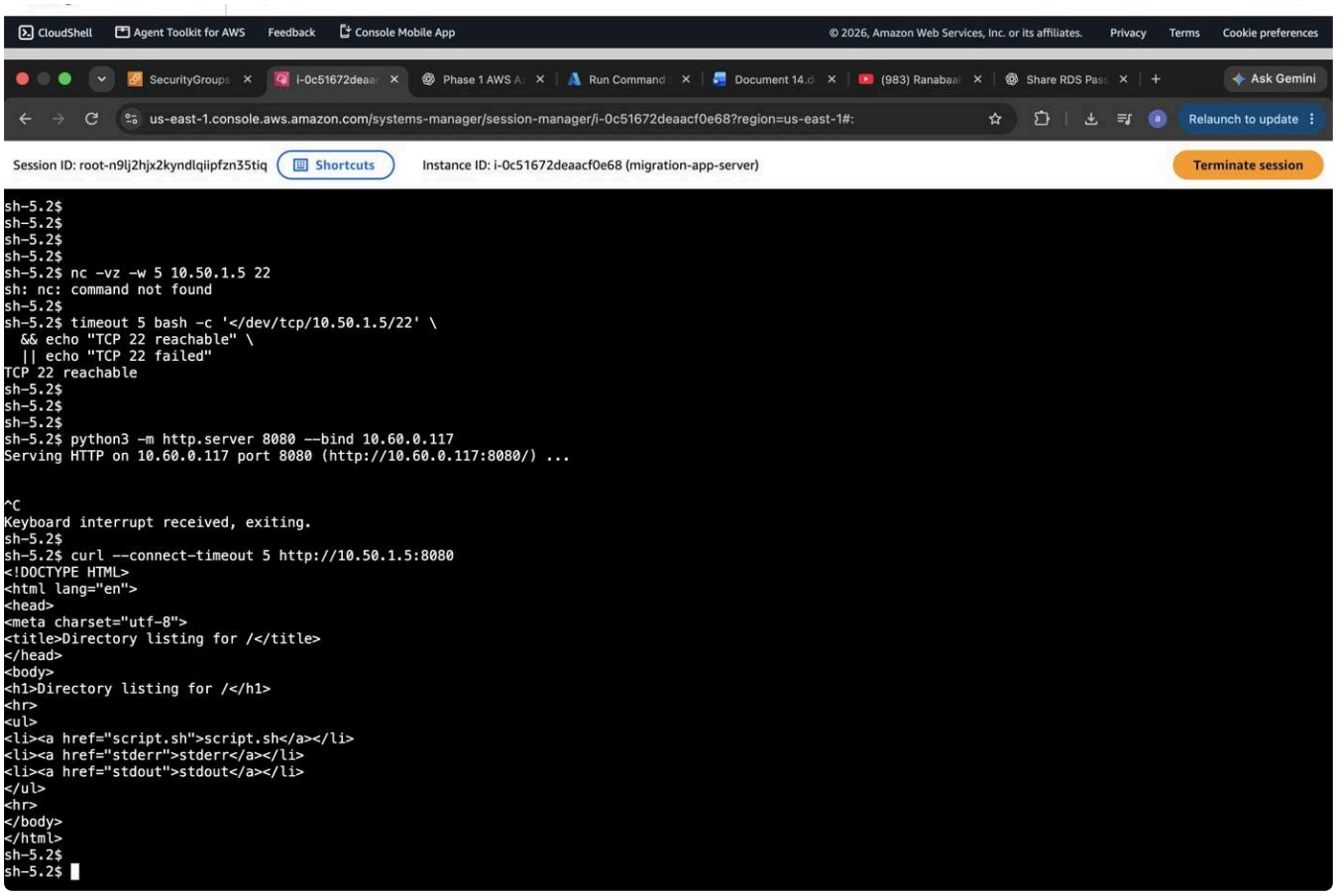



Application security groups
Network manager
Settings
Disks
Extensions + applications
Operating system
Configuration
Advisor recommendations
Add or remove favorites by pressing Cmd+Shift+F
Inbound port rules (3)
65000 AllowVnetInBound
65001 AllowAzureLoadBalancerInBound
65500 DenyAllInBound
Outbound port rules (3)
Priority * 100
Name * Allow-SSH-From-AWS-VPN
Description
Add
Cancel
Give feedback


us-east-1.console.aws.amazon.com/systems-manager/session-manager/i-0c51672deaacf0e68?region=us-east-1#:
Relaunch to update
Session ID: root-n9lj2hjx2kyndlqiipfzn35tiq
Shortcuts
Instance ID: i-0c51672deaacf0e68 (migration-app-server)
Terminate session
sh-5.2$
sh-5.2$
sh-5.2$
sh-5.2$
sh-5.2$ nc -vz -w 5 10.50.1.5 22
sh: nc: command not found
sh-5.2$
sh-5.2$ timeout 5 bash -c '</dev/tcp/10.50.1.5/22' \
&& echo "TCP 22 reachable" \
|| echo "TCP 22 failed"
TCP 22 reachable
sh-5.2$
sh-5.2$


EC2 > Security Groups > sg-0ce0aba5ad90172ad > Edit inbound rules
Edit inbound rules Info
Inbound rules control the incoming traffic that's allowed to reach the instance.
Inbound rules Info
Security group rule ID
Type Info
Protocol Info
Port range Info
Source type Info
Source Info
Description - optional Info
sgr-0a1e68c5f60baac67
SSH
TCP
22
Custom
49.43.5.124/32
Delete
–
Custom TCP
TCP
8080
Custom
10.50.0.0/16
10.50.0.0/16
Delete
Add rule
Cancel
Preview changes
Save rules


AWS Billing Conductor
EC2 > Security Groups
Reserved Instances
Dedicated Hosts
Capacity Reservations
UltraServers New
Capacity Manager New
Images
AMIs
AMI Catalog
Elastic Block Store
Volumes
Snapshots
Lifecycle Manager
Network & Security
Security Groups
Elastic IPs
Placement Groups
Key Pairs
Network Interfaces
Load Balancing
Load Balancers
Target Groups
Trust Stores
Auto Scaling
Auto Scaling Groups
Settings
Inbound security group rules successfully modified on security group (sg-0ce0aba5ad90172ad | migration-app)
Details
Security Groups (1/5) Info
Actions
Export security groups to CSV
Create security group
Find security groups by attribute or tag
Name
Security group ID
Security group name
VPC ID
sg-062494a7155330560
launch-wizard-1
sg-036517fe89e853c00
default
sg-0ce0aba5ad90172ad
migration-app
sg-09ebf11b327c1914e
default
sg-0ce0aba5ad90172ad - migration-app
Details
Inbound rules
Outbound rules
Sharing
VPC associations
Related resources - new
Tags
Inbound rules (2)
Manage tags
Edit inbound rules
Search
Name
Security group rule ID
IP version
Type
Protocol
Port range
sgr-0a1e68c5f60baac67
IPv4
SSH
TCP
22
sgr-066e69f07689f0fc5
IPv4
Custom TCP
TCP
8080



Result

Test	Path	Result
Tunnel state	AWS VPN tunnel 1 ↔ Azure connection	Up / Connected
AWS → Azure	EC2 10.60.0.117 → Azure VM	Reachable
Azure → AWS	Azure VM → EC2 :8000	HTTP response received
Isolation	Only 10.50.0.0/16 ↔ 10.60.0.0/16	No public exposure

Troubleshooting checklist used

1. Tunnel Down → check outside IPs, shared key, IKE version on both sides.
2. Tunnel Up but no traffic → check route tables (AWS route to VGW, Azure propagation).
3. Routes fine but blocked → check SG on AWS and NSG on Azure, both directions.
4. Still blocked → check OS firewall and that the service listens on the private IP.

2.8 Identity and access (sub-phase 5)

Enterprises keep one identity source and federate it into each cloud. Here Microsoft Entra ID is the identity provider and AWS IAM Identity Center is the service provider.

```
Microsoft Entra ID
      |  SAML (sign-in)  +  SCIM (user/group provisioning)
      ▼
AWS IAM Identity Center
      |  permission sets
      ▼
AWS accounts
```

1. Users and groups are created once in Entra ID.
2. SCIM pushes them automatically to IAM Identity Center.
3. Permission sets (for example read-only, network admin) map groups to AWS accounts.
4. Users sign in once (SSO) and get only the access their group allows (least privilege).

The actual Entra → AWS setup is shown step by step in section 3.14.

2.9 Workload migration (sub-phase 6)

Source	Target	Method	Validation
EC2	Azure VM	Azure Migrate / rebuild	App config, networking, dependencies
RDS PostgreSQL	Azure Database for PostgreSQL	Initial copy → replication → validation → cutover → rollback	Row counts, app queries
S3	Blob Storage	Azure Storage Mover	Object count, size, checksum
Secrets Manager	Key Vault	Recreate secrets, repoint apps	Secret retrieval by the app
EKS	AKS	Move Namespace, Deployment, Service, ConfigMap, Secret	Pods healthy, service reachable

Database migration follows a fixed sequence:

1. Initial copy of the 2–3 small representative databases.
2. Continuous replication / synchronization.
3. Validation (counts, checksums, application queries).
4. Cutover in a planned window.
5. Rollback path kept ready until sign-off.

The S3 → Blob and RDS → Azure PostgreSQL runs are documented with screenshots in sections 3.12 and 3.13.

2.10 Hardcoded IP and legacy applications (sub-phase 7)

Example problem: an application config contains `DB_HOST=10.10.10.50`. After migration that IP no longer exists.

What to investigate:

- Hardcoded IPs, hostnames and ports
- Firewall allow-lists and vendor allow-lists
- Certificates tied to names or IPs
- Application configuration files and external dependencies

Strategies, from short-term to long-term:

Strategy	When to use	Trade-off
Preserve the IP	Target subnet can own the same address and the source is shut down	Needs careful CIDR planning, no overlap
NAT / compatibility layer	Old IP must keep answering for a while	Extra hop, more to maintain
Proxy	Traffic can be redirected through a proxy endpoint	Single point to scale and secure
DNS abstraction	App can use a name instead of an IP	Best long-term: next migration needs only a DNS change
Application config change	Code/config can be edited	Cleanest, needs app team and testing

DNS abstraction is preferred long-term because it decouples the application from the network layout.

2.11 Backup, DR and data protection (sub-phase 8)

Order before any migration step:

1. Take a backup (AWS Backup into a backup vault).
2. Verify the backup by restoring it.
3. Set up replication or copy to the target side.
4. Only then start the migration.

Tests: AWS Backup plan, backup vault, recovery point restore, cross-region/cross-cloud copy, data validation after restore.

2.12 Live application cutover (sub-phase 9)

The application is assumed to be live, so downtime must be minimal.

```
LIVE AWS APPLICATION
   → Dependency discovery
   → Backup + replication
   → Target validation
   → Controlled cutover
        ├── SUCCESS → continue on target
        └── FAILURE → roll back to source
```

Key terms managed during cutover:

Term	Meaning
RPO	Maximum data loss allowed, measured in time
RTO	Maximum time to restore service
Replication lag	How far the target is behind the source
Deployment freeze	No releases during the window
Cutover window	Agreed time slot for the switch
Smoke testing	Quick checks right after switching
Rollback	Planned return to the source if checks fail

2.13 Troubleshooting and validation (sub-phase 10)

Layer	What is tested on purpose
Network	DNS, routes, security groups, NSGs, VPN, ports, NAT
Application	App → DB, App → API, App → Redis, App → Storage
Data	Row counts, object counts, checksums, application transactions

2.14 Terraform finalization (sub-phase 11)

Console → Import → Terraform → Plan → Apply → Validate, with the module layout from section 2.5 and no hardcoded values.

2.15 Cleanup (sub-phase 12)

1. Migration completed.
2. Validation completed.
3. Rollback window completed.
4. Destroy lab resources (VPN gateways first — they bill hourly).
5. Verify billing in AWS and Azure after reports refresh.

3. Phase 2 — On-prem (MacBook) → AWS connectivity

Phase 2 gives a local MacBook, standing in for an on-prem administrator workstation, secure private access to AWS through AWS Client VPN — the same principles used for enterprise data-centre connectivity. This phase used the AWS Console, not Terraform.

3.1 What is being built

```
Local MacBook (administrator workstation, LAN 192.168.29.0/24)
      |  encrypted AWS Client VPN connection (OpenVPN, TLS, UDP 443)
      ▼
AWS Client VPN endpoint (us-east-1)
      |  authentication · routing · authorization · logging
      ▼
Existing AWS VPC 10.60.0.0/16
      ├── Application EC2 migration-app-server  10.60.0.117
      └── Private RDS PostgreSQL
```

3.2 Implementation scope (in order)

1. **Pre-checks:** verify AWS permissions, existing networking, and a non-overlapping VPN client CIDR.
2. **Authentication:** create and import the certificates for mutual TLS authentication.
3. **Client VPN endpoint:** create the endpoint in us-east-1.
4. **VPC association:** associate the target subnet (two AZs recommended for resilience).
5. **Routing:** add private VPC routes without disturbing the existing AWS–Azure routes.
6. **Security:** authorization rules and security groups with least privilege.
7. **Client setup:** download the configuration and connect from macOS.

8. **Connectivity tests:** reach only the authorized private AWS resources.
9. **Monitoring and cleanup:** connection logs on; remove temporary resources when done.

3.3 What already existed

Component	Current setup
AWS VPC	10.60.0.0/16
Application subnet	10.60.1.0/24
Database subnets	10.60.2.0/24, 10.60.3.0/24
Application EC2	migration-app-server, 10.60.0.117
Database	Private PostgreSQL RDS
AWS–Azure VPN	Existing Site-to-Site VPN — must keep working

The EC2 instance and RDS had been stopped to save cost, so their state was checked before testing.

3.4 Pre-check: CIDR plan

Network	CIDR	Purpose
MacBook LAN	192.168.29.0/24	Current local network
AWS VPC	10.60.0.0/16	Existing AWS private network
Azure VNet	10.50.0.0/16	Existing Azure private network
Client VPN pool	10.250.0.0/22	Addresses given to VPN clients

The client pool must not overlap with the VPC, Azure or the local LAN, and AWS requires it to be between /12 and /22.

3.5 Laptop vs data centre

- **This phase:** MacBook → AWS using **Client VPN** (one user/device per connection).
- **Later:** a real on-prem data centre → AWS using **Site-to-Site VPN**, with **AWS Direct Connect** where bandwidth or latency needs it.

A laptop connection is a useful first step but does not connect a whole on-prem network.

3.6 Completion criteria

The phase is complete when the MacBook can establish an authenticated VPN connection, reach only the authorized private AWS resources, and generate connection logs — without breaking the AWS–Azure VPN.

3.7 Certificate authority and certificates (Easy-RSA)

Client VPN uses mutual TLS: the server proves its identity to the client and each client proves its identity to the server. A private CA created on the MacBook signs both.

```
Certificate Authority  Project13-AWS-Client-VPN-CA
   ├── Server certificate  server-vpn   → imported into AWS ACM
   └── Client certificate  macbook-v3   → embedded in the .ovpn profile
       (one client certificate per user/device)
```

Step 1 — Install tools

```
brew install git openssl easy-rsa
git --version
openssl version
brew list easy-rsa
```

Step 2 — Prepare a private working folder

```
mkdir -p ~/aws-client-vpn && chmod 700 ~/aws-client-vpn
cd ~/aws-client-vpn && mkdir -p pki-work && cd pki-work
git clone https://github.com/OpenVPN/easy-rsa.git
cd easy-rsa/easyrsa3
```

Step 3 — Initialise the PKI and build the CA

```
./easyrsa init-pki
./easyrsa build-ca
# CA passphrase: set (min 4 characters)
# Common Name: Project13-AWS-Client-VPN-CA
```

Creates `pki/ca.crt` (public, shared) and `pki/private/ca.key` (secret, stays protected with a passphrase).

Step 4 — Server certificate

The first server certificate had no Subject Alternative Name, which ACM/Client VPN needs. It was rebuilt with a SAN that matches the name the client verifies:

```
./easyrsa --san=DNS:server-vpn build-server-full server-vpn nopass
# confirm: yes · enter CA passphrase
# Certificate valid 825 days (until Jan 2029)
```

Check it:

```
openssl x509 -in pki/issued/server-vpn.crt -noout -subject -issuer -dates \
  -ext subjectAltName -ext extendedKeyUsage
# subject=CN=server-vpn
# issuer=CN=Project13-AWS-Client-VPN-CA
# X509v3 Subject Alternative Name: DNS:server-vpn
# X509v3 Extended Key Usage: TLS Web Server Authentication

openssl verify -CAfile pki/ca.crt pki/issued/server-vpn.crt
# pki/issued/server-vpn.crt: OK
```

Step 5 — Client certificate

```
./easyrsa build-client-full macbook-v3 nopass
openssl verify -CAfile pki/ca.crt pki/issued/macbook-v3.crt
# pki/issued/macbook-v3.crt: OK
```

The client certificate shows *TLS Web Client Authentication*. `nopass` means the client key has no passphrase; the CA key keeps its passphrase.

Step 6 — Check the key matches the certificate

```
openssl x509 -in pki/issued/server.crt -pubkey -noout | openssl pkey -pubin -
outform DER | shasum -a 256
openssl pkey -in pki/private/server.key -pubout -outform DER | shasum -a 256
# both hashes identical → key and certificate belong together
```

Step 7 — Lock down permissions and stage files for ACM

```
chmod 600 pki/private/*.key pki/ca.crt pki/issued/*.crt

mkdir -p ~/aws-client-vpn/aws-import-v2 && chmod 700 ~/aws-client-vpn/aws-
import-v2
cp pki/issued/server-vpn.crt ~/aws-client-vpn/aws-import-v2/server.crt
cp pki/private/server-vpn.key ~/aws-client-vpn/aws-import-v2/server.key
cp pki/ca.crt               ~/aws-client-vpn/aws-import-v2/ca.crt
chmod 600 ~/aws-client-vpn/aws-import-v2/*
```

PKI folder layout

```
pki/
├── ca.crt
├── issued/   server-vpn.crt, macbook-v3.crt
├── private/  ca.key, server-vpn.key, macbook-v3.key
├── reqs/     *.req
├── revoked/
└── index.txt, serial
```

Note on the Easy-RSA warning: "INCOMPLETE Inline file created — Missing TLS Key" refers to an optional `tls-crypt` key. AWS Client VPN does not need it, so the warning was safe to ignore.

3.8 AWS Client VPN endpoint — step by step

Step 1 — Import the server certificate into ACM

1. AWS Certificate Manager (us-east-1) → Import certificate.
2. Fill the three fields from `~/aws-client-vpn/aws-import-v2/`:

ACM field	Local file
Certificate body	server.crt
Certificate private key	server.key
Certificate chain	ca.crt

3. Import. Status shows *Issued*, type *Imported*, domain `server-vpn`.
4. The client certificate is not uploaded: the endpoint trusts the CA, so any client cert signed by that CA is accepted.

Step 2 — Security group for the VPN endpoint

VPC → Security groups → Create `cvpn-endpoint-sg` in `vpc-migration`. No inbound rules are needed; outbound allows traffic to the VPC.

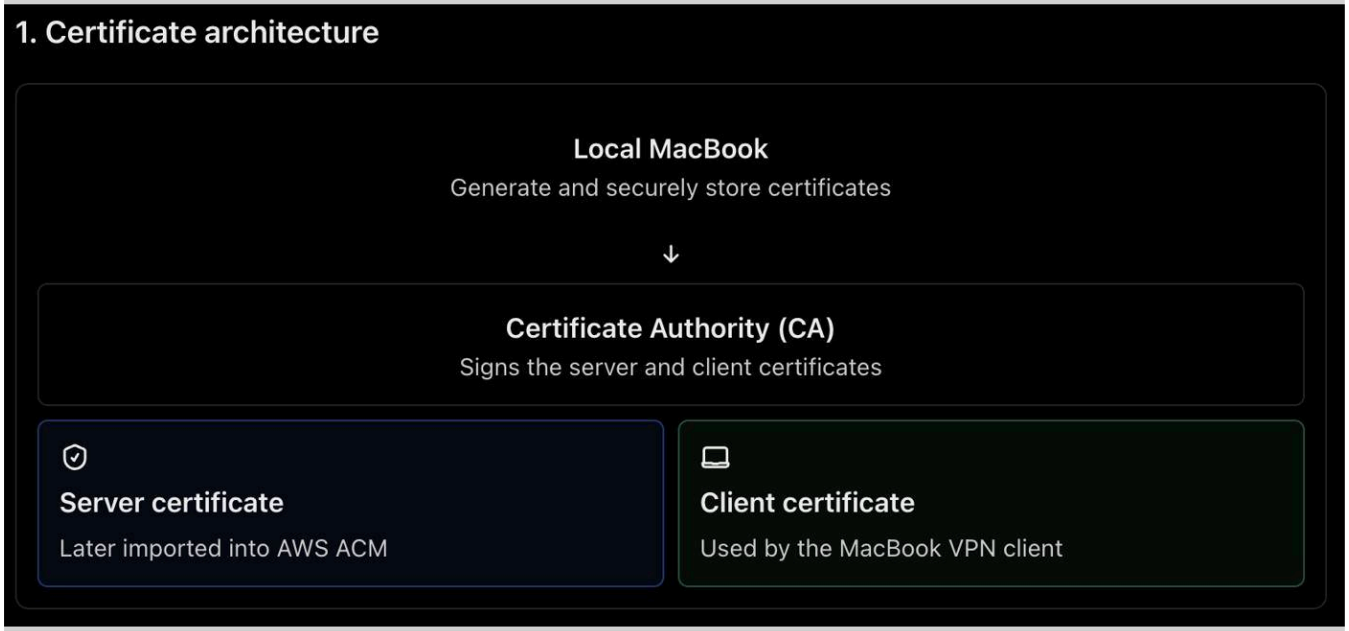
Step 3 — Create the Client VPN endpoint

1. VPC → Client VPN endpoints → Create.
2. Name `cvpn-macbook`, description "Secure MacBook access to migration VPC".
3. Client IPv4 CIDR 10.250.0.0/22.
4. Server certificate ARN: the imported `server-vpn` certificate.
5. Authentication: *Use mutual authentication*, client certificate ARN = same CA-trusted certificate.

6. Connection logging enabled (CloudWatch log group), split tunnel enabled so only VPC traffic uses the VPN.
7. VPC `vpc-migration`, security group `cvpn-endpoint-sg`, transport UDP, port 443.
8. Create. Endpoint ID `cvpn-endpoint-0ed0e999bcfa3f5c9`, state *Pending-associate*.

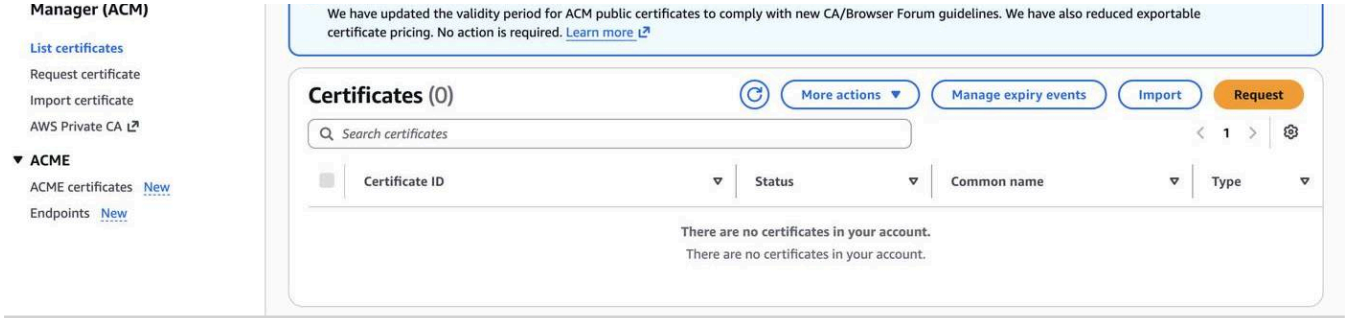


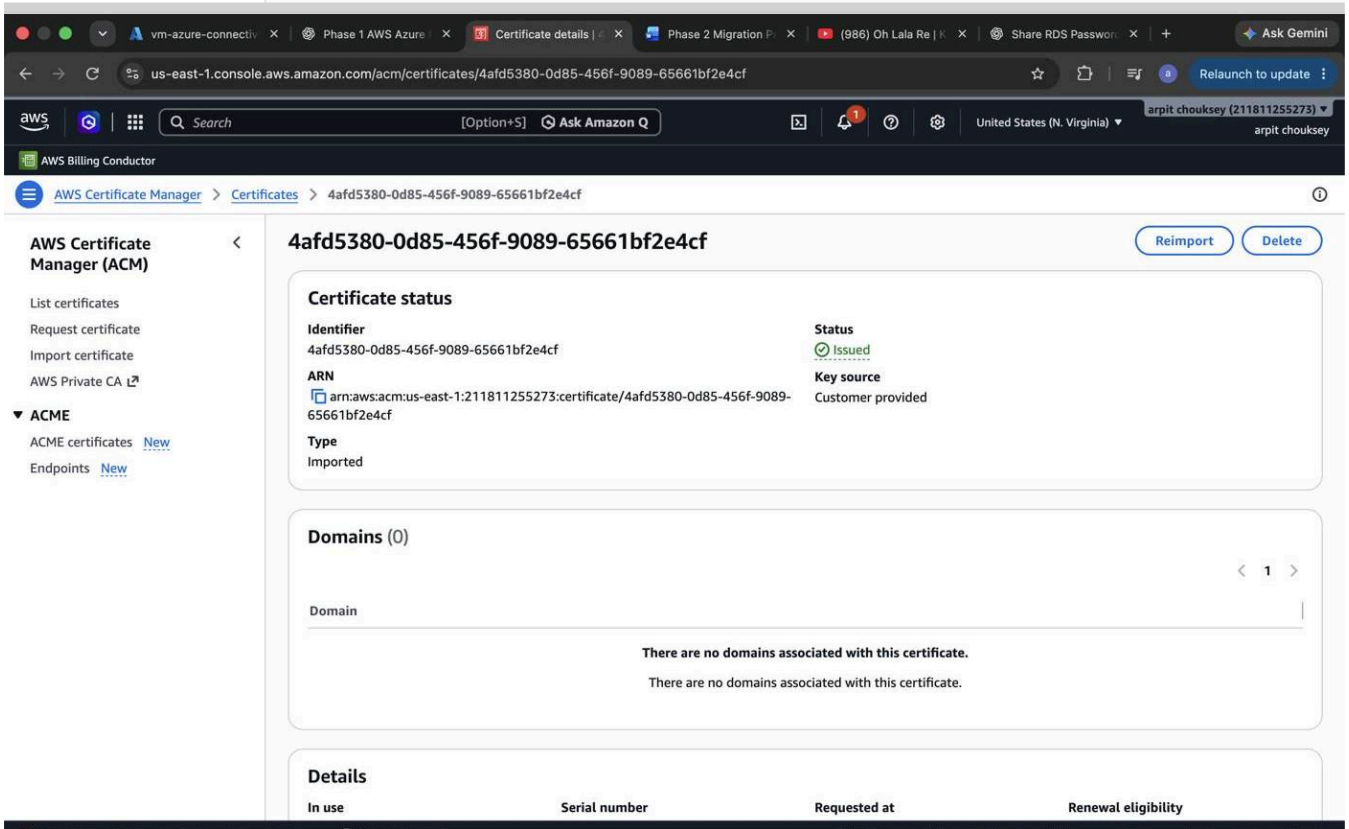
1. Certificate architecture

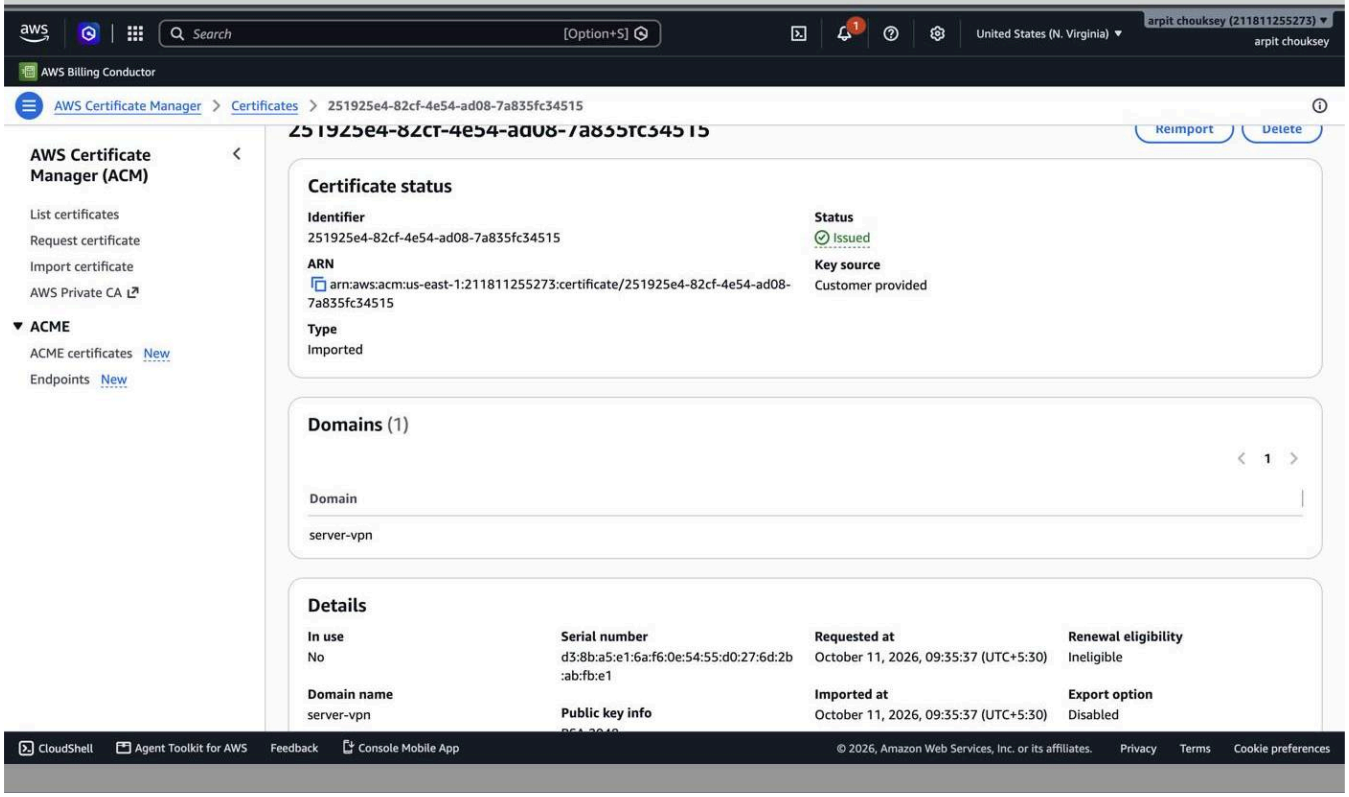


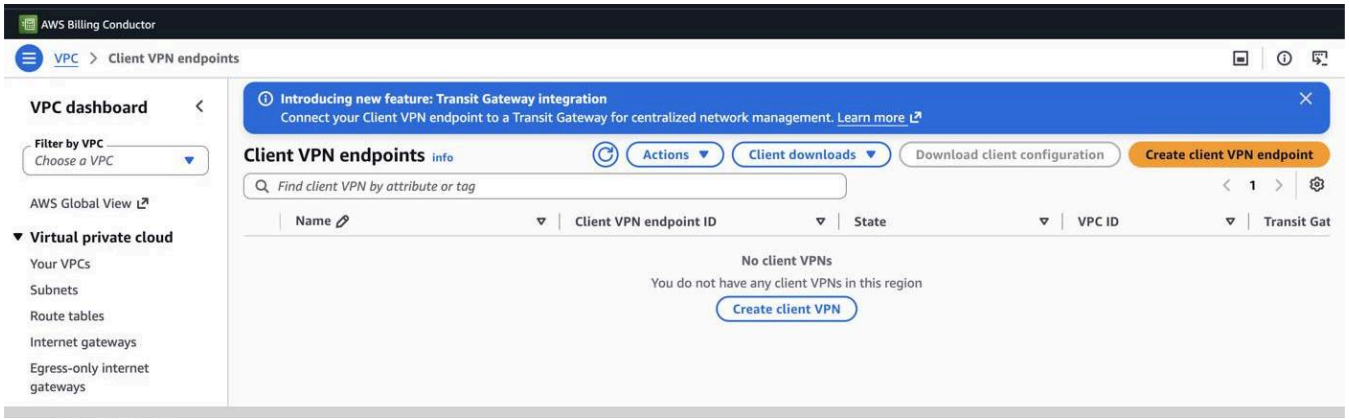


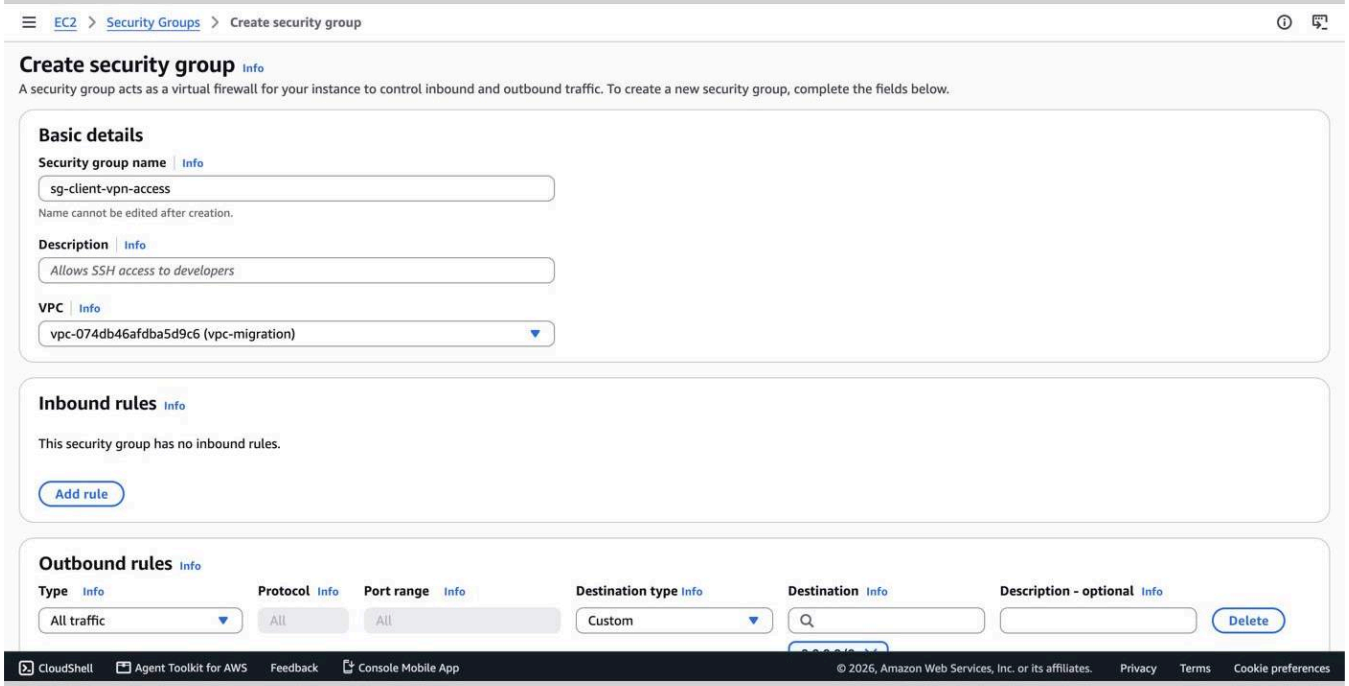

AWS VPC — 10.60.0.0/16
Private resources, subject to routing and security rules
MacBook
AWS VPN Client · OpenVPN profile
Certificate-based authentication
Client certificate + private key verified against trusted CA
AWS Client VPN endpoint
cvpn-endpoint-0ed0e999bcfa3f5c9
Client CIDR: 10.250.0.0/22 · UDP 443
VPN target subnet + routing
subnet-060bc710412df5d15
Route and authorization: 10.60.0.0/16
AWS VPC
vpc-074db46afdba5d9c6 · 10.60.0.0/16
Migration EC2
i-0c51672deaacf0e68 · 10.60.0.117
Amazon Linux 2023 · SSH TCP/22
TCP connectivity verified
aws
Search
[Option+S]
Ask Amazon Q
United States (N. Virginia)
arpit chouksey (211811255273)
arpit chouksey
AWS Billing Conductor
AWS Certificate Manager
Certificates
AWS Certificate
Update on ACM public certificate validity

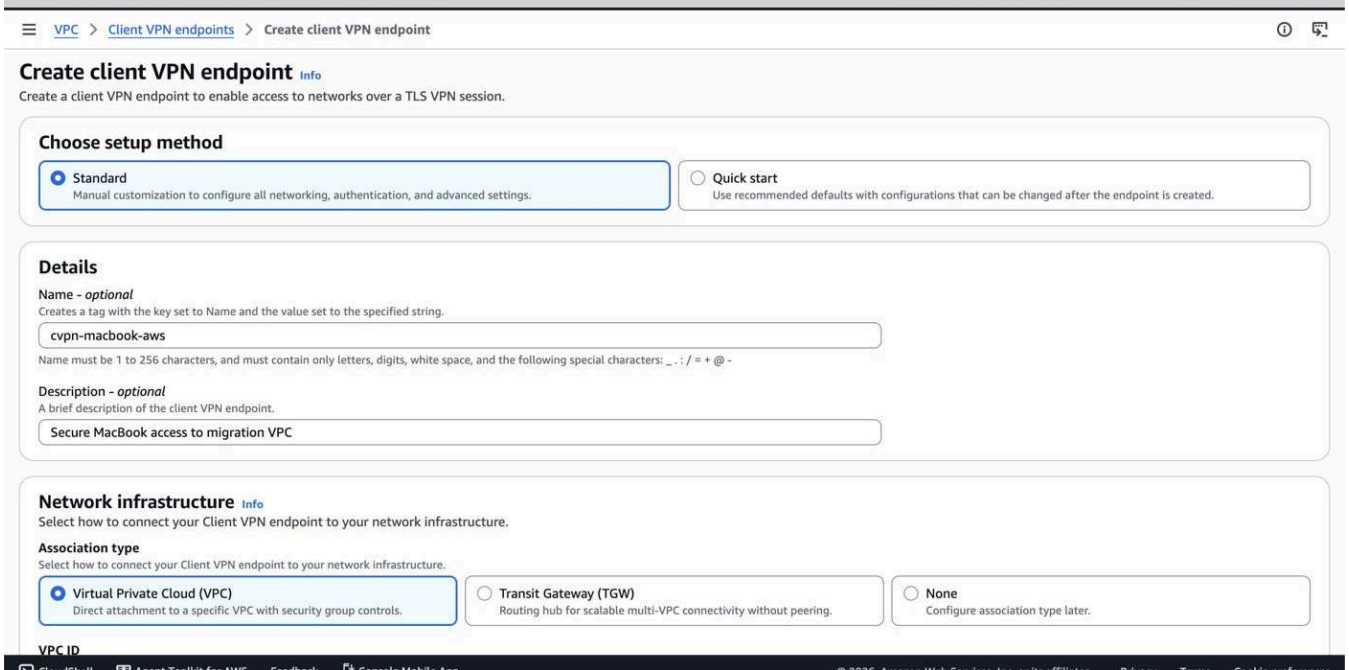

Manager (ACM)
List certificates
Request certificate
Import certificate
AWS Private CA
ACME
ACME certificates New
Endpoints New
We have updated the validity period for ACM public certificates to comply with new CA/Browser Forum guidelines. We have also reduced exportable certificate pricing. No action is required. Learn more
Certificates (0)
More actions
Manage expiry events
Import
Request
Search certificates
Certificate ID
Status
Common name
Type
There are no certificates in your account.
There are no certificates in your account.

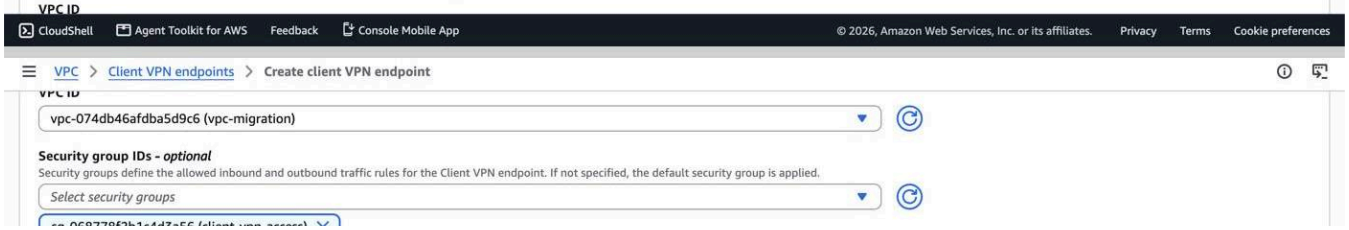

us-east-1.console.aws.amazon.com/acm/certificates/4afd5380-0d85-456f-9089-65661bf2e4cf
Search
[Option+S]
Ask Amazon Q
United States (N. Virginia)
arpit chouksey (211811255273)
arpit chouksey
AWS Billing Conductor
AWS Certificate Manager > Certificates > 4afd5380-0d85-456f-9089-65661bf2e4cf
AWS Certificate Manager (ACM)
List certificates
Request certificate
Import certificate
AWS Private CA
ACME
ACME certificates New
Endpoints New
4afd5380-0d85-456f-9089-65661bf2e4cf
Reimport
Delete
Certificate status
Identifier
4afd5380-0d85-456f-9089-65661bf2e4cf
Status
Issued
ARN
arn:aws:acm:us-east-1:211811255273:certificate/4afd5380-0d85-456f-9089-65661bf2e4cf
Key source
Customer provided
Type
Imported
Domains (0)
Domain
There are no domains associated with this certificate.
There are no domains associated with this certificate.
Details
In use
Serial number
Requested at
Renewal eligibility
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Search
[Option+S]
United States (N. Virginia)
arpit chouksey (211811255273)
arpit chouksey
AWS Billing Conductor
AWS Certificate Manager > Certificates > 251925e4-82cf-4e54-ad08-7a835fc34515
AWS Certificate Manager (ACM)
List certificates
Request certificate
Import certificate
AWS Private CA
ACME
ACME certificates New
Endpoints New
Reimport
Delete
Certificate status
Identifier
251925e4-82cf-4e54-ad08-7a835fc34515
Status
Issued
ARN
arn:aws:acm:us-east-1:211811255273:certificate/251925e4-82cf-4e54-ad08-7a835fc34515
Key source
Customer provided
Type
Imported
Domains (1)
Domain
server-vpn
Details
In use
No
Serial number
d3:8b:a5:e1:6a:f6:0e:54:55:d0:27:6d:2b:ab:fb:e1
Requested at
October 11, 2026, 09:35:37 (UTC+5:30)
Renewal eligibility
Ineligible
Domain name
server-vpn
Public key info
Imported at
October 11, 2026, 09:35:37 (UTC+5:30)
Export option
Disabled
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


AWS Billing Conductor
VPC > Client VPN endpoints
VPC dashboard
Filter by VPC
Choose a VPC
AWS Global View
Virtual private cloud
Your VPCs
Subnets
Route tables
Internet gateways
Egress-only internet gateways
Introducing new feature: Transit Gateway integration
Connect your Client VPN endpoint to a Transit Gateway for centralized network management. Learn more
Client VPN endpoints info
Actions
Client downloads
Download client configuration
Create client VPN endpoint
Find client VPN by attribute or tag
Name
Client VPN endpoint ID
State
VPC ID
Transit Gat
No client VPNs
You do not have any client VPNs in this region
Create client VPN


EC2 > Security Groups > Create security group
Create security group info
A security group acts as a virtual firewall for your instance to control inbound and outbound traffic. To create a new security group, complete the fields below.
Basic details
Security group name info
sg-client-vpn-access
Name cannot be edited after creation.
Description info
Allows SSH access to developers
VPC info
vpc-074db46afdba5d9c6 (vpc-migration)
Inbound rules info
This security group has no inbound rules.
Add rule
Outbound rules info
Type info
Protocol info
Port range info
Destination type info
Destination info
Description - optional info
All traffic
All
All
Custom
Delete
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


VPC > Client VPN endpoints > Create client VPN endpoint
Create client VPN endpoint info
Create a client VPN endpoint to enable access to networks over a TLS VPN session.
Choose setup method
Standard
Manual customization to configure all networking, authentication, and advanced settings.
Quick start
Use recommended defaults with configurations that can be changed after the endpoint is created.
Details
Name - optional
Creates a tag with the key set to Name and the value set to the specified string.
cvpn-macbook-aws
Name must be 1 to 256 characters, and must contain only letters, digits, white space, and the following special characters: _ . : / = + @ -
Description - optional
A brief description of the client VPN endpoint.
Secure MacBook access to migration VPC
Network infrastructure info
Select how to connect your Client VPN endpoint to your network infrastructure.
Association type
Select how to connect your Client VPN endpoint to your network infrastructure.
Virtual Private Cloud (VPC)
Direct attachment to a specific VPC with security group controls.
Transit Gateway (TGW)
Routing hub for scalable multi-VPC connectivity without peering.
None
Configure association type later.
VPC ID
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


VPC > Client VPN endpoints > Create client VPN endpoint
VPC ID
vpc-074db46afdba5d9c6 (vpc-migration)
Security group IDs - optional
Security groups define the allowed inbound and outbound traffic rules for the Client VPN endpoint. If not specified, the default security group is applied.
Select security groups



Step 4 — Associate the target network

1. Endpoint → Target network associations → Associate.
2. VPC `vpc-migration`, subnet AppSubnet (`subnet-060bc710412df5d15`).
3. Wait until state is *Associated*; the endpoint becomes *Available*. A second subnet in another AZ adds resilience in production.

Step 5 — Authorization rule

1. Endpoint → Authorization rules → Add.
2. Destination network 10.60.0.0/16, grant access to all users (lab). Production would scope by group and by smaller CIDRs.

Step 6 — Route

The association adds route 10.60.0.0/16 via the AppSubnet automatically. No route to 10.50.0.0/16 was added, so the existing AWS–Azure VPN is untouched.

Step 7 — Allow VPN clients on the EC2 security group

Edit inbound rules of `migration-app`: SSH (22) from the VPN client CIDR 10.250.0.0/22 (or from `cvpn-endpoint-sg`). Existing Azure rules stay as they are.

Step 8 — Download and build the client profile

1. Endpoint → Download client configuration → saved as `~/Downloads/project13-correct.ovpn`.
2. Extract a clean PEM copy of the client certificate (no human-readable text block — that text broke the first profile):

```
cd ~/aws-client-vpn/pki-work/easy-rsa/easyrsa3
openssl x509 -in pki/issued/macbook-v3.crt -out ~/Downloads/macbook-v3-clean.crt
```

3. Append the client cert and key to a copy of the profile:

```
cd ~/Downloads
cp project13-correct.ovpn project13-final-v3.ovpn
cat >> project13-final-v3.ovpn <<EOF
<cert>
$(cat ~/Downloads/macbook-v3-clean.crt)
</cert>
<key>
$(cat ~/aws-client-vpn/pki-work/easy-rsa/easyrsa3/pki/private/macbook-v3.key)
</key>
EOF
chmod 600 project13-final-v3.ovpn
```

4. Sanity-check the profile structure (prints only tags, never the key):

```
grep -E '^remote |^verify-x509-
name|^<ca>|^</ca>|^<cert>|^</cert>|^<key>|^</key>' project13-final-v3.ovpn
# remote cvpn-endpoint-0ed0e999bcfa3f5c9.prod.clientvpn.us-east-
1.amazonaws.com 443
# <ca> … </ca>
# verify-x509-name server-vpn name
# <cert> … </cert>
# <key> … </key>
```

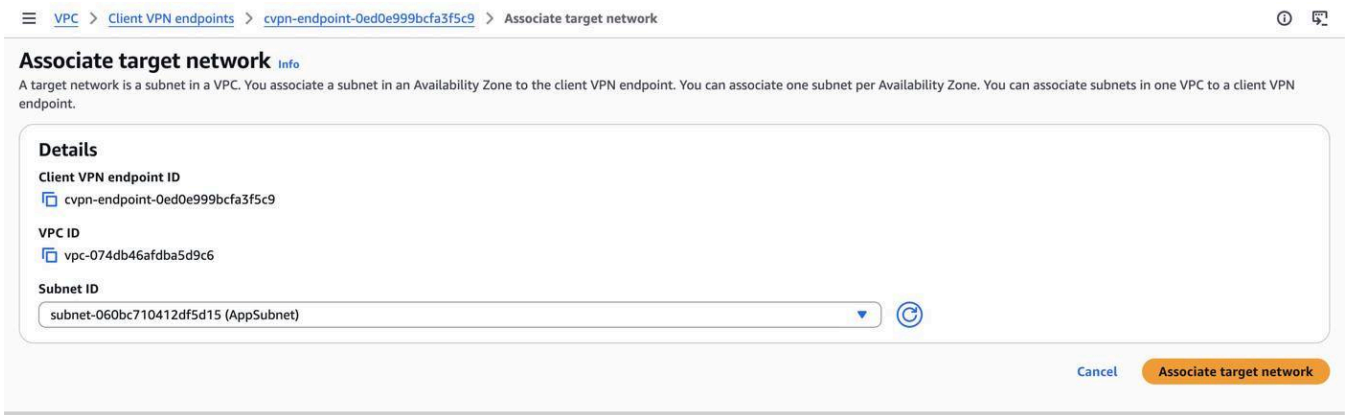
Step 9 — Connect from macOS

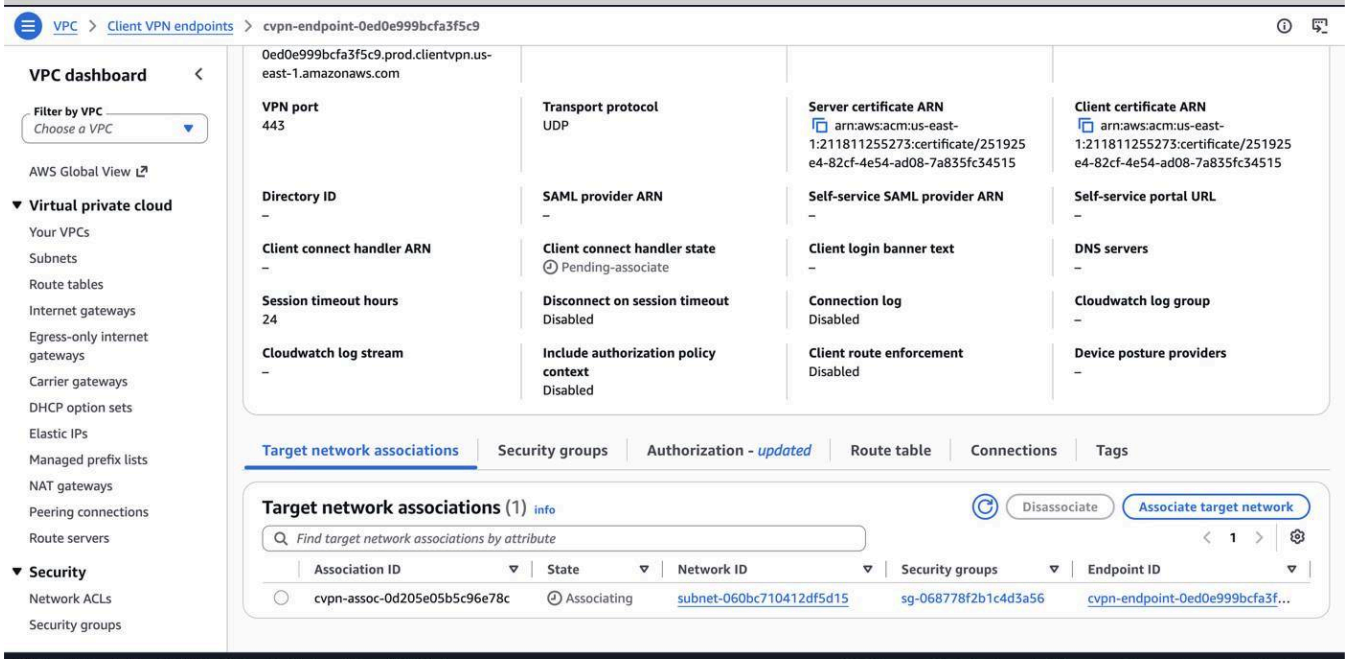
1. Install AWS VPN Client for macOS.
2. File → Manage profiles → Add profile → name `localaws`, choose `project13-final-v3.ovpn`.
3. Connect. Status: *Connected*.

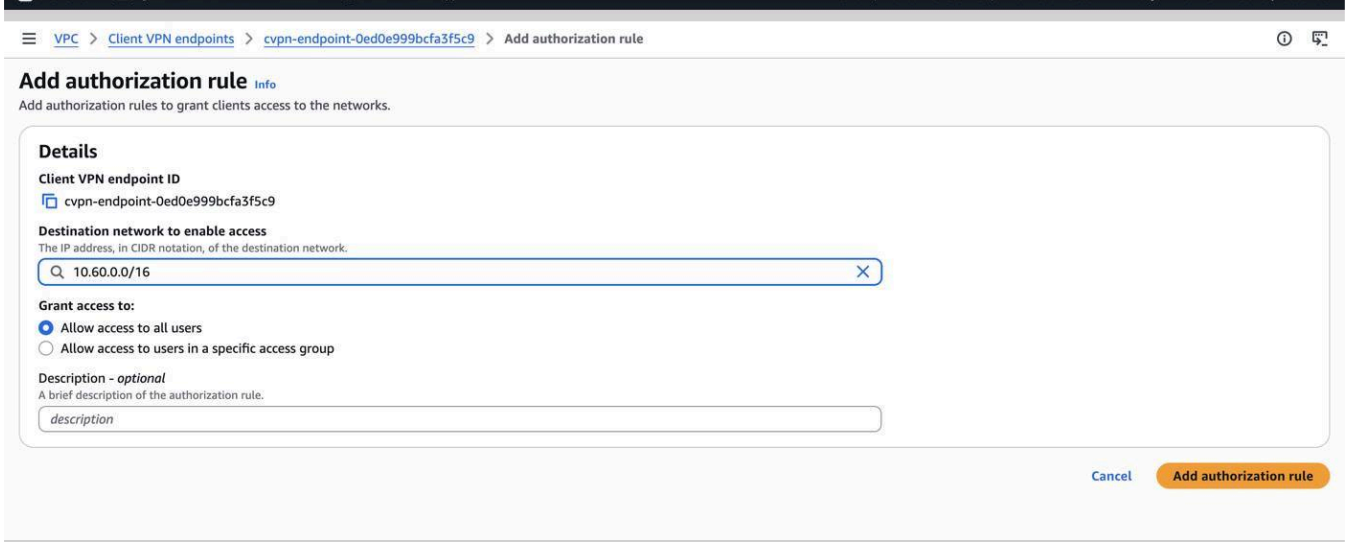
Step 10 — Test private access

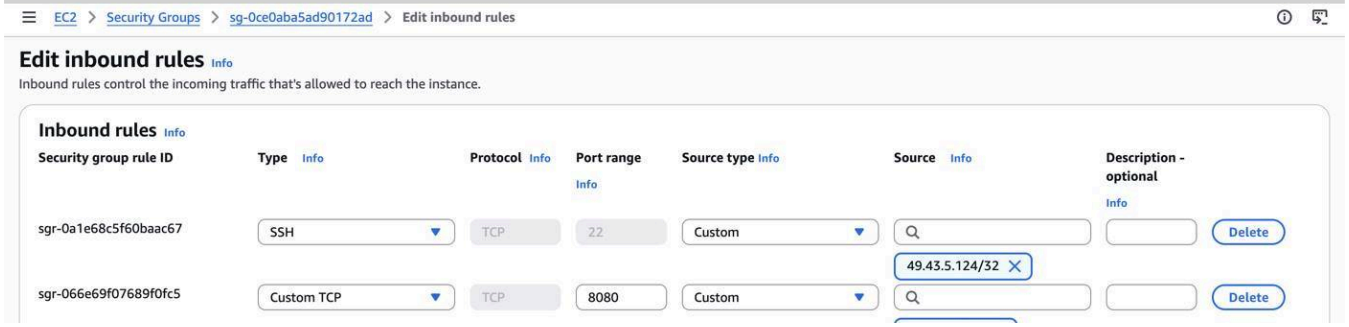
```
nc -vz -G 5 10.60.0.117 22
# Connection to 10.60.0.117 port 22 [tcp/ssh] succeeded!
```

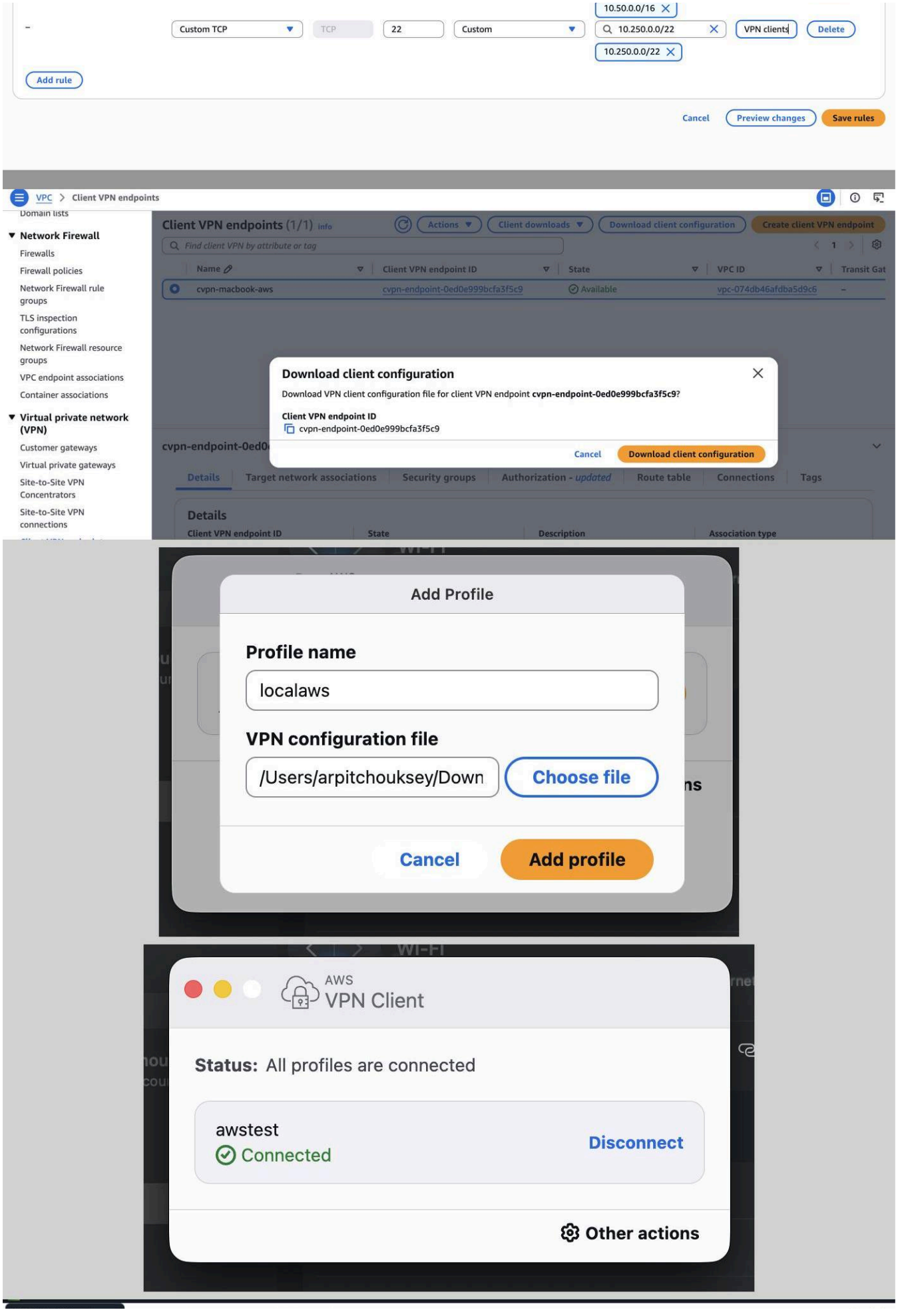
The MacBook reached the private EC2 IP with no public IP involved.

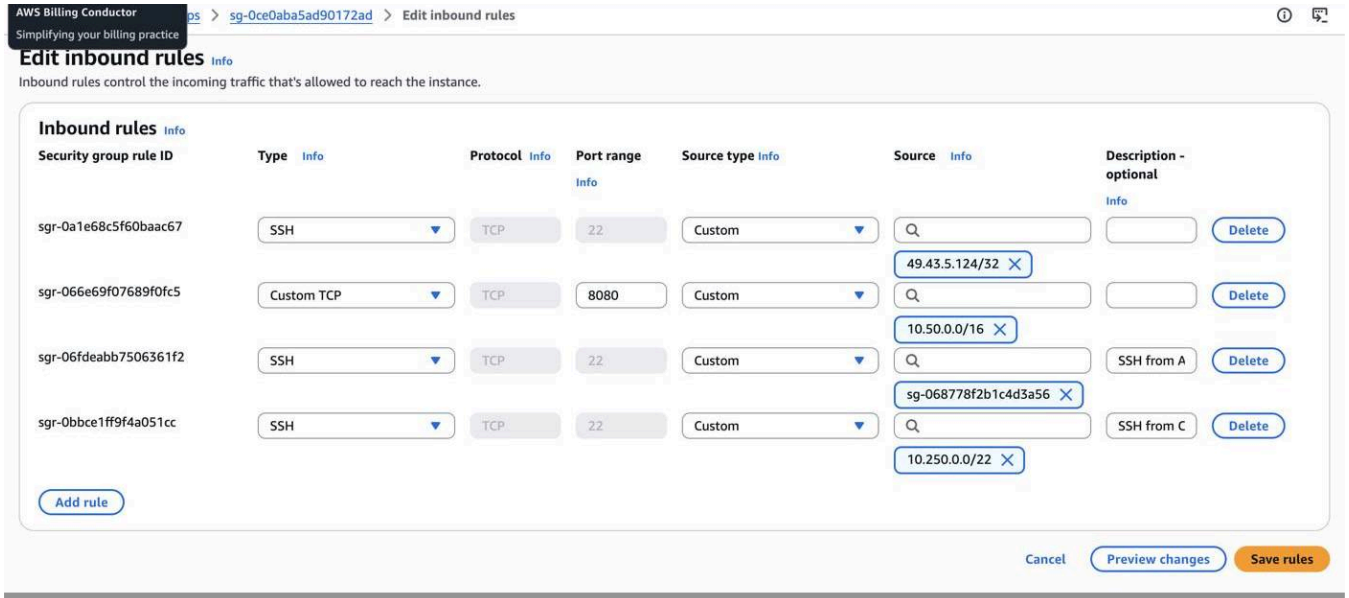

VPC > Client VPN endpoints > cvpn-endpoint-0ed0e999bcfa3f5c9 > Associate target network
Associate target network Info
A target network is a subnet in a VPC. You associate a subnet in an Availability Zone to the client VPN endpoint. You can associate one subnet per Availability Zone. You can associate subnets in one VPC to a client VPN endpoint.
Details
Client VPN endpoint ID
cvpn-endpoint-0ed0e999bcfa3f5c9
VPC ID
vpc-074db46afdba5d9c6
Subnet ID
subnet-060bc710412df5d15 (AppSubnet)
Cancel
Associate target network

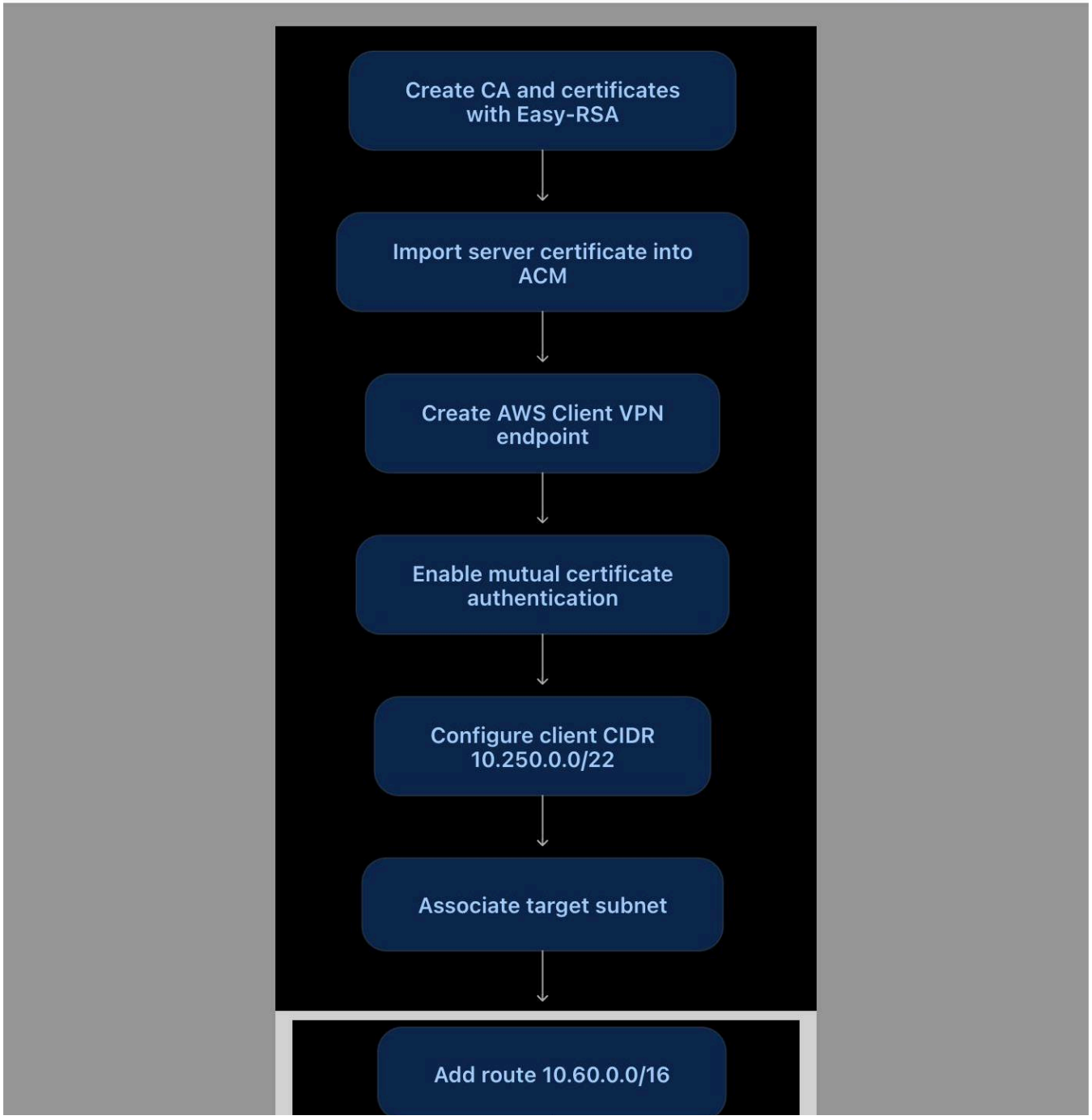

VPC > Client VPN endpoints > cvpn-endpoint-0ed0e999bcfa3f5c9
VPC dashboard
Filter by VPC
Choose a VPC
AWS Global View
Virtual private cloud
Your VPCs
Subnets
Route tables
Internet gateways
Egress-only internet gateways
Carrier gateways
DHCP option sets
Elastic IPs
Managed prefix lists
NAT gateways
Peering connections
Route servers
Security
Network ACLs
Security groups
0ed0e999bcfa3f5c9.prod.clientvpn.us-east-1.amazonaws.com
VPN port
443
Transport protocol
UDP
Server certificate ARN
arn:aws:acm:us-east-1:211811255273:certificate/251925e4-82cf-4e54-ad08-7a835fc34515
Client certificate ARN
arn:aws:acm:us-east-1:211811255273:certificate/251925e4-82cf-4e54-ad08-7a835fc34515
Directory ID
–
SAML provider ARN
–
Self-service SAML provider ARN
–
Self-service portal URL
–
Client connect handler ARN
–
Client connect handler state
Pending-associate
Client login banner text
–
DNS servers
–
Session timeout hours
24
Disconnect on session timeout
Disabled
Connection log
Disabled
Cloudwatch log group
–
Cloudwatch log stream
–
Include authorization policy context
Disabled
Client route enforcement
Disabled
Device posture providers
–
Target network associations
Security groups
Authorization - updated
Route table
Connections
Tags
Target network associations (1) info
Disassociate
Associate target network
Find target network associations by attribute
Association ID
State
Network ID
Security groups
Endpoint ID
cvpn-assoc-0d205e05b5c96e78c
Associating
subnet-060bc710412df5d15
sg-068778f2b1c4d3a56
cvpn-endpoint-0ed0e999bcfa3f...
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

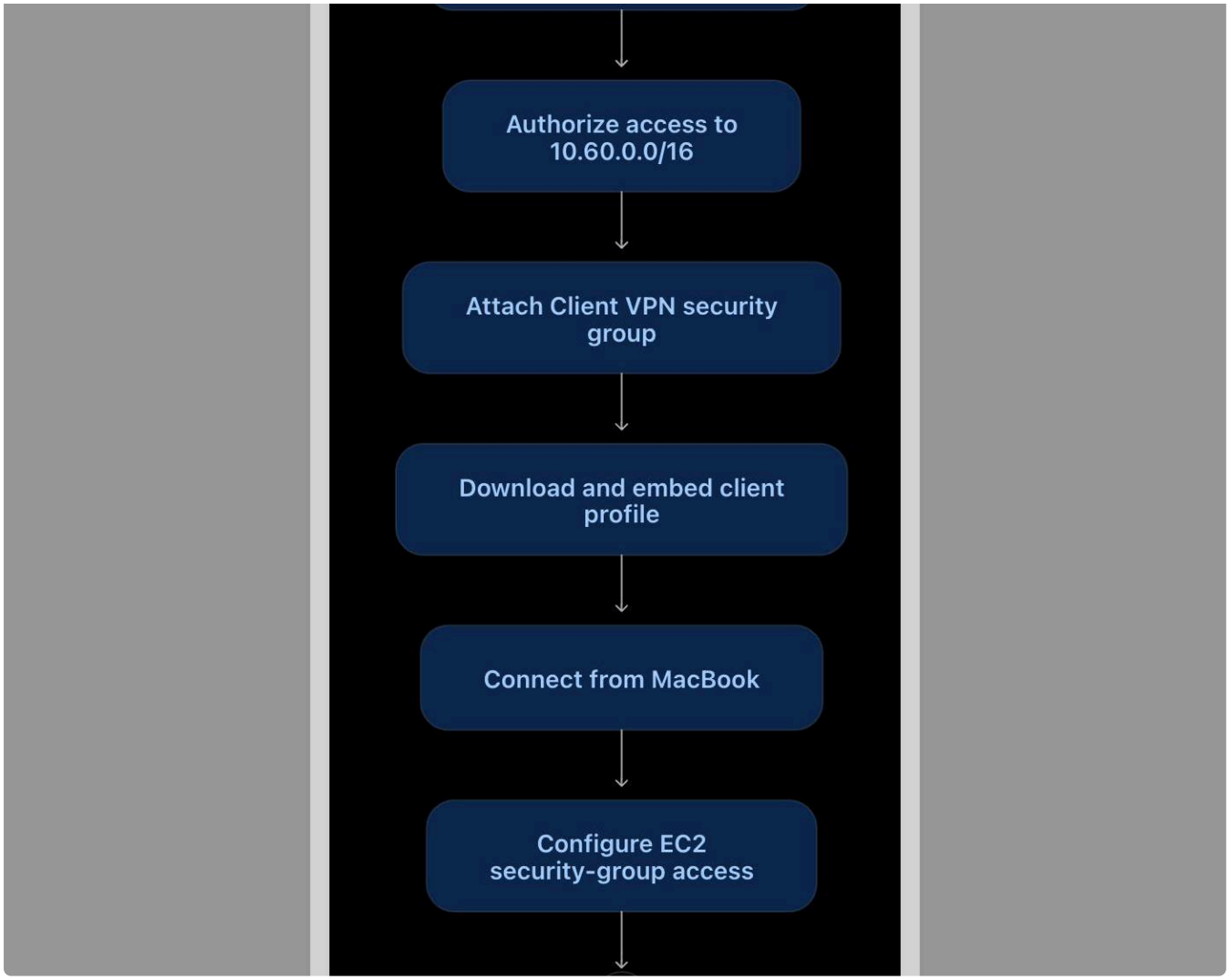

VPC > Client VPN endpoints > cvpn-endpoint-0ed0e999bcfa3f5c9 > Add authorization rule
Add authorization rule Info
Add authorization rules to grant clients access to the networks.
Details
Client VPN endpoint ID
cvpn-endpoint-0ed0e999bcfa3f5c9
Destination network to enable access
The IP address, in CIDR notation, of the destination network.
10.60.0.0/16
Grant access to:
Allow access to all users
Allow access to users in a specific access group
Description - optional
A brief description of the authorization rule.
description
Cancel
Add authorization rule


EC2 > Security Groups > sg-0ce0aba5ad90172ad > Edit inbound rules
Edit inbound rules Info
Inbound rules control the incoming traffic that's allowed to reach the instance.
Inbound rules Info
Security group rule ID
Type Info
Protocol Info
Port range Info
Source type Info
Source Info
Description - optional Info
sgr-0a1e68c5f60baac67
SSH
TCP
22
Custom
49.43.5.124/32
Delete
sgr-066e69f07689f0fc5
Custom TCP
TCP
8080
Custom
Delete


10.50.0.0/16
Custom TCP
TCP
22
Custom
10.250.0.0/22
VPN clients
Delete
10.250.0.0/22
Add rule
Cancel
Preview changes
Save rules
VPC > Client VPN endpoints
Client VPN endpoints (1/1) info
Actions
Client downloads
Download client configuration
Create client VPN endpoint
Find client VPN by attribute or tag
Name
Client VPN endpoint ID
State
VPC ID
Transit Gat
cvpn-macbook-aws
cvpn-endpoint-0ed0e999bcfa3f5c9
Available
vpc-074db46afdba5d9c6
Network Firewall
Firewalls
Firewall policies
Network Firewall rule groups
TLS inspection configurations
Network Firewall resource groups
VPC endpoint associations
Container associations
Virtual private network (VPN)
Customer gateways
Virtual private gateways
Site-to-Site VPN Concentrators
Site-to-Site VPN connections
Download client configuration
Download VPN client configuration file for client VPN endpoint cvpn-endpoint-0ed0e999bcfa3f5c9?
Client VPN endpoint ID
cvpn-endpoint-0ed0e999bcfa3f5c9
Cancel
Download client configuration
Details
Target network associations
Security groups
Authorization - updated
Route table
Connections
Tags
Details
Client VPN endpoint ID
State
Description
Association type
Add Profile
Profile name
localaws
VPN configuration file
/Users/arpitchouksey/Down
Choose file
Cancel
Add profile
AWS VPN Client
Status: All profiles are connected
awstest
Connected
Disconnect
Other actions


AWS Billing Conductor
Simplifying your billing practice
ps > sg-0ce0aba5ad90172ad > Edit inbound rules
Edit inbound rules Info
Inbound rules control the incoming traffic that's allowed to reach the instance.
Inbound rules Info
Security group rule ID
Type Info
Protocol Info
Port range Info
Source type Info
Source Info
Description - optional Info
sgr-0a1e68c5f60baac67
SSH
TCP
22
Custom
49.43.5.124/32
Delete
sgr-066e69f07689f0fc5
Custom TCP
TCP
8080
Custom
10.50.0.0/16
Delete
sgr-06fdeabb7506361f2
SSH
TCP
22
Custom
sg-068778f2b1c4d3a56
SSH from A
Delete
sgr-0bbce1ff9f4a051cc
SSH
TCP
22
Custom
10.250.0.0/22
SSH from C
Delete
Add rule
Cancel
Preview changes
Save rules


Create CA and certificates with Easy-RSA
Import server certificate into ACM
Create AWS Client VPN endpoint
Enable mutual certificate authentication
Configure client CIDR 10.250.0.0/22
Associate target subnet
Add route 10.60.0.0/16



End-to-end flow

```
Create CA and certificates (Easy-RSA)
 → Import server certificate into ACM
 → Create Client VPN endpoint (mutual TLS, 10.250.0.0/22)
 → Associate target subnet
 → Add route 10.60.0.0/16 and authorize 10.60.0.0/16
 → Attach security group
 → Download and embed client profile
 → Connect from MacBook
 → Allow VPN CIDR on EC2 security group
 → TCP connectivity verified
```

Problems hit and fixes

Problem	Cause	Fix
Endpoint could not use the first server cert	No Subject Alternative Name	Rebuilt as `server-vpn` with `--san=DNS:server-vpn`

Problem	Cause	Fix
Profile failed to import/connect	Certificate text block pasted with the PEM	Exported clean PEM with `openssl x509 -out`
Client key exposed during troubleshooting	Key pasted into a chat	Treat as compromised: revoke and issue a new client cert

3.9 Granting access to an additional user

One Client VPN endpoint serves many users. Each user gets their own client certificate signed by the same CA; no new endpoint is needed.

Workflow

1. The VPN administrator generates a unique client certificate and key for the user.
2. The existing Project13 CA signs it.
3. The administrator takes the downloaded endpoint configuration.
4. The user's certificate and key are embedded into a separate `.ovpn` profile.
5. The profile is sent to the user through a secure channel.
6. The user imports it into AWS VPN Client and connects.
7. Access is limited by authorization rules, routes, security groups and NACLs.

Step 1 — Generate the user's certificate (on the admin MacBook)

```
cd ~/aws-client-vpn/pki-work/easy-rsa/easyrsa3
./easyrsa build-client-full friend-laptop nopass
# creates pki/issued/friend-laptop.crt, pki/private/friend-laptop.key,
pki/reqs/friend-laptop.req
```

Step 2 — Validate it

```
openssl verify -CAfile pki/ca.crt pki/issued/friend-laptop.crt
# pki/issued/friend-laptop.crt: OK
```

Step 3 — Build the user's profile

```
openssl x509 -in pki/issued/friend-laptop.crt -out ~/Downloads/friend-laptop-
clean.crt
cd ~/Downloads
cp project13-correct.ovpn friend-laptop.ovpn
cat >> friend-laptop.ovpn <<EOF
<cert>
$(cat ~/Downloads/friend-laptop-clean.crt)
</cert>
<key>
$(cat ~/aws-client-vpn/pki-work/easy-rsa/easyrsa3/pki/private/friend-
laptop.key)
</key>
EOF
chmod 600 friend-laptop.ovpn
```

Step 4 — User side

1. Install AWS VPN Client.
2. Receive `friend-laptop.ovpn` through an approved secure transfer.
3. AWS VPN Client → Add profile → import the file → Save → Connect.
4. Confirm the connection is established. The user generates nothing locally.

Step 5 — AWS-side checks

- The endpoint trusts the CA that signed the new certificate.
- The target network association is active.
- Routes and authorization rules exist (10.60.0.0/16 in this lab).
- Security groups and NACLs allow the intended traffic on the required ports.

Authorization to 10.60.0.0/16 does not open every resource: resource-level security groups still decide. In production, authorize the minimum CIDRs and ports per group.

Removing access: revoke the user's certificate and publish a certificate revocation list (CRL) to the endpoint.

```
./easyrsa revoke friend-laptop
./easyrsa gen-crl
# then import pki/crl.pem into the Client VPN endpoint (Actions → Import
client certificate CRL)
```

3.10 Security and credential rules

Rule	Why
Never share `pki/private/ca.key`	Whoever has it can mint valid VPN identities
Never share the server private key	It proves the endpoint's identity
One certificate and key per user	Lets you revoke one person without affecting others
Treat every `.ovpn` as a secret	It contains the user's private key
Never commit profiles or keys to Git	Repos get cloned and leaked
Back up the CA key offline, encrypted	Losing it means rebuilding all trust
Revoke on exposure	Exposed client keys are issued fresh and the old cert revoked

Validation checklist

- [x] ~~Unique client certificate and key generated~~
- [x] ~~Certificate verifies against the trusted CA~~
- [x] ~~User-specific `.ovpn` created~~
- [x] ~~Profile imported and VPN connected~~
- [x] ~~Access to the intended resource and port tested (EC2 :22 succeeded)~~
- [x] ~~No access outside the authorization scope~~

3.11 Final Client VPN configuration reference

Component	Value
AWS region	us-east-1
VPC	vpc-074db46afdba5d9c6 (10.60.0.0/16)
Client VPN endpoint	cvpn-endpoint-0ed0e999bcfa3f5c9
VPN client CIDR	10.250.0.0/22
Target subnet	subnet-060bc710412df5d15 (AppSubnet)
VPN security group	sg-068778f2b1c4d3a56
EC2 security group	sg-0ce0aba5ad90172ad (migration-app)
EC2	i-0c51672deaacf0e68, 10.60.0.117

Component	Value
Route	10.60.0.0/16
Authorization	10.60.0.0/16, access for all users (lab)
Server certificate	server-vpn, imported in ACM
CA	Project13-AWS-Client-VPN-CA

3.12 AWS S3 → Azure Blob with Azure Storage Mover

With connectivity in place, data started moving from AWS to Azure using managed services instead of manual copying.

```
AWS S3  migration-source-v1
   → Azure Storage Mover
       (AWS connector → source endpoint → target endpoint → migration job)
   → Azure Blob Storage  (objects copied and validated)
```

Step 1 — Target storage account and container

1. Storage accounts → Create in `rg-migration-lab`.
2. Name `project13migrationblob`, performance Standard, redundancy LRS (cheapest for a lab).
3. Containers → + Container → private access, for the migrated objects.

Step 2 — Create the Storage Mover

1. Storage Mover → Create in `rg-migration-lab`, name `project13-storage-mover`.
2. Region East US (close to the us-east-1 source to cut transfer latency).
3. Monitoring: copy logs to a Log Analytics workspace.

Step 3 — Create a project

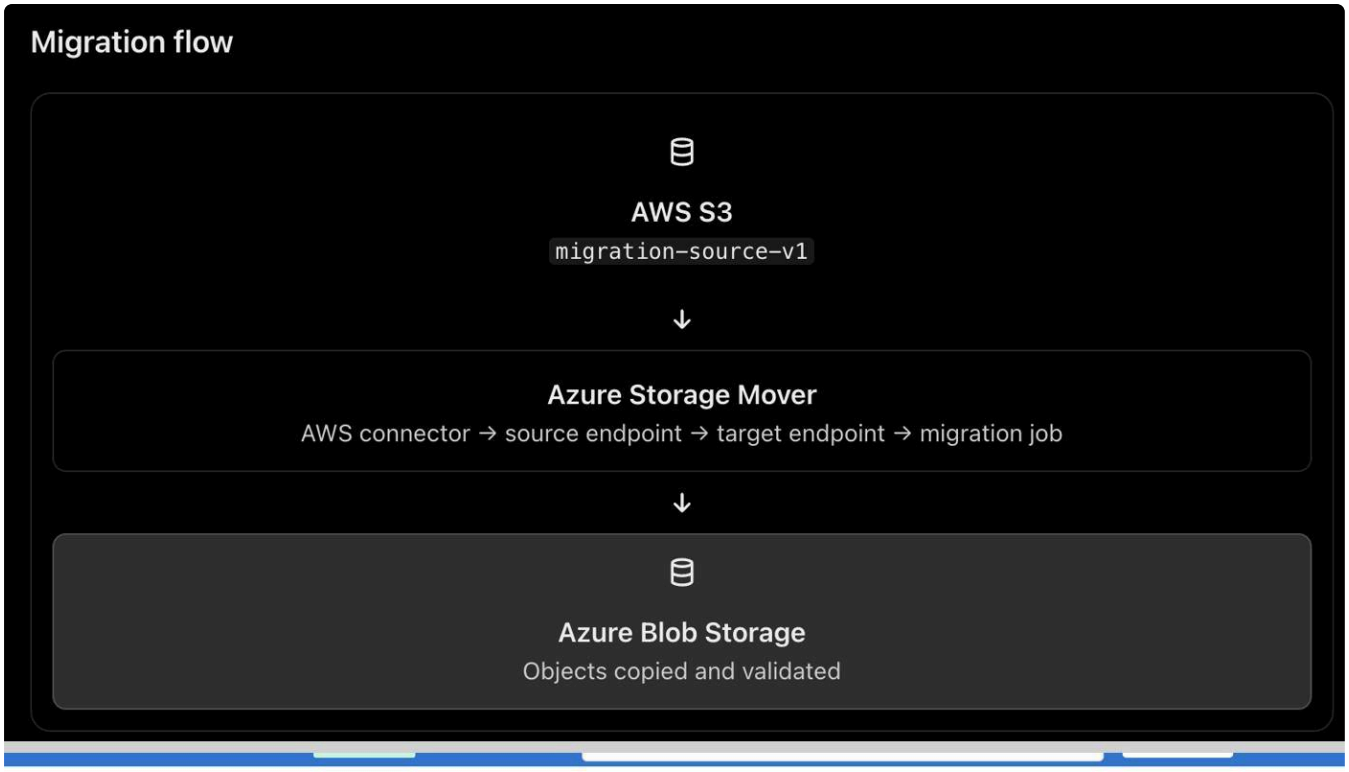
1. Storage Mover → Projects → Create.
2. Migration type: multicloud migration; source type: AWS Object Storage (S3).

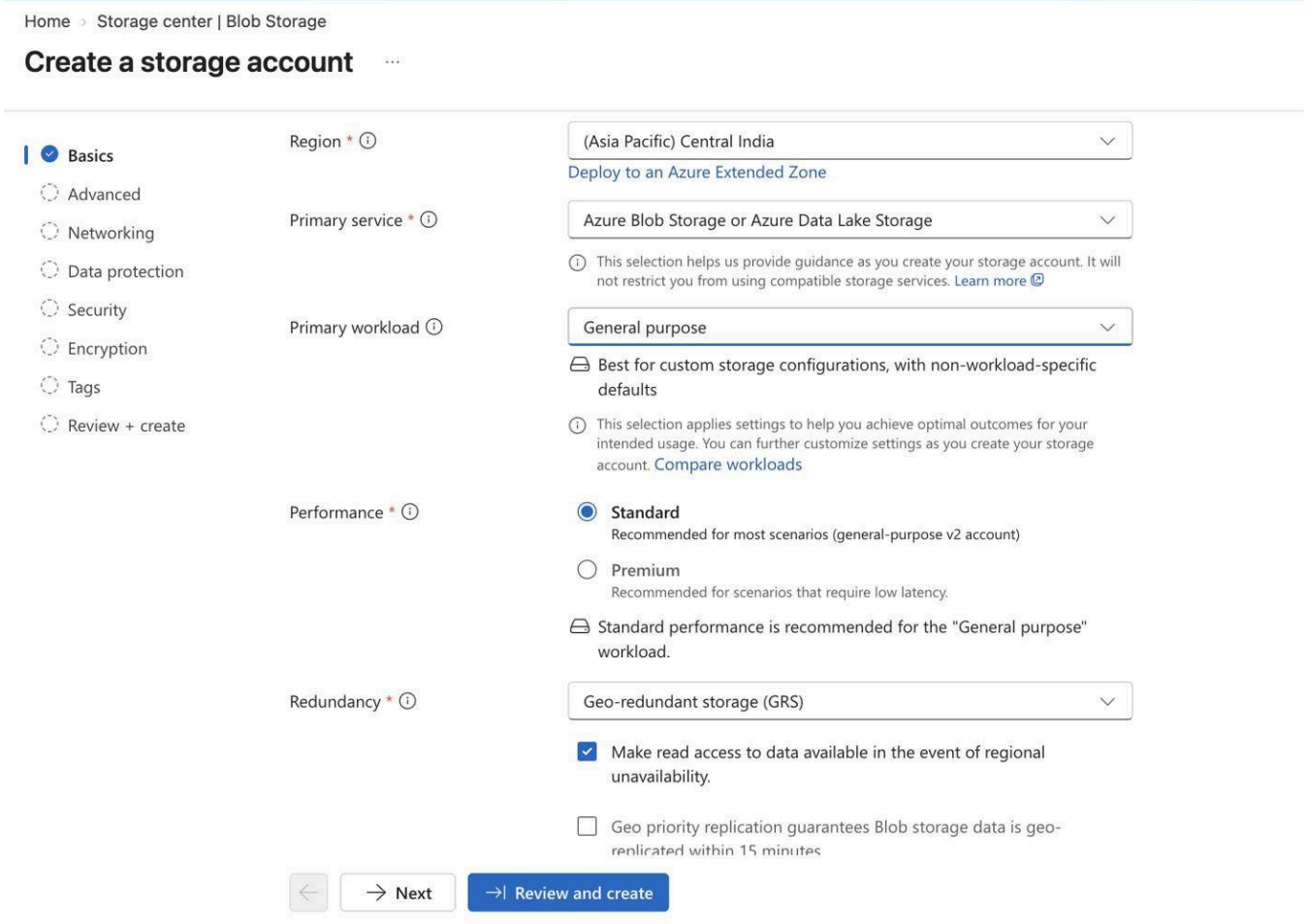
Step 4 — Private S3 access on the AWS side

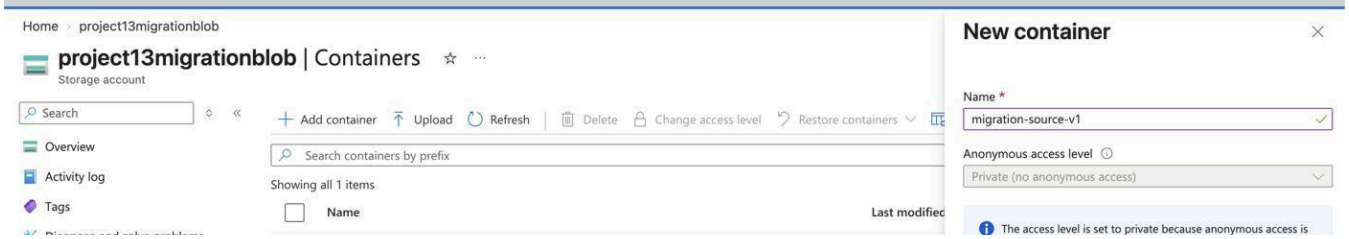
1. VPC → Security groups → create a group for the S3 interface endpoint, HTTPS (443) from the VPC only.
2. VPC → Endpoints → Create → service `com.amazonaws.us-east-1.s3`, type Interface, in `vpc-migration`, with that security group.
3. Endpoint status becomes Available.

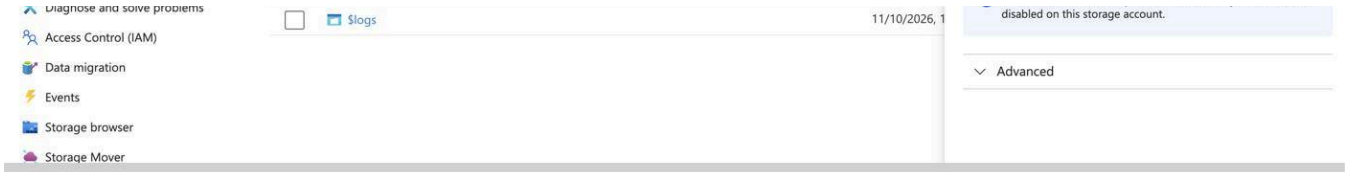
Step 5 — Add the AWS connector

1. Storage Mover → Storage endpoints → Create → Add AWS connector.
2. Connector name `migration-aws-connector`, AWS account type: single account.
3. Solutions: Inventory and Storage – Data Management (needed for S3 discovery and copy).


Migration flow
AWS S3
migration-source-v1
Azure Storage Mover
AWS connector → source endpoint → target endpoint → migration job
Azure Blob Storage
Objects copied and validated


Home › Storage center | Blob Storage
Create a storage account
Basics
Advanced
Networking
Data protection
Security
Encryption
Tags
Review + create
Region *
(Asia Pacific) Central India
Deploy to an Azure Extended Zone
Primary service *
Azure Blob Storage or Azure Data Lake Storage
This selection helps us provide guidance as you create your storage account. It will not restrict you from using compatible storage services. Learn more
Primary workload
General purpose
Best for custom storage configurations, with non-workload-specific defaults
This selection applies settings to help you achieve optimal outcomes for your intended usage. You can further customize settings as you create your storage account. Compare workloads
Performance *
Standard
Recommended for most scenarios (general-purpose v2 account)
Premium
Recommended for scenarios that require low latency.
Standard performance is recommended for the "General purpose" workload.
Redundancy *
Geo-redundant storage (GRS)
Make read access to data available in the event of regional unavailability.
Geo priority replication guarantees Blob storage data is geo-replicated within 15 minutes
Next
Review and create


Home › project13migrationblob
project13migrationblob | Containers
Storage account
Search
Overview
Activity log
Tags
Add container
Upload
Refresh
Delete
Change access level
Restore containers
Search containers by prefix
Showing all 1 items
Name
Last modified
New container
Name *
migration-source-v1
Anonymous access level
Private (no anonymous access)
The access level is set to private because anonymous access is



Home › Storage center | Storage Mover

Create a storage mover ...

Basics Monitoring Tags Review + create

Azure Storage Mover simplifies large scale migrations to Azure Storage. Learn more

Deployment details

Select a subscription for your new storage mover. You'll be able to manage migrations into other subscriptions, as long as they are in the same Azure tenant as the subscription you select for your storage mover resource.

Subscription * Azure subscription 1

Resource group * rg-migration-lab

Create new

Instance details

Typically, a single storage mover resource is needed to manage your storage migration needs. Deploy multiple storage movers only if your migration plan requires different logical grouping.

Name * project13-storage-mover

Region * (US) East US

A storage mover can be deployed to any region without affecting the migration speed. It does not need to be co-located with any other Azure resources to perform migrations. Learn more

< Previous Next > Review + create

Home › Storage center | Storage Mover

Create a storage mover ...

Running final validation...

Basics Monitoring Tags Review + create

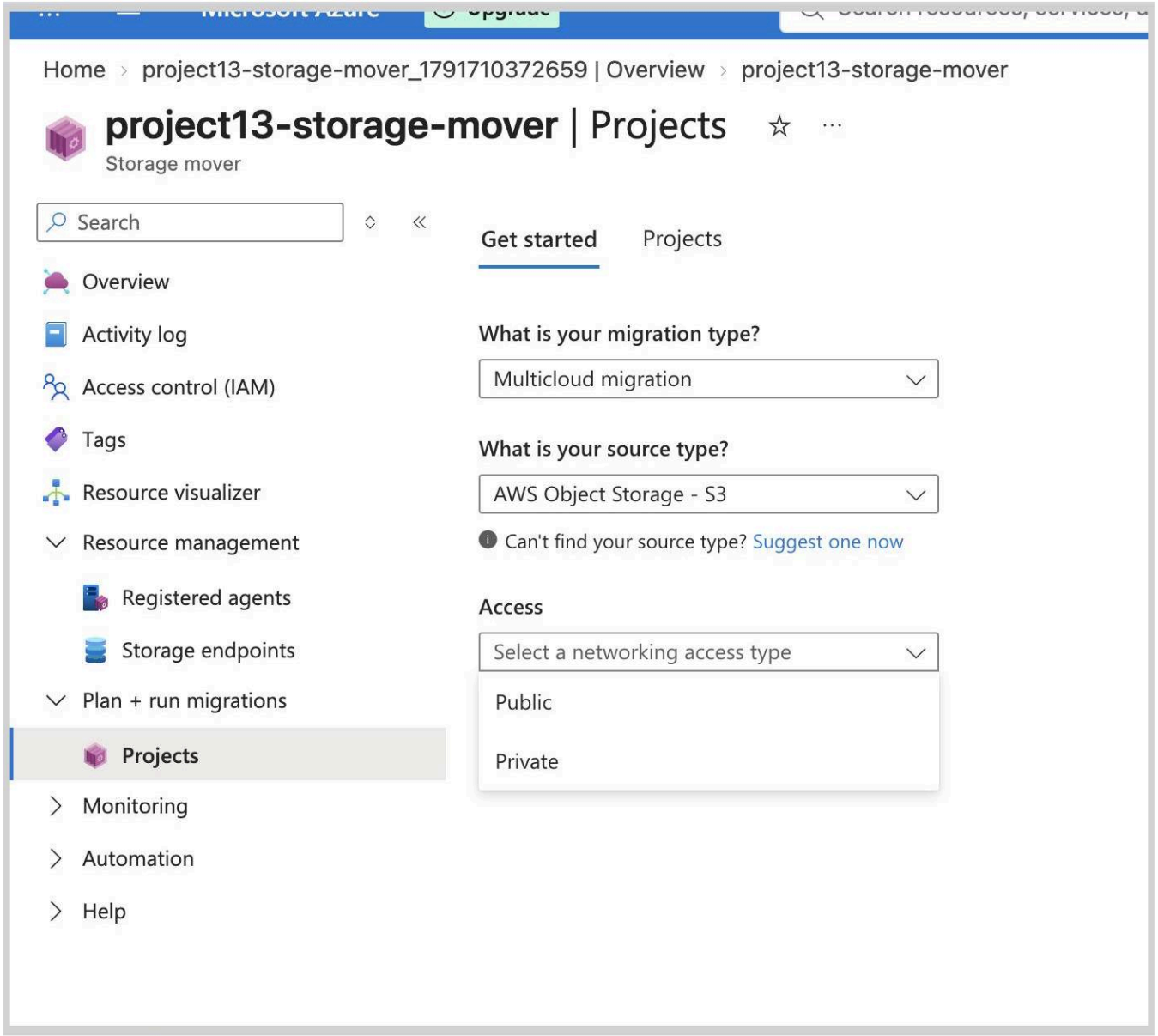
Basics

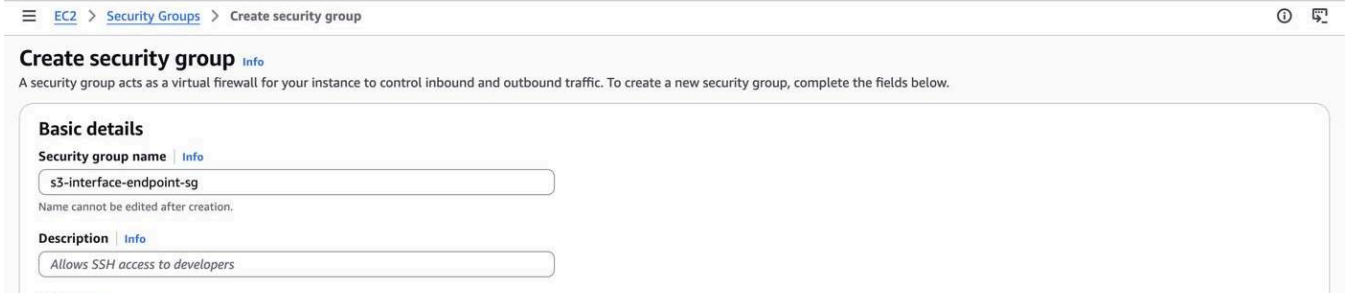
Subscription	Azure subscription 1
Resource group	rg-migration-lab
Region	East US

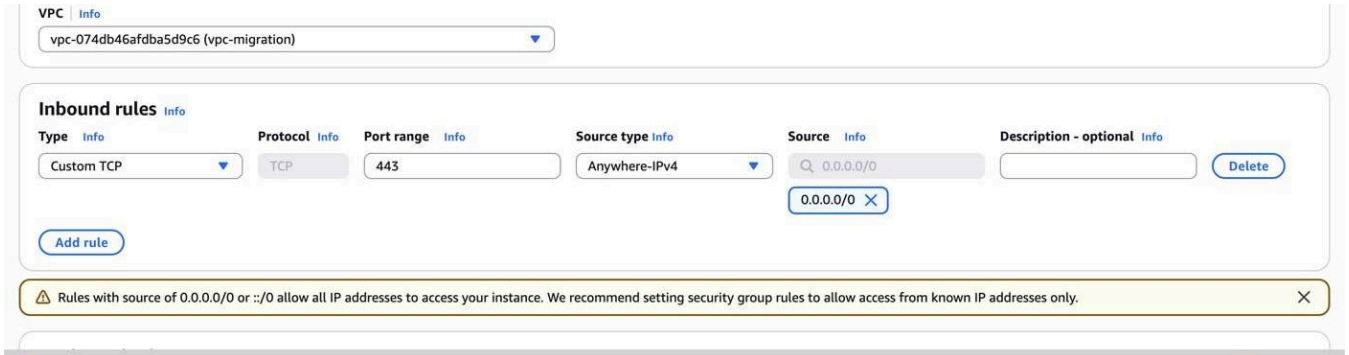
Region	East US
Name	project13-storage-mover

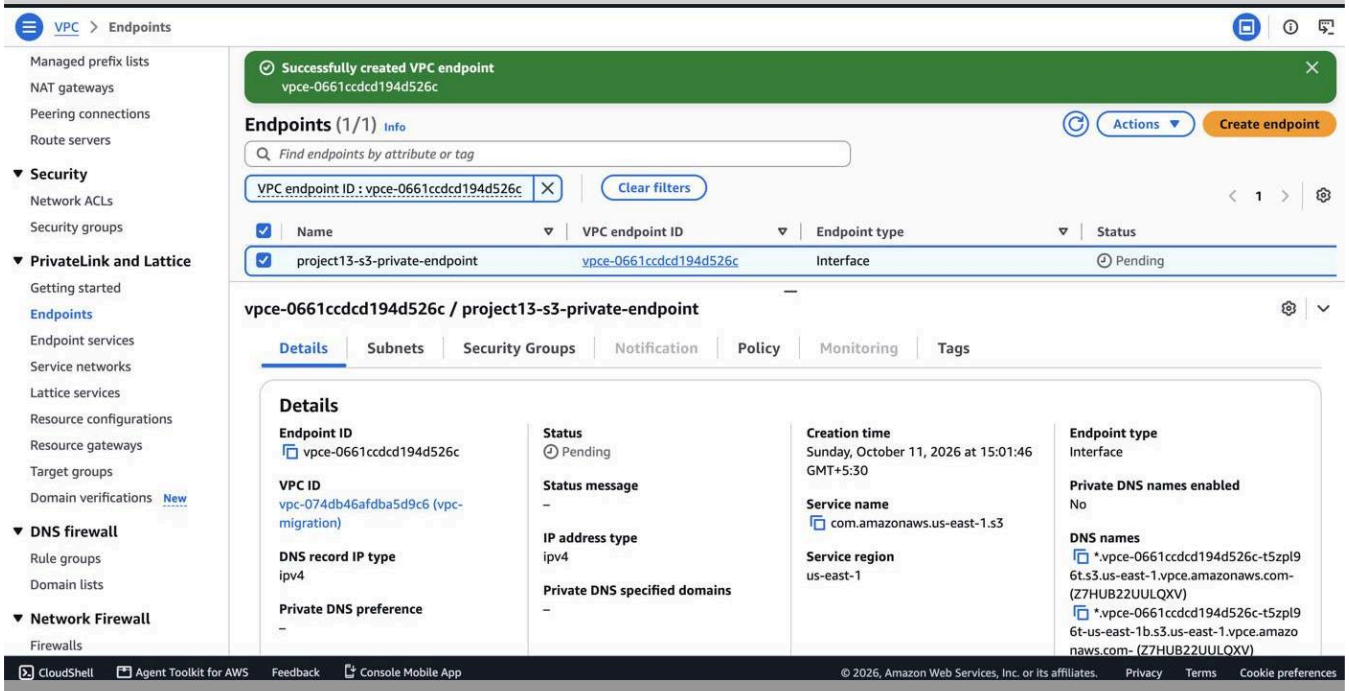
Monitoring

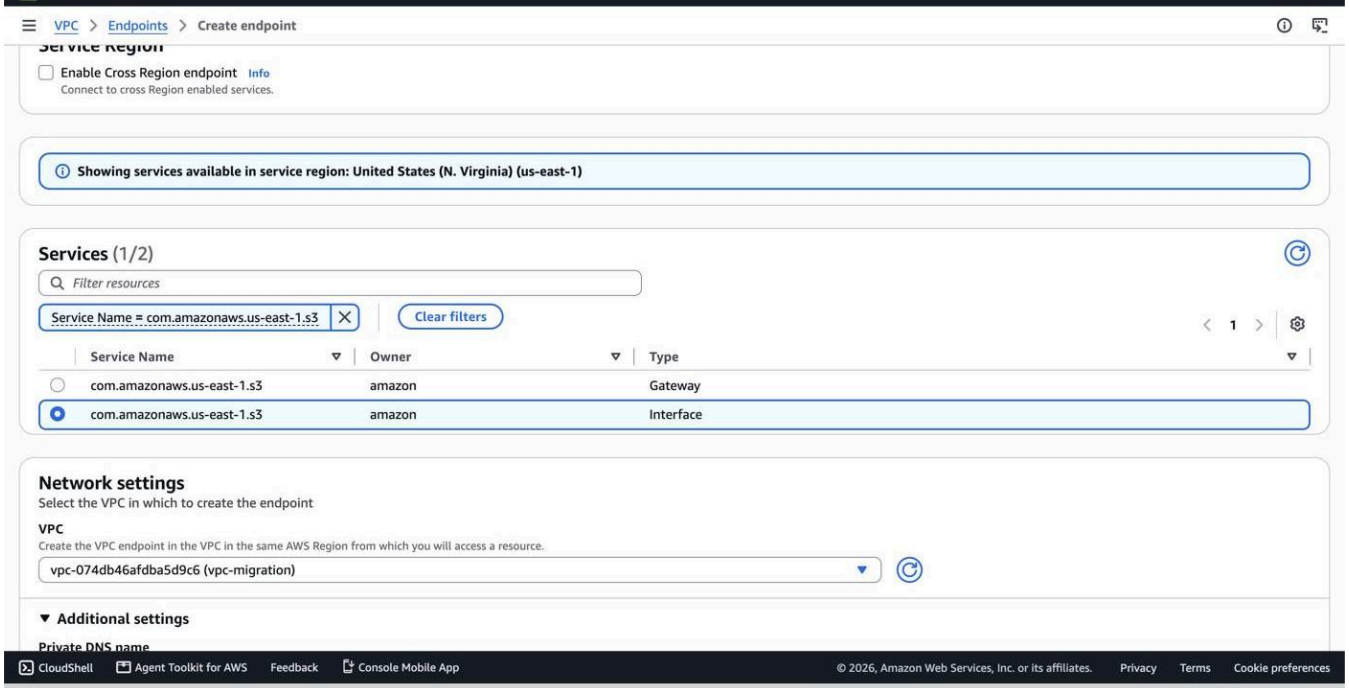
Enable copy logs	Yes
Log Analytics workspace	DefaultWorkspace-440b48d6-073f-4588-9499-24c49906a898-EUS (new)

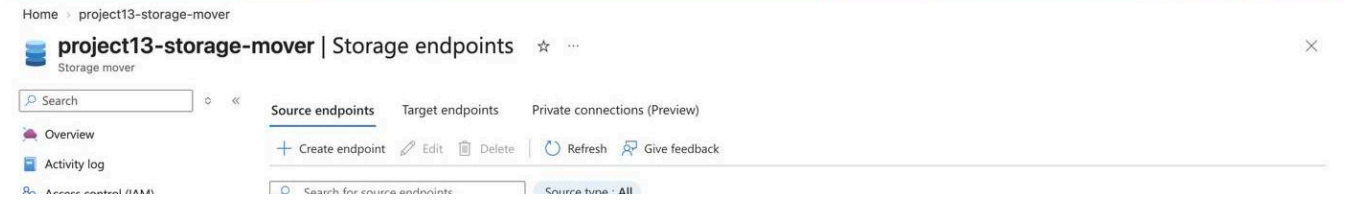


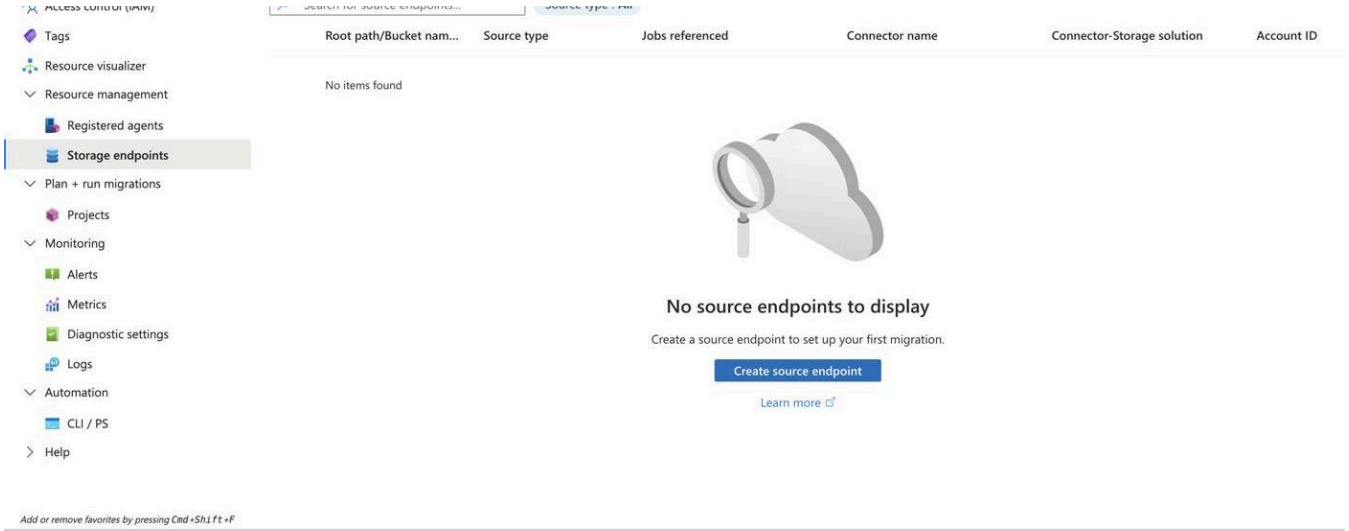


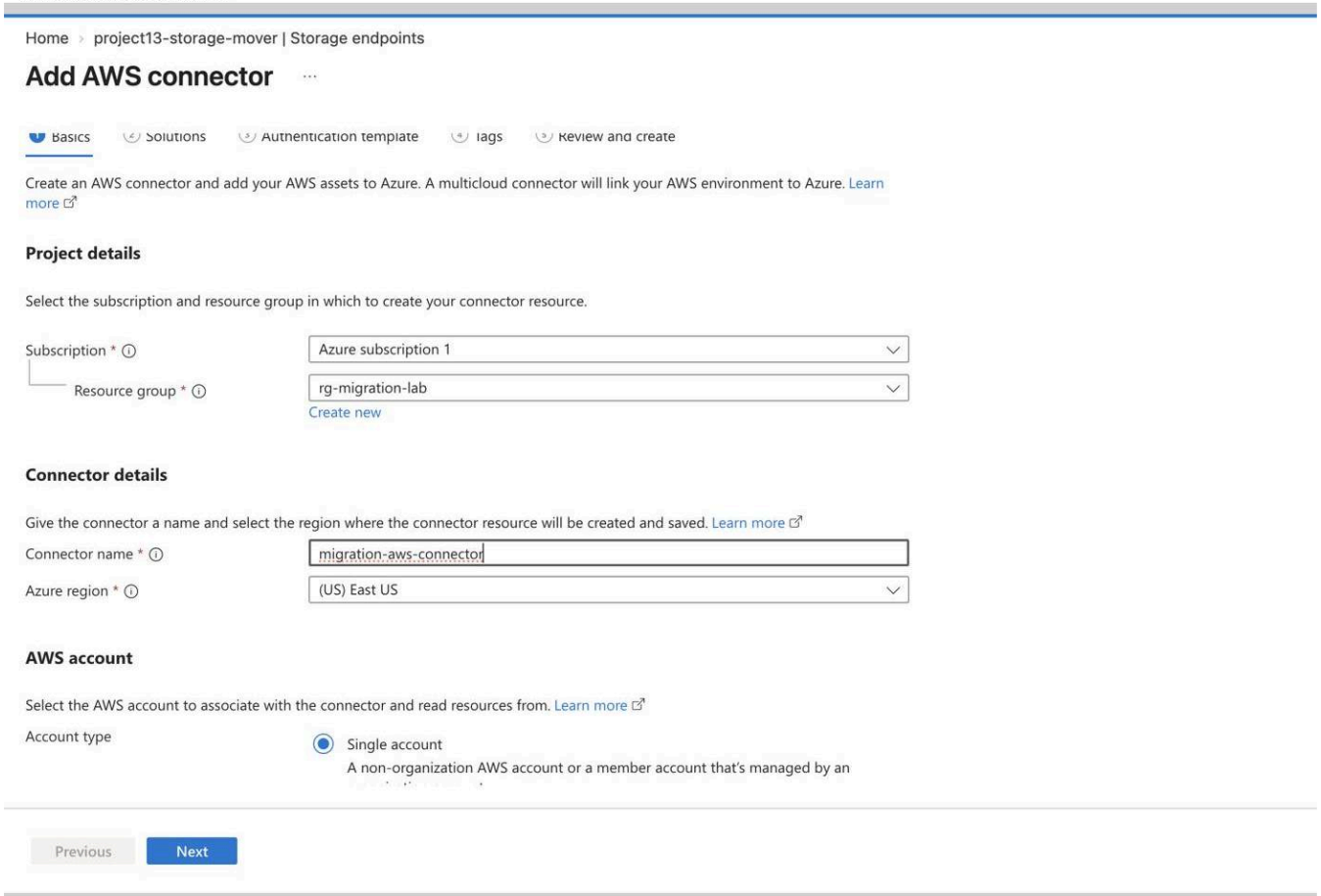

VPC Info
vpc-074db46afdba5d9c6 (vpc-migration)
Inbound rules Info
Type Info
Protocol Info
Port range Info
Source type Info
Source Info
Description - optional Info
Custom TCP
TCP
443
Anywhere-IPv4
0.0.0.0/0
0.0.0.0/0
Delete
Add rule
Rules with source of 0.0.0.0/0 or ::/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.

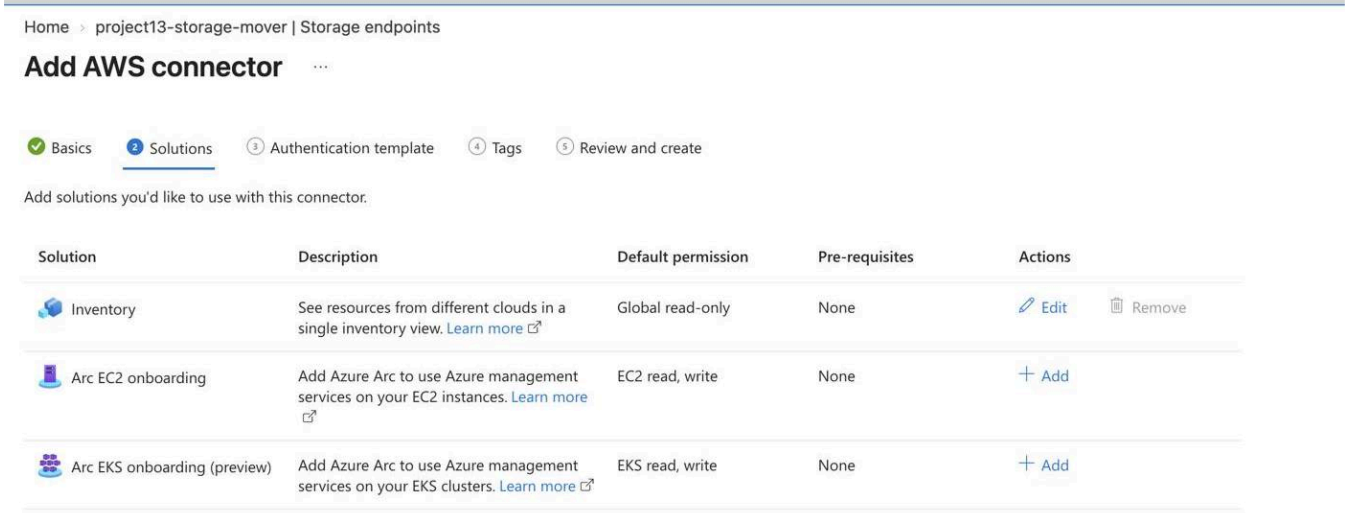

VPC > Endpoints
Managed prefix lists
NAT gateways
Peering connections
Route servers
Security
Network ACLs
Security groups
PrivateLink and Lattice
Getting started
Endpoints
Endpoint services
Service networks
Lattice services
Resource configurations
Resource gateways
Target groups
Domain verifications New
DNS firewall
Rule groups
Domain lists
Network Firewall
Firewalls
Successfully created VPC endpoint
vpce-0661ccdcd194d526c
Endpoints (1/1) Info
Find endpoints by attribute or tag
VPC endpoint ID : vpce-0661ccdcd194d526c
Clear filters
Actions
Create endpoint
Name
VPC endpoint ID
Endpoint type
Status
project13-s3-private-endpoint
vpce-0661ccdcd194d526c
Interface
Pending
vpce-0661ccdcd194d526c / project13-s3-private-endpoint
Details
Subnets
Security Groups
Notification
Policy
Monitoring
Tags
Endpoint ID
vpce-0661ccdcd194d526c
VPC ID
vpc-074db46afdba5d9c6 (vpc-migration)
DNS record IP type
ipv4
Private DNS preference
–
Status
Pending
Status message
–
IP address type
ipv4
Private DNS specified domains
–
Creation time
Sunday, October 11, 2026 at 15:01:46 GMT+5:30
Service name
com.amazonaws.us-east-1.s3
Service region
us-east-1
Endpoint type
Interface
Private DNS names enabled
No
DNS names
*.vpce-0661ccdcd194d526c-t5zpl96t.s3.us-east-1.vpce.amazonaws.com- (Z7HUB22UULQXV)
*.vpce-0661ccdcd194d526c-t5zpl96t-us-east-1b.s3.us-east-1.vpce.amazonaws.com- (Z7HUB22UULQXV)
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

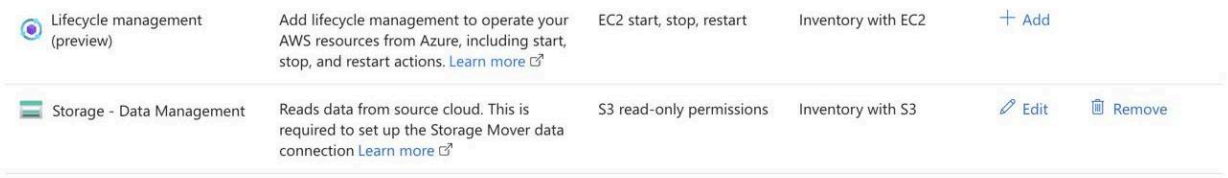

VPC > Endpoints > Create endpoint
Enable Cross Region endpoint Info
Connect to cross Region enabled services.
Showing services available in service region: United States (N. Virginia) (us-east-1)
Services (1/2)
Filter resources
Service Name = com.amazonaws.us-east-1.s3
Clear filters
Service Name
Owner
Type
com.amazonaws.us-east-1.s3
amazon
Gateway
com.amazonaws.us-east-1.s3
amazon
Interface
Network settings
Select the VPC in which to create the endpoint
VPC
Create the VPC endpoint in the VPC in the same AWS Region from which you will access a resource.
vpc-074db46afdba5d9c6 (vpc-migration)
Additional settings
Private DNS name
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Home > project13-storage-mover
project13-storage-mover | Storage endpoints
Storage mover
Search
Overview
Activity log
Source endpoints
Target endpoints
Private connections (Preview)
Create endpoint
Edit
Delete
Refresh
Give feedback


Tags
Resource visualizer
Resource management
Registered agents
Storage endpoints
Plan + run migrations
Projects
Monitoring
Alerts
Metrics
Diagnostic settings
Logs
Automation
CLI / PS
Help
Root path/Bucket nam...
Source type
Jobs referenced
Connector name
Connector-Storage solution
Account ID
No items found
No source endpoints to display
Create a source endpoint to set up your first migration.
Create source endpoint
Learn more
Add or remove favorites by pressing Cmd+Shift+F


Home › project13-storage-mover | Storage endpoints
Add AWS connector
Basics
Solutions
Authentication template
Tags
Review and create
Create an AWS connector and add your AWS assets to Azure. A multicloud connector will link your AWS environment to Azure. Learn more
Project details
Select the subscription and resource group in which to create your connector resource.
Subscription *
Azure subscription 1
Resource group *
rg-migration-lab
Create new
Connector details
Give the connector a name and select the region where the connector resource will be created and saved. Learn more
Connector name *
migration-aws-connector
Azure region *
(US) East US
AWS account
Select the AWS account to associate with the connector and read resources from. Learn more
Account type
Single account
A non-organization AWS account or a member account that's managed by an
Previous
Next


Home › project13-storage-mover | Storage endpoints
Add AWS connector
Basics
2 Solutions
3 Authentication template
4 Tags
5 Review and create
Add solutions you'd like to use with this connector.
Solution
Description
Default permission
Pre-requisites
Actions
Inventory
See resources from different clouds in a single inventory view. Learn more
Global read-only
None
Edit
Remove
Arc EC2 onboarding
Add Azure Arc to use Azure management services on your EC2 instances. Learn more
EC2 read, write
None
Add
Arc EKS onboarding (preview)
Add Azure Arc to use Azure management services on your EKS clusters. Learn more
EKS read, write
None
Add





Step 6 — Deploy the connector permissions with CloudFormation

1. The Azure wizard generates a CloudFormation template; download it.
2. AWS Console → CloudFormation → Create stack → Upload a template file → select it.
3. Stack name `project13-storage-mover-connector`; parameters prefilled from Azure.
4. Create stack and wait for every resource to reach CREATE_COMPLETE (IAM roles and the OIDC trust that let Azure read the bucket).
5. Back in Azure, review and create the connector. It shows up under Azure Arc → Multicloud connectors.

Step 7 — Endpoints and migration job

1. Source endpoint: the S3 bucket discovered through the connector.
2. Target endpoint: the Blob container.
3. Create the job definition, start it, and watch copy logs.
4. Validate object count and total size against S3.

The connector inventory sync takes time, so the RDS migration (3.13) was started in parallel.

3.13 RDS PostgreSQL → Azure Database for PostgreSQL

Confirmed source and target

	Source	Target
Service	AWS RDS `database-1`	Azure `project13-pg-target` (flexible server)
Engine	PostgreSQL 18.3	PostgreSQL 18.6
Size	db.t4g.micro, 20 GiB	Burstable B2s, 32 GiB
Location	us-east-1, private, not public	Central India, HA disabled
Port	5432	5432

Step 1 — Create the target server

1. Azure Database for PostgreSQL flexible servers → Create.
2. Resource group `rg-migration-lab`, server name `project13-pg-target`, region Central India.
3. Version matching the source major version, workload type Dev/Test, Burstable B2s, 32 GiB.
4. Networking: private access in `vnet-migration` so it can reach AWS over the Site-to-Site VPN.

Step 2 — Configure the migration

1. Open the target server → Migration → Create.
2. Source server type: Amazon RDS for PostgreSQL; migration option: validate and migrate; mode: offline (lab).
3. Source: RDS private endpoint and port 5432, reachable over the VPN (no public access needed).
4. Target: the flexible server; select `migration_db`.
5. Prerequisites: source reachable from target, SG on RDS allows 10.50.0.0/16 on 5432, migration user has the needed privileges.

Step 3 — Validate and run

1. Run validation; fix any connectivity or extension warnings.
2. Start migration and monitor status.
3. Compare table row counts and run application queries against the target.
4. Switch the application connection string (via Key Vault) during cutover.

3.14 Microsoft Entra ID → AWS single sign-on and SCIM user sync

Step 1 — Entra side

1. Microsoft Entra admin center → Default Directory → Enterprise applications → New application.
2. Browse the gallery → AWS application (AWS Single-Account Access shown in the gallery; AWS IAM Identity Center is the multi-account option).
3. Create, then open Single sign-on → SAML.

Step 2 — AWS side

1. IAM Identity Center → Applications → Add application.
2. Setup preference: "I have an application I want to set up"; application type SAML 2.0.

3. Exchange metadata: Entra federation metadata into AWS, AWS ACS URL and entity ID into Entra.

Step 3 — SCIM provisioning

1. In IAM Identity Center enable automatic provisioning → copy the SCIM endpoint and access token.
2. In Entra → the enterprise app → Provisioning → Automatic → paste endpoint and token → Test connection.
3. Assign users and groups in Entra; start provisioning. Users appear in IAM Identity Center automatically.
4. Map groups to permission sets and AWS accounts.

Image — CloudFormation connector stack, Azure PostgreSQL target and migration, Entra to…

Outcome: one identity source (Entra ID) controls AWS access; disabling a user in Entra removes their AWS access through SCIM.

4. Phase 3 — Enterprise AWS → GCP migration

A company running on AWS has decided to move to Google Cloud. Phase 3 assesses the AWS estate, maps dependencies, connects the clouds, migrates with Google's managed services, validates, cuts over and keeps a rollback path — while deploying only a handful of real resources.

Item	Value
Project type	Enterprise cloud migration and hybrid networking
Source	Amazon Web Services (us-east-1)
Target	Google Cloud Platform (us-central1)
Execution	AWS Management Console and Google Cloud Console only
Objective	Migrate an existing AWS environment to GCP using managed migration services wherever supported

Image — AWS to GCP migration architecture and flow

4.1 Enterprise-scale simulation

Resource	Simulated inventory	Real lab resources
EC2 instances	500	1 small pilot VM
Databases	50	1 small supported database
S3 buckets	100	1 source bucket
EBS volumes	As required	Minimal volume set
KMS keys	Multiple	1 or more test keys
Secrets	Multiple	1 test secret
VPCs and subnets	Multiple segments	Minimal source and target topology
VPN connections	High availability	1 pilot cross-cloud VPN
Application dependencies	Multiple services	A small representative app

The large numbers live in an inventory spreadsheet; only the pilot resources are deployed.

4.2 Mandatory rules

1. Use AWS Console and GCP Console for all creation, migration, verification and cleanup.
2. No AWS CLI, gcloud, Terraform CLI or scripts for infrastructure operations.
3. Use managed migration services wherever source and destination are supported.
4. No manual DB dumps or file-by-file copies as the primary method.
5. Build the AWS source before migrating it.
6. Build the target with GCP-native services.
7. Practise private IP preservation and alternatives for hardcoded-IP apps.
8. Use inventory manifests and migration waves to represent hundreds of workloads.
9. Keep costs low, data small, delete disposable resources promptly.
10. Document every phase, resource ID, configuration, validation, limitation and cleanup action.

4.3 Source and target components

AWS source (today's production)	GCP target (future production)
Account and region	Project and required APIs
VPC, public and private subnets	Custom-mode VPC and subnets

AWS source (today's production)	GCP target (future production)
Route tables, IGW, security groups	Routes and firewall rules
EC2 + EBS	Compute Engine VM + Persistent Disk
S3 bucket	Cloud Storage bucket
RDS (small dataset)	Cloud SQL or AlloyDB
KMS key, Secrets Manager secret	Cloud KMS, Secret Manager
IAM roles	IAM roles and service accounts
CloudWatch	Cloud Logging and Monitoring
Site-to-Site VPN	Cloud VPN (HA), Cloud Router, BGP
Route 53 / ALB	Cloud DNS / Cloud Load Balancing

4.4 Migration control process

AWS source inventory → compatibility assessment → GCP target design → connectivity → managed replication or transfer → test migration → final synchronization → cutover → validation → rollback readiness → cleanup.

4.5 AWS → GCP service mapping

AWS	GCP	Planned method
VPC	VPC network	Recreate equivalent design
Subnets	Subnets	Recreate with compatible CIDRs
Route tables	VPC routes	Recreate and validate
Security groups	VPC firewall rules	Translate inbound/outbound rules
EC2	Compute Engine	Migrate to Virtual Machines
EBS	Persistent Disk / Hyperdisk	VM workflow or disk migration
S3	Cloud Storage	Storage Transfer Service
RDS PostgreSQL	Cloud SQL for PostgreSQL / AlloyDB	Database Migration Service
RDS MySQL	Cloud SQL for MySQL	Database Migration Service

AWS	GCP	Planned method
AWS KMS	Cloud KMS	Create target keys, reconfigure encryption
Secrets Manager	Secret Manager	Recreate securely, repoint consumers
IAM roles/policies	GCP IAM	Map permissions to identities and roles
ALB/NLB	Cloud Load Balancing	Recreate behaviour
Route 53	Cloud DNS	Recreate records
CloudWatch	Cloud Monitoring + Logging	Rebuild dashboards, logs, alerts
EKS	GKE	Separate Kubernetes workstream

Service mappings are not automatic conversions. Engine versions, regions, permissions, networking, data sizes, replication modes and product limits must be checked first. Keys, IAM policies, firewall rules and networks are configuration migrations: their identities and semantics do not transfer unchanged.

4.6 Three ways enterprises migrate

Approach	What it means	Best for
1. Build a new destination	Create GCP network, VM, DB, storage, then move app and data	Replatforming / modernizing
2. Replicate existing workloads	Migration tools move compatible VMs, disks, databases	Lift-and-shift with minimal changes
3. Automate the destination	Terraform/IaC defines reusable networks, IAM, VMs	Repeated, enterprise-scale migrations

This lab follows **Approach 2**: migrate existing AWS workloads to GCP with migration tools.

4.7 What migrates vs what is configured separately

Component	Behaviour
OS, installed apps, files on migrated disks	Replicated with the VM disks
CPU and RAM	Chosen on the GCP target
VPC and subnets	Designed and built in GCP

Component	Behaviour
Security groups	Recreated as firewall rules
Route tables, NAT, load balancers	Configured as GCP equivalents
Private and public IPs	Planned and assigned in GCP
DNS, endpoints, secrets	Validated and updated
IAM roles and instance profiles	Mapped to service accounts and IAM roles

4.8 Phase 0 — Governance, accounts and cost controls

Objective: confirm source and target accounts, set naming, set budgets — before creating anything.

1. **Verify the AWS account:** sign in, record the account ID from the account menu, choose one source region (us-east-1) to keep the pilot simple.
2. **Verify the GCP project:** create a dedicated project `aws-gcp-migration-lab`, record project ID, number and billing account, confirm billing is enabled, check quotas and permissions.
3. **Naming convention:**

Resource	Name
AWS VPC	vpc-aws-gcp-migration
AWS EC2	ec2-migration-pilot
AWS S3	s3-migration-source (globally unique variant)
AWS database	rds-migration-pilot
GCP VPC	vpc-migration-target
GCP VM	vm-migration-pilot
GCP bucket	gcs-migration-target
GCP database	db-migration-target

4. **Budgets:** AWS Billing → Budgets → cost budget with actual and forecast alerts. GCP Billing → Budgets & alerts → threshold alerts. Budget alerts are not caps; usage is checked manually.

Done when: source account and target project confirmed, regions recorded, cost alerts on, naming documented, nothing unnecessary created.

4.9 Phase 1 — Build the AWS source network

Step 1 — VPC: VPC → Create VPC → VPC only → `vpc-aws-gcp-migration`, 10.60.0.0/16 (checked for overlap first), DNS resolution and hostnames on.

Step 2 — Subnets:

Subnet	CIDR	AZ
subnet-aws-migration-app	10.60.1.0/24	us-east-1a
subnet-aws-migration-db-a	10.60.10.0/24	us-east-1a
subnet-aws-migration-db-b	10.60.11.0/24	us-east-1b

Step 3 — Routing: create `rt-aws-migration-app` (associate the app subnet) and `rt-aws-migration-db` (associate both DB subnets, local route only). No NAT Gateway — it bills hourly plus data processing.

Step 4 — Security groups:

- `aws-migration-app`: only required app and admin access.
- `aws-migration-db`: database port only from `aws-migration-app` or intended private sources.
- No "all ports" or unrestricted admin access.

Step 5 — Verify and record: VPC ID and CIDR, subnet IDs and CIDRs, route table IDs, SG IDs, AZs, internet/private routing decisions.

Image — AWS source VPC, subnets, route tables, security groups and IAM role

4.10 Phase 2 — Create the AWS source workloads

Step 1 — IAM role for EC2: IAM → Roles → Create → trusted entity EC2 → attach Session Manager access and only what the pilot needs.

Step 2 — EC2 pilot:

1. EC2 → Launch instances → `ec2-migration-pilot`.
2. Amazon Linux 2023 (this choice matters later — see 4.13), small instance type.
3. App subnet, SG `aws-migration-app`, encrypted minimal EBS root volume, tag `migration-lab`.
4. No public IPv4 unless the admin access method needs it.

Step 3 — EBS data disk: EC2 → Volumes → small encrypted volume in the same AZ → attach to the pilot → record its ID.

Step 4 — S3 source bucket: S3 → Create bucket → globally unique name, us-east-1, Block Public Access on, encryption on, small non-sensitive test dataset; record object names, sizes, metadata and permissions.

Step 5 — DB subnet group: RDS → Subnet groups → both DB subnets (two AZs).

Image — EC2 pilot, EBS volume, S3 bucket, DB subnet group

Step 6 — RDS source database:

1. RDS → Create database → Full configuration → PostgreSQL (checked first that Cloud SQL and DMS support it).
2. Free tier template, single-AZ, identifier `rds-migration-pilot`.
3. Credentials managed in AWS Secrets Manager (no password typed into documents).
4. Small burstable class, minimal gp storage.
5. VPC `vpc-aws-gcp-migration`, the DB subnet group, Public access = No, SG `aws-migration-db`.
6. Encryption on. Create, then load a small test schema and dataset.

Result: RDS available with private IP 10.60.11.73 inside 10.60.0.0/16.

Image — RDS configuration and created database

Step 7 — KMS key and secret:

1. KMS → Create key → symmetric, encrypt and decrypt → set key administrators and users → record the key ARN.
2. Secrets Manager → Store a new secret → *Other type of secret* → key/value with a non-sensitive lab value → encrypt with the KMS key → name `aws-gcp-migration/lab-demo` → rotation off for the lab.

Step 8 — Logs and monitoring: a logs bucket for migration evidence; CloudWatch log group with retention set; a CPU metric alarm on the pilot instance.

Step 9 — Record dependencies:

```
EC2 → EBS
EC2 → database
EC2 → secret
S3  → application
Database → subnet group + security group
Encrypted resources → KMS key
```

Image — KMS key, secret, logs bucket and CloudWatch

Done when: one EC2 running and verified, EBS attached, S3 objects present, DB reachable on its private path, secret and encryption recorded, IDs and dependencies documented.

4.11 Phase 3 — Source discovery and migration readiness

Objective: assess the source estate and find what managed services can migrate.

Step 1 — Inventory the pilot resources from EC2 (instances, volumes), S3, RDS, KMS, Secrets Manager, VPC, IAM and CloudWatch. For each resource record:

Field	Example for the EC2 pilot
Source identifier	ec2-migration-pilot
Service type	EC2
Region / AZ	us-east-1 / us-east-1a
Size and configuration	Small instance, Amazon Linux 2023, encrypted EBS
Encryption	EBS encrypted
Network dependencies	App subnet, SG aws-migration-app
Application dependencies	RDS, secret, S3
Migration eligibility	Check Migrate to VMs OS support
Target service	Compute Engine
Estimated downtime	Cutover window
Migration wave	Wave 0 (pilot)
Validation criteria	Boots, app starts, DB reachable

Step 2 — Enterprise simulation inventory: a browser spreadsheet with 500 EC2, 50 DB and 100 S3 records plus volumes, keys, secrets, network and app dependencies. Only pilot rows are real; the rest are marked *simulated*.

Step 3 — Identify blockers:

- Unsupported OS or DB versions
- Engine compatibility
- Encryption and key dependencies
- IP overlap between AWS and GCP
- Hardcoded IP addresses
- Unsupported instance or storage configurations
- Downtime requirements, replication bandwidth, service quotas

Done when: pilot inventory complete, synthetic inventory ready, blockers listed, target service and method chosen per pilot resource.

4.12 Phase 4 — GCP destination foundation

Step 1 — Project and APIs

1. Google Cloud → New project → `aws-gcp-migration-lab`, linked to the billing account.
2. APIs & Services → Library → enable only what is needed: Compute Engine, Cloud Storage, Logging/Monitoring, Cloud KMS, Secret Manager, Database Migration Service, Storage Transfer Service, VM Migration (Migrate to VMs), Cloud VPN/Router. Others are enabled when their phase starts.

Step 2 — IAM: review project admins, use dedicated service accounts for migration services, grant only workflow roles, no broad Owner/Editor, document which identity runs each operation.

Step 3 — Custom-mode VPC

1. VPC network → Create VPC network → name `aws-gcp-migration-vpc`, subnet creation mode **Custom**.
2. Add subnets in us-central1 inside 10.70.0.0/16 (no overlap with AWS 10.60.0.0/16 or Azure 10.50.0.0/16).
3. Private Google Access on, so VMs reach Google APIs without external IPs.
4. Create.

A GCP VPC is **global** and its subnets are **regional** — unlike AWS, where a VPC is regional and subnets are zonal. The design reflects that.

Image — GCP project, Compute Engine API and custom VPC with subnets

Step 4 — Firewall rules

1. Network Security → Firewall policies → Create firewall rule.
2. Direction ingress, action allow, targets by network tag (or service account).
3. Source ranges limited to what is needed (for example the AWS CIDR 10.60.0.0/16 for migration traffic, IAP range for admin SSH).
4. Only required protocols and ports; logging on where useful.
5. No SSH, RDP or database ports open to 0.0.0.0/0.

Step 5 — Outbound access: Private Google Access where suitable; Cloud NAT only if a workload truly needs internet egress.

Step 6 — Target VM baseline and bucket

1. Compute Engine → Create instance `gcp-migration-app-vm`, small machine type, us-central1, in the custom subnet, no external IP.
2. Cloud Storage → Create bucket → unique name, location, uniform bucket-level access, public access prevention, encryption choice.

Image — GCP firewall rules and VM creation

Done when: project and APIs ready, IAM documented, VPC and subnets built, firewall verified, bucket ready, CIDRs compatible with AWS.

4.13 Phase 5 — AWS ↔ GCP private connectivity (HA VPN + BGP)

The database migration needed a private path: Database Migration Service offers IP allow-list, reverse-SSH, VPC peering or PSC options, and for a private RDS the right choice is a Site-to-Site IPsec VPN.

```
AWS RDS PostgreSQL   private IP 10.60.11.73  (AWS VPC 10.60.0.0/16)
        ⇅
Site-to-Site IPsec VPN  (encrypted tunnels + BGP routes + firewall rules)
        ⇅
Google Cloud VPC 10.70.0.0/16  → Database Migration Service → Cloud SQL
```

Step 1 — IP and BGP plan

Item	Value
AWS VPC CIDR	10.60.0.0/16
GCP VPC CIDR	10.70.0.0/16
GCP Cloud Router ASN	Private ASN (e.g. 65xxx)
AWS VGW ASN	Amazon default
Ports needed	PostgreSQL 5432 from GCP migration range to RDS

Overlapping CIDRs are never connected without a tested translation design.

Step 2 — GCP HA VPN gateway

1. Network Connectivity → VPN → Create VPN → **High-availability (HA) VPN** (two interfaces, 99.99% SLA option).
2. Name `aws-gcp-ha-vpn`, network `aws-gcp-migration-vpc`, region us-central1, IPv4.
3. Create → note the two gateway interface IPs.

Step 3 — AWS side

1. VPC → Virtual private gateways → Create `aws-gcp-migration-vgw` → attach to the VPC.
2. Customer gateway(s) using the GCP HA VPN interface IPs.
3. Site-to-Site VPN connection → target VGW → routing **Dynamic (BGP)** → create → download configuration.
4. Pre-shared keys stay out of documents and chat.

Step 4 — GCP peer VPN gateway (represents AWS)

1. Add peer VPN gateway `aws-migration-vpn` → **two interfaces** → AWS tunnel outside IPs.

Step 5 — Cloud Router

1. Create router `aws-gcp-router` in the VPC and region, with the chosen private ASN.
2. Advertise all subnets visible to the router (default).

Step 6 — VPN tunnels

1. Add tunnel `aws-vpn-tunnel-1` (and `-2`): peer interface, IKEv2, pre-shared key from the AWS config.
2. The console warns the key cannot be retrieved later — store it in a secret manager, not in notes.

Step 7 — BGP sessions

1. For each tunnel: Configure BGP session → peer ASN = AWS VGW ASN.
2. Cloud Router BGP IP and peer BGP IP = the inside tunnel addresses from the AWS config.
3. Save and finish. Summary shows tunnels and BGP sessions.

Image — DMS connectivity choice, VPN design, HA VPN, AWS VGW, peer gateway, Cloud Route…

Step 8 — Routes and firewall

1. Confirm GCP learns 10.60.0.0/16 and AWS learns 10.70.0.0/16 over BGP.
2. AWS route tables: enable route propagation from the VGW.
3. GCP firewall: allow only migration traffic from 10.60.0.0/16; AWS SG `aws-migration-db` : allow 5432 from the GCP migration range only.

Step 9 — Test and record

Record	Where
Tunnel status	AWS VPN tunnel details · GCP VPN tunnels
BGP status	GCP Cloud Router · AWS tunnel details
Advertised / learned routes	Cloud Router routes · AWS route tables
Reachability	DMS connectivity test to 10.60.11.73:5432
Failover	Second tunnel behaviour
Cost	HA VPN and AWS VPN bill per hour per tunnel

Done when: tunnels established, expected routes present, required ports allowed, cross-cloud reachability validated, VPN cleanup steps written. If a managed transfer (like Storage Transfer from S3) does not need private connectivity, it does not have to use the VPN.

4.14 Phase 6 — EC2 → Compute Engine with Migrate to Virtual Machines

Objective: migrate the pilot EC2 instance to Compute Engine through Google's managed VM migration workflow.

Step 1 — Enable the service

1. Search "Migrate to Virtual Machines" → enable the **VM Migration API**.

2. Open Compute Engine → Migrate to Virtual Machines → Dashboard → Sources.

Step 2 — Give Google read access to AWS

1. AWS IAM → Policies → Create a policy (for example `GCPMigrateToVMsAccess`) with the EC2 describe, snapshot and related permissions listed in Google's documentation.
2. IAM → Users → create a dedicated user, attach the policy directly.
3. Security credentials → Create access key → use case *Third-party service*. The access key ID and secret go only into the Google form, never into notes.

Step 3 — Add the AWS source

1. Sources → Add source → **Add AWS source**.
2. Name `aws-migration-source`, GCP region us-central1, AWS region us-east-1, access key credentials.
3. Optional filter by tags or security groups to limit discovery.
4. Create. Status moves from *Pending* to *Active* and the source VMs appear.

Image — Migrate to VMs setup, AWS source and IAM policy

Step 4 — Start migration and set target details

1. Select `ec2-migration-pilot` → Migrate.
2. Target details: project `aws-gcp-migration-lab`, zone in us-central1, machine type, network `aws-gcp-migration-vpc`, subnet, internal IP choice, no external IP, target service account.
3. Replication starts: first sync of the disks.

Step 5 — The failure: OS adaptation

```
Failed to adapt the operating system of VM ec2-migration-pilot:
Unsupported operating system: amazonlinux2023.12
```

What it meant:

- This was an **OS compatibility issue, not a networking issue** — discovery and replication had worked.
- Google's supported-OS list does not include Amazon Linux 2023 for this workflow (the documented experimental conversion is for Amazon Linux 2).
- Rule followed: **do not click Cut-over and do not delete the AWS source**. Keep the source until a supported path is chosen.

Image — Access key, source discovery, target details and the OS adaptation failure

Options evaluated

Option	What it is	When to use
1. Request OS support	Contact the Migrate to VMs team (m2vm-os-support-request@google.com) about the exact AL2023 release	Production estate that must keep its OS
2. Disk migration and boot test	Move disks to Persistent Disk with the VM disk migration workflow, then try to boot	Proof of concept; boot not guaranteed
3. Rebuild on a supported OS	New supported VM, move app data and config	Safer fallback when direct boot fails
4. Replace the lab pilot	Create a new AWS pilot on Ubuntu 22.04 LTS and migrate that	Lab VM with no important data — chosen

How to decide in production

Situation	Can you migrate?
OS supported by Migrate to VMs	Usually yes, through the normal workflow
OS unsupported, disk migration available	Possibly; boot compatibility must be tested
OS conversion supported	Yes if conversion succeeds; the OS may change
Neither OS nor disk-boot path supported	Not directly; rebuild or use another method

Step 6 — New pilot on Ubuntu

1. AWS → launch `ec2-migration-ubuntu-pilot` (Ubuntu, same subnet and SG).
2. Back in Migrate to VMs it appears in the source list.
3. First attempt failed with *"VM is not in ready state"* — the instance was still initialising. After it reached *Running* with status checks passed, replication started.
4. Target details set as before, network interface in `aws-gcp-migration-vpc`.

Step 7 — Replicate and test-clone

1. Replication status: *Active (Idle)* after first sync.

2. Cut-over and Test-clone menu → **Test-Clone**.
3. Test-clone history: *Preparing VM disks* → *Instantiating migrated VM* → **Succeeded**.
4. Compute Engine shows `ec2-migration-ubuntu-pilot-gcp` *Running* in us-central1 — the migrated VM boots in GCP.

Image — Ubuntu pilot replication and successful test clone

Step 8 — Validate the clone: boot, disks, application startup, logs, dependencies (DB, secret, bucket). Document any issue and fix.

Step 9 — Cutover (only after a passing test):

1. Approve the maintenance window.
2. Stop or quiesce writes on the source.
3. Let final sync complete.
4. Run the Cut-over action.
5. Validate the GCP workload.
6. Keep the AWS source only as long as the rollback plan needs.

Never run two active copies with the same IP, or let both write to the same dataset.

Done when: pilot discovered and replicated, test clone validated, target VM boots, app and disk checks pass, cutover and rollback documented.

4.15 Phase 7 — RDS PostgreSQL → Cloud SQL with Database Migration Service

Objective: move the pilot database with a managed migration job, not a manual dump and restore.

Step 1 — Choose the destination. For RDS PostgreSQL the target is Cloud SQL for PostgreSQL (AlloyDB is the alternative). Confirm source and target versions, supported path, extensions and collation, size, downtime limit, networking, authentication and encryption.

Step 2 — Enable and start DMS

1. Database Migration → Get started → choose source **PostgreSQL** and destination Cloud SQL.
2. Enable the **Database Migration API** when prompted.
3. Create a migration job: name, source engine Amazon RDS for PostgreSQL, destination Cloud SQL for PostgreSQL, region us-central1, migration type *Continuous* (initial load + change replication).

Step 3 — Prepare the source

1. Review DMS source requirements (logical replication settings on RDS via a parameter group, replication-capable user).
2. Connection profile: RDS private IP 10.60.11.73, port 5432, credentials from the secret.
3. Reachability: private, over the HA VPN from Phase 5.
4. SG `aws-migration-db`: allow 5432 from the GCP range.

Step 4 — Prepare the target: Cloud SQL instance with private IP in the VPC, minimal capacity, encryption, compatible version.

Step 5 — Connectivity method: of the four DMS options (IP allow-list, reverse-SSH tunnel, VPC peering, PSC interface), the private VPN path with VPC peering/private IP was used, so RDS stays non-public.

Step 6 — Run

1. Test the job (connectivity and compatibility checks).
2. Start; monitor full dump, then CDC and replication lag.

Step 7 — Validate and cut over

1. Compare objects and row counts.
2. Run application queries.
3. Review errors, unsupported objects, permissions.
4. Quiesce source writes, let lag reach zero, **Promote** the Cloud SQL instance.
5. Point the app to Cloud SQL (secret update) and verify.

Done when: connection profiles valid, job reaches the expected state, data checks pass, app connects, cutover and rollback criteria recorded.

4.16 Phase 8 — S3 → Cloud Storage with Storage Transfer Service

Step 1 — Source: confirm the bucket and object inventory, check encryption, policies, metadata, versioning. Create least-privilege access for the transfer (IAM user or role with `s3:ListBucket` and `s3:GetObject` on that bucket only). Bucket stays private.

Step 2 — Target: Cloud Storage bucket, location, permissions for the transfer service agent, encryption.

Step 3 — Transfer job

1. Storage Transfer → Create transfer job → source **Amazon S3**, destination Cloud Storage.
2. Bucket name, access credentials, destination bucket, run once (batch).
3. Start.

Step 4 — First run failed

```
Status: Failed
AccessDenied: ... is not authorized to perform: s3:ListBucket on resource:
arn:aws:s3:::<source-bucket>
```

Cause: the AWS identity used by the job lacked `s3:ListBucket` on the source bucket.

Step 5 — Fix and re-run: added `s3:ListBucket` (bucket ARN) and `s3:GetObject` (bucket/*) to the policy, then re-ran the job. Result: **Success, 100%**, no errors.

Step 6 — Validate: job report, object counts and sizes on both sides, selected checksums, metadata and permissions, encryption, app read/write test.

Image — Database Migration Service setup and Storage Transfer Service failure → success

Check	Result
First run	Failed — AccessDenied s3:ListBucket
Fix	Least-privilege policy with ListBucket + GetObject
Second run	Success, 100%

Done when: job succeeds, counts and sizes validated, permissions correct, unsupported metadata documented.

4.17 Phase 9 — KMS, secrets and IAM

Step 1 — Identity mapping document

AWS	GCP
IAM role for EC2 (instance profile)	Service account attached to the VM
IAM policies	Predefined or custom IAM roles
Service roles for migration	Dedicated migration service accounts
Application identities	Workload service accounts

Only required permissions are granted.

Step 2 — Destination keys: Security → Key Management → create key ring and key in the right location → permissions → record the resource name → test encrypt/decrypt with dummy data. An AWS KMS ARN cannot be used in GCP.

Step 3 — Secrets: Security → Secret Manager → create the secret → transfer the lab value securely → grant access to the app's service account → repoint the app → test retrieval → set rotation if needed.

Step 4 — Validate security: apps read only their own secrets, encrypted resources use the new key, least privilege verified, temporary migration permissions (like the Migrate to VMs access key) removed.

Done when: identities work, encryption tested, secret access validated, no dependence on source-only credentials.

4.18 Phase 10 — Private IP preservation and application networking

Step 1 — Record address dependencies: EC2 and DB private IPs, hardcoded IPs in config, firewall allow-lists, DNS records, third-party allow-lists, source and target CIDRs.

Step 2 — Can the IP be kept? Example: the EC2 uses 10.60.1.249, but the GCP subnet is 10.70.1.0/24 — the address is outside the subnet, so the VM cannot simply take it. And the same IP must never be active on both sides of a connected network.

Step 3 — Pick a strategy

Strategy	How	Notes
A. Preserve the address	Move that CIDR to GCP, shut the source, allocate the same IP	Careful subnet planning, no overlap while connected
B. Routing or NAT	Translate or route through an intermediary	Check return routing and source-IP needs
C. DNS / stable endpoint	Replace IPs with DNS names or a load-balanced endpoint	Recommended long-term

Public IPs: an AWS Elastic IP cannot move to GCP. Assign a GCP static external IP and update DNS, allow-lists and partners; keeping the exact public IP needs a specialised traffic-migration design.

Step 4 — Test the cutover: test in isolation or on a test clone, check routes and firewalls, confirm dependents reach the target, switch traffic, check health and logs, keep a rollback path.

4.19 Phase 11 — Enterprise-scale simulation (500 VMs, 50 DBs, 100 buckets)

Step 1 — Inventory workbook sheets: EC2, databases, S3, EBS, KMS keys, secrets, network dependencies, application dependencies, migration waves, validation results, issues and risks.

Step 2 — Migration waves

Wave	Content	Why this order
0	Pilot and non-production	Prove the tooling and runbook
1	Low-risk stateless apps	Easy rollback, no data sync
2	Storage and app dependencies	Data lands before the apps that need it
3	Databases and stateful workloads	Needs replication and cutover windows
4	High-dependency, business-critical	Last, with the most rehearsed process

Waves are illustrative; real order follows dependencies and downtime limits.

Step 3 — Readiness per workload: compatibility, dependencies, data volume, RTO/RPO, downtime allowance, IP needs, security/encryption, target capacity, validation owner, rollback trigger.

Step 4 — Simulated failures and decisions

Failure	Example response
Unsupported DB version	Upgrade source first or defer to a later wave
Failed source connection	Check VPN, routes, SG; retry
Insufficient target capacity	Request quota, change machine type
Missing IAM permissions	Add least-privilege permission (as with s3:ListBucket)
VPN or route failure	Fail over to second tunnel, fix BGP
Replication lag	Delay cutover until lag is near zero
Object transfer errors	Fix access, re-run job
App startup failure	Fix config on test clone, re-test
Failed post-cutover validation	Roll back per runbook
Unsupported OS (seen for real)	Rebuild or replace on a supported OS

4.20 Phase 12 — Cutover, validation and rollback

Before: all jobs healthy, capacity and connectivity confirmed, config and secrets validated, final sync known, maintenance window agreed, go/no-go criteria written, rollback ready.

During:

1. Quiesce writes if required.
2. Final synchronization.
3. Service cutover action (VM cut-over, DMS promote).
4. Update routing, DNS or endpoints.
5. Check health and data consistency.

After: functional tests, logs/metrics/errors, security and access, DB writes and object access, monitoring, results and sign-off.

Rollback (trigger defined before cutover):

1. Stop or isolate the failed target.
2. Check whether the target accepted writes.
3. Reconcile data before sending traffic back.
4. Restore routing/endpoints to the source.
5. Verify the app.

6. Document cause and fix.

Never switch back automatically after the target accepted writes without checking data consistency.

4.21 Phase 13 — Cost cleanup and closure

AWS cleanup (console)

1. Stop or terminate pilot EC2 instances.
2. Delete unneeded EBS volumes and snapshots.
3. Delete the lab RDS and its disposable resources.
4. Delete the S3 buckets once objects are confirmed disposable.
5. Remove temporary secrets and keys (respect KMS deletion waiting periods).
6. Delete VPN connections, customer gateways and virtual private gateways.
7. Release Elastic IPs and unused public IPv4 addresses.
8. Remove NAT Gateways and load balancers if any.
9. Remove lab networking after dependencies.
10. Delete the lab VPC without touching unrelated resources.

GCP cleanup (console)

1. Stop or delete test and migrated VMs.
2. Review disks, snapshots, images.
3. Delete the Cloud SQL instance.
4. Delete test objects and buckets.
5. Schedule KMS key deletion only when safe.
6. Delete migration jobs and the Migrate to VMs source; delete the AWS access key used by it.
7. Delete VPN tunnels, HA VPN gateway, peer gateway and Cloud Router.
8. Release static IPs.
9. Review Cloud NAT, load balancers, log storage.
10. Remove the lab VPC last.

Billing check: AWS Billing by service and region, GCP Billing by project and service, look for leftover disks, snapshots, logs, objects, IPs and gateways, re-check after billing data refreshes.

4.22 Phase 3 status

Area	Status
AWS source build (VPC, EC2, EBS, S3, RDS, KMS, secret, logs)	Done
GCP foundation (project, APIs, VPC, firewall, VM)	Done
Migrate to VMs — Amazon Linux 2023 pilot	Blocked: unsupported OS (documented)
Migrate to VMs — Ubuntu pilot	Replicating, test clone succeeded
HA VPN + BGP for DMS	Built (tunnels and BGP sessions configured)
Database Migration Service	Job set up over the private path
Storage Transfer Service	Failed once (permissions), then 100% success
Enterprise waves, cutover, cleanup	Documented runbook

Tech stack and learnings

Stack: AWS (VPC, EC2, RDS PostgreSQL, S3, Client VPN, Site-to-Site VPN, ACM, KMS, Secrets Manager) · Azure (VNet, NSG, Route Tables, VPN Gateway) · GCP (Compute Engine, Cloud SQL, Cloud Storage, Cloud VPN, Cloud Router, Migrate to VMs, DMS, Storage Transfer Service) · Terraform · OpenSSL / Easy-RSA.

Learnings

- Plan non-overlapping CIDRs across every cloud and site before building anything.
- Console first to understand; Terraform import to manage without recreating.
- Certificate-based VPN gives per-user access and clean revocation.
- Managed migration tools still need OS, engine and region compatibility checks.
- Build small, think enterprise: minimum real resources, documented at production scale, destroyed after.

LinkedIn post

I built a multi-cloud migration lab from scratch — AWS, Azure, GCP and on-prem — on a tiny budget. ☁️

The rule: build small infrastructure, but think like an enterprise migration team.

🔹 Phase 1 — AWS ↔ Azure
Built the Azure network in the console (VNet, subnets, NSGs, route tables), linked it to AWS over Site-to-Site VPN, then imported everything into modular Terraform: 13 resources imported, 0 recreated.

🔹 Phase 2 — On-prem → AWS
Set up AWS Client VPN with my own Easy-RSA certificate authority and mutual TLS. My laptop now reaches private EC2 and RDS — and adding a new user is just signing a new cert.

🔹 Phase 3 — AWS → GCP
Mapped AWS services to GCP and used managed tools: Migrate to VMs, Database Migration Service and Storage Transfer Service, over a private Cloud VPN with BGP. Planned scale as 5 migration waves for a simulated 500-VM estate.

💡 Biggest lesson: migration is not copying. Amazon Linux 2023 failed OS adaptation in Migrate to VMs — so I had to choose between a supported OS, disk-based boot or a rebuild. I rebuilt the pilot on Ubuntu and the test clone booted successfully on Compute Engine. Keys, IAM, firewalls and IPs get redesigned, not moved.

Other takeaways:
✅ Plan non-overlapping CIDRs first
✅ Console to learn, Terraform import to manage
✅ Always keep a rollback path
✅ Destroy the lab and check billing

📄 Full step-by-step documentation (screenshots, commands, configs): https://github.com/ArpitChouksey/PersonalProject/tree/P13-CloudMigration/Documents

Next up: AWS ↔ real on-prem with Site-to-Site VPN.

#DevOps #CloudMigration #AWS #Azure #GCP #Terraform #SRE #MultiCloud #Networking